University of Amsterdam

Masters Thesis

Your Thesis Title

Author: Nitai Nijholt
First name SURNAME

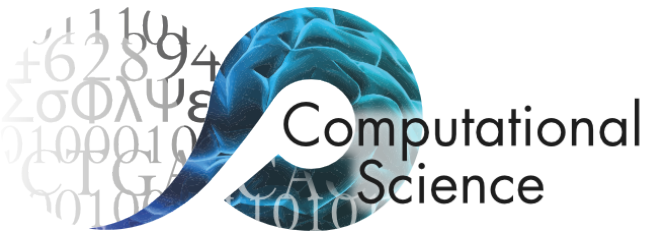
Examiner: dr. Debraj Roy
examiner name
Supervisor: dr. Rick Quax
supervisor name
Assessor: dr. Caspar van Lissa
assessor name

A thesis submitted in partial fulfilment of the requirements for the degree of Master of Science in Computational Science

in the

Computational Science Lab
Informatics Institute

January 2026



Declaration of Authorship

I, First name SURNAME, declare that this thesis, entitled 'Your Thesis Title' and the work presented in it are my own. I confirm that:

- This work was done wholly or mainly while in candidature for a research degree at the University of Amsterdam.
- Where any part of this thesis has previously been submitted for a degree or any other qualification at this University or any other institution, this has been clearly stated.
- Where I have consulted the published work of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this thesis is entirely my own work.
- I have acknowledged all main sources of help.
- Where the thesis is based on work done by myself jointly with others, I have made clear exactly what was done by others and what I have contributed myself.

Signed:

Your signature

Date: 1 August 2025

"What I cannot create, I do not understand."

Richard P. Feynman

UNIVERSITY OF AMSTERDAM

Abstract

Faculty of Science
Informatics Institute

Master of Science in Computational Science

Your Thesis Title

by First name SURNAME

We propose a multi-agent LLM research system for generating initial Causal Loop Di agrams (CLDs), harnessing LLMs as low-cost information retrievers to addresses the knowledge synthesis challenges due to rapidly expanding research literature. Using gen erated initial, literature-grounded CLDs, domain experts can focus on adding specialized knowledge rather than reiterating consensus during model building sessions. To address LLM hallucinations threatening CLD accuracy, we integrate a context-sensitive veri f ication approach (LLM-as-a-judge) with context-insensitive metrics (cosine similarity, perplexity), investigating whether applying these techniques—first separately, and then uniquely, in combination—can mitigate hallucinations, quantify uncertainty and increase f inal CLD accuracy more efficiently than regular test-time compute.

Length 200-400 words.

Acknowledgements

Thank the people that have helped, supervisors family etc.

Contents

Declaration of Authorship i

Abstract iii

Acknowledgements iv

Contents v

List of Figures xiii

List of Tables xiv

Abbreviations xv

1 Introduction 1

2 Literature review 4

2.1 Evolution of Sequential Modeling Paradigms 5
2.2 Self-Attention and Multi-Head Attention Mechanisms 6
2.2.1 Scaled Dot-Product Attention 6
2.2.2 Multi-Head Attention Architecture 6
2.3 Architectural Components and Their Impact on Model Behavior 7
2.3.1 Layer Normalization and Stability 7
2.3.2 Feed-Forward Networks and Token-Level Processing 7
2.4 Positional Encoding Schemes and Sequential Understanding 8
2.4.1 Absolute vs. Relative Position Embeddings 8
2.5 Training Paradigms and Generation Objectives 8
2.5.1 Causal Language Modeling 8
2.5.2 Generation Strategies and Their Implications 9
2.6 Memory and Computational Optimizations 9
2.6.1 Attention Complexity and FlashAttention 9
2.7 Theoretical Implications for Hallucination and Generation Quality 10
2.8 Context Vector Computation in Sequence-to-Sequence Models 11
2.9 Defenition of Hallucination 20
2.10 Test time compute 20

3 Hallucination Detection and Test-Time Compute: Advanced Techniques for LLM Reliability **21**
3.1 Hallucination Taxonomy and Detection 21
3.1.1 Introduction to LLM Hallucinations 21
3.1.2 Formal Taxonomy of Hallucinations 22
3.1.2.1 Core Distinctions: Intrinsic vs. Extrinsic Hallucinations . 22
3.1.2.2 Factuality vs. Faithfulness 22
3.1.2.3 Specific Manifestations of Hallucinations 23
3.1.3 Causes and Contributing Factors 24
3.1.3.1 Training Objective Misalignment 24
3.1.3.2 Data Quality and Coverage 24
3.1.3.3 Sampling and Decoding Strategies 25
3.1.3.4 Context Length and Attention Limitations 25
3.1.3.5 Instruction Following vs. Truthfulness Trade-offs 25
3.1.4 Hallucination Detection Methods 25
3.1.4.1 Consistency-Based Methods 25
3.1.4.2 Uncertainty-Based Methods 26
3.1.4.3 External Knowledge Verification 27
3.1.4.4 Statistical Pattern Analysis 27
3.1.5 Evaluation Metrics and Benchmarks 27
3.1.5.1 Detection Metrics 27
3.1.5.2 Factuality Benchmarks 28
3.1.6 Hallucination Mitigation Strategies 29
3.1.6.1 Retrieval-Augmented Generation (RAG) 29
3.1.6.2 Prompt Engineering 29
3.1.6.3 Fine-Tuning and Reinforcement Learning 30
3.1.6.4 Architectural Modifications 30
3.1.6.5 Post-Processing and Verification 30
3.1.7 Implications for LLM-Based Causal Discovery 30
3.2 Test-Time Compute: Scaling Inference for Improved Performance 31
3.2.1 Introduction to Test-Time Compute 31
3.2.2 Theoretical Foundations 32
3.2.2.1 Compute-Optimal Scaling Strategy 32
3.2.2.2 Connection to Meta-Reinforcement Learning 32
3.2.2.3 Scaling Laws for Test-Time Compute 33
3.2.3 Primary Test-Time Compute Mechanisms 33
3.2.3.1 Best-of-N Sampling 33
3.2.3.2 Chain-of-Thought and Self-Consistency 34
3.2.3.3 Iterative Refinement and Revision 34
3.2.3.4 Search-Based Methods with Process Reward Models . . . 35
3.2.4 Adaptive Compute Allocation 36
3.2.4.1 Estimating Problem Difficulty 36
3.2.4.2 Allocation Strategies 36
3.2.5 Applications and Practical Considerations 37
3.2.5.1 Enterprise Applications: TAO 37
3.2.5.2 Cost-Benefit Analysis 37
3.2.5.3 Latency Considerations 38

3.2.6 Future Directions and Open Questions . . . 38
3.2.6.1 Pre-training vs. Inference Compute Trade-offs . . . 38
3.2.6.2 Generalization Across Domains . . . 38
3.2.6.3 Learned Inference Algorithms . . . 39
3.2.6.4 Integration with Hallucination Detection . . . 39
3.2.7 Implications for LLM-Based Causal Discovery . . . 39
3.3 Conclusion . . . 40

4 Methods: Experimental Setup 41
4.1 Methods: Implementation . . . 43
4.1.1 System Architecture and Multi-Agent Orchestration . . . 43
Generator Agent . . . 43
Corruptor Agent . . . 43
Judge Agent Ensemble . . . 43
Infrastructure and Monitoring . . . 43
4.1.2 Ground-Truth Dataset Specification . . . 43
4.1.3 Advanced Prompting and Chain-of-Thought Integration . . . 44
Variable Generation . . . 44
Relationship Discovery . . . 44
Judge Verification . . . 44
4.1.4 Context-Insensitive Metrics Implementation . . . 44
Perplexity . . . 45
Minimum Token Probability . . . 45
Maximum Window Entropy . . . 45
Semantic Alignment (Batched) . . . 45
4.1.5 LLM-as-a-Judge Aggregation Framework . . . 45
Citation Acquisition . . . 45
Per-Citation Evaluation . . . 45
Multi-Judge Ensembles . . . 46
Quantitative Verdict Mapping . . . 46
4.1.6 Smart Test-Time Compute Allocation . . . 46
4.1.7 Experimental Design and Statistical Framework . . . 46
Parameter Space . . . 46
Reproducibility . . . 46
Statistical Analysis . . . 47
4.1.8 Output Artifacts and Evaluation Metrics . . . 47
Edge-Level CSVs . . . 47
Summary Statistics . . . 47
Export Pipeline . . . 47
4.1.9 Implementation Mapping to Research Questions . . . 47
RQ1: LLM-as-a-Judge and Corrector Efficacy . . . 47
RQ2: CI Metrics as Hallucination Indicators . . . 47
RQ3: Smart Test-Time Compute . . . 48
4.1.10 Reproducibility and Scientific Standards . . . 48
4.1.11 Mathematical Foundations and Algorithmic Complexity . . . 48
4.1.11.1 Information-Theoretic Framework . . . 48
4.1.12 Neo4j Graph database Architecture . . . 48

4.1.12.1 Schema and Properties . . . 48
4.1.12.2 Indexes and Performance . . . 49
4.1.13 Enhanced Statistical Methods and Experimental Design . . . 49
4.1.13.1 Robust Inference . . . 49
4.1.13.2 Multiple Comparisons and Power . . . 50
4.1.13.3 Bayesian Modeling . . . 50
4.1.14 Deep Learning Theory and Embedding Mathematics . . . 50
Embedding Geometry . . . 50
Attention and Representations . . . 50
Similarity Interpretation and Chunking . . . 51
4.1.15 Performance Optimization and Systems Engineering . . . 51
4.1.15.1 Parallel Processing . . . 51
4.1.15.2 Rate Limiting and Cost Optimization . . . 51
4.1.16 Variable Generation Input and Context Inference . . . 51
4.1.16.1 Target Variable Seeding . . . 52
4.1.16.2 Context Inference from Validation Nodes . . . 52
Step 1: Extract Validation Variable Names . . . 52
Step 2: LLM-Based Context Inference . . . 52
Step 3: Context List Construction . . . 53
Step 4: Variable Generation with Inferred Context . . . 53
4.1.17 Variable Matching Methodology . . . 54
4.1.17.1 Binary Matching (Stage 1) . . . 54
4.1.17.2 Cosine Similarity Metrics . . . 54
4.1.17.3 Hybrid Matching (Two-Stage Bipartite) . . . 55
Stage 1: . . . 55
Stage 2: . . . 55
Key advantages of this methodology: . . . 55
4.1.17.4 Adjusted Metrics for Seeded Variables . . . 56
Impact of Adjustment . . . 56
4.2 RQ2: Context-Insensitive Metrics for Hallucination Detection . . . 57
4.2.1 CI Metrics: Mathematical Definitions . . . 57
4.2.1.1 Generator Perplexity . . . 57
4.2.1.2 Generator Minimum Probability . . . 58
4.2.1.3 Generator Maximum Window Entropy . . . 58
4.2.1.4 Generator Cosine Similarity . . . 59
4.2.1.5 Summary of CI Metrics . . . 59
4.2.2 Statistical Analysis Methods . . . 59
4.2.3 Study Design and Data Structure . . . 60
4.2.3.1 Experimental Data Structure . . . 60
Causal Loop Diagrams (CLDs). . . . 60
Prompt Types. . . . 60
Runs. . . . 60
Total Sample Size. . . . 60
4.2.3.2 Cross-CLD Validation Design . . . 61
4.2.3.3 Ground Truth Definition . . . 61
Hallucination labeling. . . . 61
Data availability. . . . 61

Note on corruption experiments. . . . 61
4.2.4 Statistical Test Procedures . . . 62
4.2.4.1 Correlation Analysis . . . 62
Pearson Product-Moment Correlation. . . . 62
Spearman Rank Correlation. . . . 62
4.2.4.2 Classification Performance Analysis . . . 62
Receiver Operating Characteristic (ROC) Analysis. . . . 62
Optimal Threshold Selection. . . . 63
4.2.4.3 Permutation Tests . . . 63
Algorithm. . . . 64
Obtained Results. . . . 64
Multiple Comparison Correction. . . . 65
4.2.4.4 Bootstrap Confidence Intervals . . . 65
Algorithm . . . 65
Obtained Results. . . . 66
4.2.4.5 Group Comparisons . . . 66
Test statistic: . . . 67
Effect Size. . . . 67
4.2.5 Meta-Analysis Framework . . . 67
4.2.5.1 Cross-CLD Aggregation . . . 67
Mean and Standard Deviation. . . . 67
Meta-Analytic Hypothesis Tests. . . . 68
Confidence Intervals for Meta-Statistics. . . . 68
Obtained Meta-Analysis Results. . . . 68
4.2.6 Assumptions and Diagnostics . . . 69
4.2.6.1 Parametric Test Assumptions . . . 69
Normality. . . . 69
Homoscedasticity. . . . 69
Independence. . . . 69
4.2.6.2 Multiple Testing Considerations . . . 70
4.2.7 Computational Implementation . . . 70
4.2.7.1 Software Environment . . . 70
4.2.7.2 Analysis Pipeline Architecture . . . 70
Stage 1: Data Preparation. . . . 70
Stage 2: Single-CLD Analysis. . . . 71
Stage 3: Meta-Analysis Aggregation. . . . 71
4.2.7.3 Reproducibility Specifications . . . 71
Random seed control. . . . 71
Computational cost. . . . 71
Parallelization. . . . 72
4.2.8 Statistical Power and Sample Size . . . 72
Correlation detection power. . . . 72
Meta-analysis power. . . . 72
Limitations. . . . 72
4.2.9 Summary . . . 72

5 Global Sensitivity Analysis **73**

5.1 Introduction . . . 73
5.2 Theoretical Background . . . 74
5.2.1 Global Sensitivity Analysis . . . 74
5.2.2 Variance-Based Sensitivity Analysis . . . 74
5.2.3 Sobol Indices . . . 75
5.2.3.1 First-Order Sensitivity Index . . . 75
5.2.3.2 Total-Effect Sensitivity Index . . . 75
5.2.3.3 Interpretation . . . 76
5.2.3.4 Second-Order Indices . . . 76
5.2.3.5 Mathematical Properties . . . 76
5.2.4 Saltelli Sampling Method . . . 77
5.2.5 Convergence and Sample Size . . . 77
5.3 Methodology . . . 78
5.3.1 Research Questions and Parameters . . . 78
5.3.1.1 RQ1: LLM-as-a-Judge and Corrector . . . 78
5.3.1.2 RQ2: Context-Insensitive Metrics . . . 78
5.3.1.3 RQ3: Smart Test-Time Compute . . . 79
5.3.2 Experimental Design . . . 79
5.3.2.1 Sample Generation . . . 79
5.3.2.2 Experiment Configuration . . . 80
5.3.2.3 Causal Loop Diagrams . . . 80
5.3.2.4 Outcome Metrics . . . 81
5.3.3 Implementation . . . 81
5.3.4 Computational Considerations . . . 82
5.3.5 Convergence Assessment . . . 82
5.4 Statistical Interpretation . . . 83
5.4.1 Confidence Intervals . . . 83
5.4.2 Multiple Testing . . . 83
5.4.3 Limitations . . . 83
5.5 Results . . . 84
5.5.1 RQ1: Judge and Corrector Sensitivity . . . 84
5.5.2 RQ2: Context-Insensitive Metrics Sensitivity . . . 84
5.5.3 RQ3: Smart Test-Time Compute Sensitivity . . . 84
5.5.4 Convergence Analysis . . . 84
5.5.5 Generalization Across CLDs . . . 84
5.6 Discussion . . . 85
5.6.1 Parameter Prioritization for Optimization . . . 85
5.6.2 Interaction Effects and Joint Optimization . . . 85
5.6.3 Model Simplification Opportunities . . . 85
5.6.4 Implications for System Design . . . 85
5.7 Conclusion . . . 85

6 Prompt Engineering Ablation Studies: Edge and Node Generation 87
6.1 Edge Generation Ablation Study . . . 87
6.1.1 Experimental Configuration . . . 87
6.1.2 Performance Results . . . 88
6.1.2.1 Overall Rankings . . . 88

6.1.2.2 Key Observations . . . 88
6.1.3 Discussion of Edge Results . . . 90
6.1.3.1 Chain-of-Thought Progression . . . 90
6.1.3.2 Confounding Factors . . . 91
6.1.3.3 CLD 2 Underperformance . . . 91
6.1.3.4 Restrictive Grounding Mechanisms . . . 91
6.1.3.5 Decomposition vs. Chain-of-Thought . . . 91
6.2 Node Generation Ablation Study . . . 92
6.2.1 Motivation and Design . . . 92
6.2.2 Results . . . 93
6.2.3 Interpretation . . . 94
6.3 Cross-Study Comparison: Task Complexity and Prompting Effectiveness . 95
6.3.1 Task Complexity Hypothesis . . . 95
6.3.2 Consistent Findings Across Studies . . . 95
6.4 Unified Recommendations for System Design . . . 96
6.5 Limitations . . . 97
6.6 Summary . . . 97

7 Experiments and Results 99
7.1 Experiment 1.1 . . . 99
Protocol. . . . 99
Findings. . . . 100
7.2 Experiment 1.1c: parallel ensemble judging . . . 101
7.3 Cost scaling . . . 102
7.4 Parameter experiments . . . 102
7.5 Cost scaling with variable N . . . 102
7.6 Temperature scaling . . . 102

8 Multi-CLD Validation of Context-Insensitive Hallucination Metrics (RQ2) 104
8.1 Dataset Coverage . . . 104
8.2 Performance Across All Metrics . . . 105
8.3 Statistical Significance Summary . . . 105
8.3.1 Correlation with Hallucinations . . . 105
8.3.2 Classification Performance . . . 105
8.3.3 Cross-CLD Stability . . . 105
8.3.4 Practical Performance . . . 106
8.4 The Cosine Similarity Paradox . . . 106
8.4.1 Expected vs. Observed Behavior . . . 106
8.4.2 Hypotheses for the Paradox . . . 106
8.4.2.1 Hypothesis 1: Ground Truth Incompleteness . . . 107
8.4.2.2 Hypothesis 2: Max-Aggregation Flaw . . . 107
8.4.3 Implications . . . 108
8.5 Answer to Research Question 2 . . . 108
8.5.1 Research Question . . . 108
8.5.2 Answer: PARTIALLY - with Important Caveats . . . 108
8.5.2.1 What Works . . . 108

8.5.2.2 What Doesn't Work . . . 109
8.5.2.3 Practical Implications . . . 109
8.6 Limitations and Future Work . . . 109
8.6.1 Limitations . . . 109
8.6.2 Future Work . . . 110
8.7 Conclusion . . . 110

9 Discussion **112**

10 Conclusion and future work **113**

11 Ethics and Data Management **114**

A RQ2 Detailed Statistics and Analysis **115**
A.1 Correlation Statistics: Complete Analysis . . . 115
A.1.1 Full Correlation Table with Confidence Intervals . . . 115
A.1.2 Correlation Stability Analysis . . . 116
A.2 Classification Performance: Detailed Breakdown . . . 116
A.2.1 AUC Statistics with Bootstrap Confidence Intervals . . . 116
A.3 Optimal Threshold Analysis . . . 117
A.3.1 F1 Score at Optimal Decision Boundaries . . . 117
A.3.2 Interpretation of F1 Performance . . . 117
A.4 Statistical Validation Summary . . . 117
A.4.1 Hypothesis Testing Results . . . 117
A.5 Per-CLD Performance Breakdown . . . 118
A.6 Limitations of Statistical Validation . . . 118
A.6.1 Sample Size Considerations . . . 118
A.6.2 Multiple Testing Concerns . . . 118
A.7 Computational Efficiency Notes . . . 119

List of Figures

6.1 Average F1 scores for prompt variants. Nitai_C achieved the highest performance (F1=0.460), while Cillian_C failed completely (F1=0.000). Color tiers: green (0.45), blue (0.35–0.45), orange (0.25–0.35), red (¡0.25). Mean = 0.321 (red dashed); Median = 0.356 (purple dotted). 89

6.2 F1 scores by CLD and prompt variant. CLD 1 (blue), CLD 2 (red), CLD 3 (green), Average (orange, labeled). Note consistent underperformance on CLD 2 across all variants and high variance within prompts. 89

6.3 Node generation performance by prompt variant. All variants achieved Hybrid F1 scores between 0.62–0.68 with overlapping confidence intervals, indicating minimal practical differences. ANOVA: F=0.807, p=0.567 (not significant). 93

7.1 Experiment 1.1 with corruption rate = 1.0 (run A). 99

7.2 Experiment 1.1 with corruption rate = 1.0 (run B). 100

7.3 Experiment 1.1 with corruption rate = 0.5 (run A). 100

7.4 Experiment 1.1 with corruption rate = 0.5 (run B). 100

7.5 Confusion matrices for parallel ensemble judging (Experiment 1.1c). . . . 101

7.6 Comparison plot for parallel ensemble judging (Experiment 1.1c). 101

7.7 Estimated impact and cost savings from hallucination reduction. 101

7.8 Preliminary cost scaling with N. 102

7.9 Model/prompt comparison against 3-CLD ground truth. 102

7.10 Cost scaling as a function of variable N. 102

7.11 Enter Caption 103

7.12 Enter Caption 103

7.13 Enter Caption 103

List of Tables

4.1 CI metrics summary. *Counterintuitive: higher similarity predicts *more* hallucinations. . . . 59
4.2 RQ2 experimental file structure. Each file contains one complete CLD generation. . . . 60

5.1 RQ1 Sensitivity Analysis Parameters . . . 78
5.2 RQ2 Sensitivity Analysis Parameters . . . 78
5.3 RQ3 Sensitivity Analysis Parameters . . . 79

6.1 Prompt Engineering Techniques Applied per Variant . . . 88
6.2 F1 Scores by Prompt and CLD . . . 90
6.3 Node Generation Prompt Variants and Applied Techniques . . . 92
6.4 Node Generation Performance by Prompt Variant . . . 93
6.5 Edge vs. Node Generation Ablation Study Comparison . . . 95

8.1 CI Metrics Performance: Multi-CLD Meta-Analysis (N=3 CLDs, 6 runs) . 105
8.2 Cosine Similarity Paradox: Expected vs. Observed Correlations . . . 106

A.1 Complete Correlation Analysis: All CI Metrics . . . 115
A.2 Metric Stability Classification . . . 116
A.3 Complete AUC Analysis: All CI Metrics . . . 116
A.4 F1 Score Performance at Optimal Thresholds . . . 117
A.5 Statistical Hypothesis Tests: Summary . . . 118

Abbreviations

CSL	**C**omputational **S**ceince **L**ab
UvA	**U**niversitiet **v**an **A**msterdam

Chapter 1

Introduction

Uniquely combining LLM-as-a-judge with context-insensitive hallucination mitigation, our CLD building system generates self-corrected, uncertainty-quantified intial CLDs, redirecting experts attention in CLD-building session from reiterating consensus to adding specialized knowledge to models. Rigorous CLDs translate into coherent computational models for research and (intervention) simulation. Findings can reduce hallucinations and increase efficiency of test-time compute in multi-agent LLM-based research systems.

An agent is a specialized unit with an objective function and unique state. This heterogeneity of state makes the agent suited to adress a smaller subtask given the whole. As long as tasks can be broken up into smaller, less complex subtasks, and the architecture behind the agent decision making allows for higher effiency in solving these subtasks than serving the whole. Transformer based large language models are candidate for such an architecture, as the computational requirements for solving the tasks scale with the input sequence length, L, squared, or O(L2). Also, the possibility for these models to operate as a markovchain, where the next token prodability dsitrbution is predicted given the previous tokens in the context, allows the output to be heavily influenced by how a specific task is broken down into subtasks, and also the specific wording of the instructions. The fact that Language models operate on the level of langauge, allows less clear cut instructional definitions and gives the architecture flexbility, in terms that it can give an output even if the instructions are less specifically or even imperfectly defined, although this ofcoursee will influence output quality.

This makes the core task for a developer of an transformer LLM based multi agent system to build a system that preloads the context window with usefull information, or to give the LLM tools to gather this usefull information itself and load it into its context. This can be through Retrieval (yielding Retrieval Augmented Generation or

RAG architecture), invoking computation (function calling, tool use), and is facilitated by structured input and output formats. These innovations come together in Anthropics recent Model Context Protocol, which provides a standardized structured format for the model to access and use tools and prompts, and thereby allowing LLM's to access 10000's of tools allowing variations of above.

Another aspect that needs consideration for a system to be considered agentic is control flow of the application. If this is determined by the programmer and deterministic, the LLM powered functions operate in a static control flow unchangable by the agent itself. However, a truely agentic system should, by virtue of the information it gathers about the problem context, be able to steer the control flow to more optimally lead to a soltion to a problem. i.e. the intermediate outputs of computation done should, if usefull, help make a better decision about the next operation needed. (information gathering, processing, evaluating task progress, halting, etc.). Therefor a true agentic system whould be able to infleunce the control flow to solve a problem.

Deciding then if a multi-agent approach (where agent(s) decide the control flow) to your problem is the optimal one, one should therefor consider the following aspects.

- Task complexity Is your task too complex to be solvable by a single agent? - Task decompositionality & interdepedence This the task meaningfully and divisible into smaller coherent parts? What is the dependency structure between these parts

Reasoning is a balance between exporlation (information gathering) and exploitation (using or processing that information). In an automated reasoning process, there exists a balance between obtaining enough information to solve the problem, and processing this information in such a way that a solution is arived at in an efficient manner. As both information retrieval and processing incur costs, and it may not be clear beforehand if the right solution to a problem can be found until the computation is ran, exploring this search space efficiently is a difficult problem, especially because when these systems are employed for knowledge gain, the solutions of the problems is by definition unkonwn.

Having agent decide the control flow allows for dynamism in this process, because through its training it has acquired a framework to evaluate new information stored in its weights due to the training process. However, this requires a call to an LLM to evaluate and decide, and therefor also incur associated cost. If the criteria is clear and measureable, such as for example accuracy against a set baseline, or in a scenario where a target is not known, we can use other indicators to indicate which action should be taken, this can reduce the cost and result in a smart test time compute strategy.

To this end, we test the following LLM based and measure based metrics with the goal of finding a more efficient test time compute deployment strategy:

- LLM as a judge; analogous to self reflection - LLM as a corrector; analogous to incorporating feedback - Contect insensitive white box; model logits; analogous to felt uncertainty in humans - context insensitive black box; cosine similarity, analogous to a double check against source materials

Chapter 2

Literature review

Causal Loop Diagrams

A causal loop diagram (CLD) is a qualitative system-dynamics representation of hypothesized causal structure within a bounded system. It is a signed, directed graph whose nodes denote system variables and whose arcs encode *ceteris paribus* monotonic influences: a "+" arc indicates that, relative to a reference level, an increase (decrease) in the source tends to produce an increase (decrease) in the target; a "–" arc indicates the opposite tendency. Salient delays may be marked on arcs to indicate that the dominant effect is time-lagged. Unlike directed acyclic graphs (DAGs), CLDs admit cycles and explicitly emphasize feedback as a first-class structural feature.

Formalization. A CLD can be defined as $G = (V, E, \sigma, \delta)$, where V is a finite set of variables; $E \subseteq V \times V$ is a set of directed arcs; $\sigma : E \to \{+1, -1\}$ assigns each arc a polarity; and $\delta : E \to \{0, 1\}$ flags salient delays. The intended semantics are that for each arc $i \to j \in E$ there exists a (possibly nonlinear) structural relation

$$x_j \;=\; f_j\big(x_{\mathrm{pa}(j)},\, u_j,\, t\big)$$

such that, over the domain of interest and holding other parents approximately constant, $\sigma(i \to j) = +1$ implies $\partial f_j / \partial x_i \geq 0$ and $\sigma(i \to j) = -1$ implies $\partial f_j / \partial x_i \leq 0$.

Feedback loops. A *feedback loop* is any simple directed cycle. Its polarity is the product of its arc signs: $\prod_{e \in C} \sigma(e) = +1$ denotes a *reinforcing* loop (R), which tends to amplify deviations; $\prod_{e \in C} \sigma(e) = -1$ denotes a *balancing* loop (B), which tends toward goal-seeking or stabilization. (Equivalently, reinforcing loops contain an even number of

negative arcs, balancing loops an odd number.) Loop polarity characterizes qualitative tendencies and does not presume linearity, stationarity, or constant effect sizes.

Scope and use. CLDs deliberately abstract from magnitudes, functional forms, stock-and-flow accounting, and statistical identification of causal effects. They encode theory- or evidence-informed hypotheses about directional dependencies and dominant feedbacks to support elicitation, communication, and high-level reasoning about system behavior, and they are commonly used as a precursor or complement to quantified stock-and-flow models, experiments, and statistical/causal analyses.

Practical limitations. While there is ongoing research on extending CLDs toward quantitatively rigorous computational formalisms (add citations), many such approaches remain assumption-heavy and still maturing. A more fundamental obstacle is the scarcity of high-quality CLDs for the systems of interest, largely due to the prohibitive time investment required to construct them. For a CLD with $|V|$ variables, considering all possible pairwise relationships scales as $O(|V|^2)$, quickly turning large-system modeling into a time-intensive exercise for expert groups, with associated coordination challenges and costs.

Transformer Architecture and Attention Mechanisms

2.1 Evolution of Sequential Modeling Paradigms

The development of modern language models represents a paradigmatic shift in sequential data processing architectures. Traditional Recurrent Neural Networks (RNNs), while foundational for sequence modeling, exhibited critical limitations that constrained their scalability and effectiveness. The sequential processing paradigm inherent in RNNs precluded efficient parallelization, while the vanishing gradient problem fundamentally limited their capacity to model long-range dependencies—a crucial requirement for understanding complex linguistic structures.

The seminal contribution of Bahdanau et al. [**?**] introduced the attention mechanism as a solution to the information bottleneck problem in encoder-decoder architectures. Rather than compressing entire sequences into fixed-length representations, attention mechanisms enabled dynamic focus allocation across input elements, fundamentally altering how neural networks process sequential information. This innovation laid the groundwork for the revolutionary Transformer architecture proposed by Vaswani et al. [**?**], which demonstrated that self-attention mechanisms alone could achieve state-of-the-art performance without recurrence or convolution.

2.2 Self-Attention and Multi-Head Attention Mechanisms

2.2.1 Scaled Dot-Product Attention

The cornerstone of Transformer architectures lies in the scaled dot-product attention mechanism, formalized as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{2.1}$$

This formulation operationalizes attention through four fundamental operations: (1) query-key similarity computation via dot products, (2) normalization scaling by $\sqrt{d_k}$, (3) probability distribution generation through softmax, and (4) value aggregation through weighted summation.

Relevance to Hallucination and Generation Accuracy: The softmax operation in attention is particularly critical for understanding hallucination phenomena. The temperature-like scaling factor $\sqrt{d_k}$ controls the sharpness of attention distributions. When d_k is large without proper scaling, attention weights become increasingly concentrated (approaching one-hot distributions), potentially leading to overconfident predictions and reduced robustness. This concentration effect can contribute to hallucinations when the model becomes overly certain about spurious correlations in the training data.

2.2.2 Multi-Head Attention Architecture

Multi-head attention extends the representational capacity by computing attention in parallel across multiple subspaces:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O \tag{2.2}$$

$$\text{where head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \tag{2.3}$$

Relevance to Generation Accuracy: The multi-head mechanism enables the model to simultaneously capture diverse linguistic relationships—syntactic, semantic, and pragmatic. This parallel processing of multiple attention patterns is crucial for generation accuracy, as it allows the model to maintain awareness of various contextual factors simultaneously. However, it can also contribute to hallucination when different heads learn to attend to conflicting or spurious patterns.

2.3 Architectural Components and Their Impact on Model Behavior

2.3.1 Layer Normalization and Stability

Modern Transformer implementations employ Root Mean Square Layer Normalization (RMSNorm):

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \epsilon}} \cdot \gamma \tag{2.4}$$

where x is the input tensor, d is the dimension, ϵ is a small constant for numerical stability, and γ is a learned scaling parameter.

Relevance to Generation Accuracy: Normalization techniques are crucial for training stability and, consequently, generation quality. Poor normalization can lead to activation distributions that are difficult to model accurately, potentially increasing hallucination rates by making the model less reliable in its predictions.

2.3.2 Feed-Forward Networks and Token-Level Processing

The feed-forward component operates independently on each token position, typically expanding dimensionality before contracting back to the model dimension. This design choice (wide rather than deep) optimizes for parallel computation while enabling complex non-linear transformations.

Relevance to Hallucination: The feed-forward layers can be viewed as associative memory components that store factual knowledge. Overfitting in these layers or inadequate capacity can lead to hallucinations when the model retrieves incorrect associations or fails to generalize properly.

2.4 Positional Encoding Schemes and Sequential Understanding

2.4.1 Absolute vs. Relative Position Embeddings

Traditional absolute positional embeddings assign fixed representations to sequence positions, while relative approaches like Rotary Position Embedding (RoPE) encode distances between tokens through rotation matrices:

$$\theta_{m,i} = m \cdot \omega_i, \quad \text{where } \omega_i = \frac{1}{10000^{2i/d}} \tag{2.5}$$

$$R_\theta = \begin{pmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{pmatrix} \tag{2.6}$$

For each vector x at position m, the rotation is applied to dimension pairs $(2i, 2i+1)$:

$$\begin{pmatrix} x'_{2i} \\ x'_{2i+1} \end{pmatrix} = \begin{pmatrix} \cos(m \cdot \omega_i) & -\sin(m \cdot \omega_i) \\ \sin(m \cdot \omega_i) & \cos(m \cdot \omega_i) \end{pmatrix} \begin{pmatrix} x_{2i} \\ x_{2i+1} \end{pmatrix} \tag{2.7}$$

Relevance to Generation Accuracy and Hallucination: Position encoding directly impacts the model's understanding of temporal and structural relationships. Absolute embeddings can lead to hallucinations when the model encounters sequences longer than those seen during training, as position embeddings for unseen positions may have unpredictable behavior. Relative encodings, while more robust to length variations, can still contribute to hallucinations if the distance relationships learned during training don't generalize appropriately to new contexts.

2.5 Training Paradigms and Generation Objectives

2.5.1 Causal Language Modeling

The autoregressive training objective optimizes:

$$p(w_i^t | w_{t-1}, w_{t-2}, \ldots, w_1) \propto \frac{\exp(s_i)}{\sum_{j \in \mathcal{V}} \exp(s_j)} \tag{2.8}$$

where s_i represents the logit scores for token i and $\mathcal{V}$ is the vocabulary.

Critical Relevance to Hallucination: The next-token prediction objective, while enabling powerful generative capabilities, inherently creates conditions conducive to hallucination. The model learns to maximize likelihood on training data, which can lead to:

1. **Exposure bias**: During training, the model always sees ground truth context, but during generation, it must condition on its own potentially incorrect outputs
2. **Distributional mismatch**: The model optimizes for single-token accuracy rather than sequence-level coherence
3. **Confidence miscalibration**: The softmax objective can produce overconfident predictions even for uncertain situations

2.5.2 Generation Strategies and Their Implications

Two primary decoding strategies demonstrate different trade-offs:

- **Greedy Decoding**: $w_t = \arg\max_w p(w|w_{t-1}, w_{t-2}, \ldots, w_1)$
- **Top-p (Nucleus) Sampling**: Sample from the smallest set $\mathcal{S} \subset \mathcal{V}$ such that:

$$\sum_{w \in \mathcal{S}} p(w|w_{t-1}, w_{t-2}, \ldots, w_1) \geq \tau \tag{2.9}$$

Relevance to Hallucination and Accuracy: Greedy decoding can lead to repetitive and deterministic hallucinations when the model enters high-confidence incorrect states. Stochastic sampling methods like top-p can help mitigate some forms of hallucination by maintaining diversity, but they can also introduce errors when the probability calibration is poor. The choice of sampling strategy thus represents a fundamental trade-off between consistency and diversity in generation quality.

2.6 Memory and Computational Optimizations

2.6.1 Attention Complexity and FlashAttention

The quadratic complexity of attention with respect to sequence length $O(n^2 d)$ poses significant scalability challenges. FlashAttention addresses this through memory-efficient computation by:

1. **Tiling**: Computing attention in blocks to reduce memory footprint
2. **Recomputation**: Trading computation for memory by recomputing attention scores during backpropagation
3. **Fused kernels**: Optimizing GPU utilization through combined operations

Relevance to Model Behavior: While FlashAttention is primarily an optimization technique, the computational constraints it addresses can indirectly affect model behavior. Memory limitations that prevent training on longer sequences can impact the model's ability to learn long-range dependencies, potentially affecting both generation accuracy and hallucination patterns.

2.7 Theoretical Implications for Hallucination and Generation Quality

The Transformer architecture's propensity for hallucination emerges from several theoretical considerations:

1. **Attention Concentration**: The softmax attention mechanism can create overconfident focus on spurious correlations
2. **Parametric Memory**: The model's factual knowledge is embedded in parameters, making it difficult to distinguish between learned facts and learned patterns
3. **Autoregressive Accumulation**: Errors in early tokens can compound through the generation process
4. **Training Objective Limitations**: Next-token prediction doesn't directly optimize for factual accuracy or consistency

Understanding these mechanisms provides the theoretical foundation for analyzing and potentially mitigating hallucination phenomena in large language models, while the architectural components described above form the basis for the model's remarkable generative capabilities.

2.8 Context Vector Computation in Sequence-to-Sequence Models

In the original attention mechanism for neural machine translation, the context vector c_i for each target word y_i is computed as:

$$c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j \tag{2.10}$$

where the attention weights α_{ij} are calculated as:

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T_x} \exp(e_{ik})} \tag{2.11}$$

and $e_{ij} = a(s_{i-1}, h_j)$ represents an alignment function that scores the compatibility between the decoder hidden state s_{i-1} and the encoder annotation h_j.

This formulation demonstrates the evolution from fixed context vectors to dynamic attention-based representations, laying the groundwork for the self-attention mechanisms that define modern Transformer architectures.

LLM for CLD generation and Hallucination reduction

Although LLM-based systems for CLD generation show promise to make expert model building sessions more efficient [7, 16, 3], LLM hallucinations—fabricated or inconsistent outputs—undermine reliability for scientific applications [20, 15]. Black-box mitigation strategies (prompting [18, 1], retrieval-augmented generation [5, 19], multi-agent orches tration [12, 8], and LLM-as-a-judge [21] and test-time compute [9, 12, 2]) and white-box approaches (fine-tuning [4], RLHF [10] activation monitoring [17]) have been found to reduce LLM hallucinations. LLM-as-a-judge advantage is context-awareness, but its vul nerable to (transformer-architectural) biases [6, 11]. We combine LLM-as-a-judge with a set of context-insensitive hallucination detection metrics (cosine similarity [5, 19], per plexity [14, 13], minimum probability [15])—an under-explored pairing that could allow for bias reduction and more efficient deployment of test-time compute for the system to self-correct hallucinations

CLDs are traditionally built during expert knowledge sharing sessions, and since expert time is scarce and the modelling process laborious, these sessions are difficult to organize and proceed with limited efficiently. [**?**]

Expert time is too valuable to be spent on reiterating the variables already known to be of importance to the system, yet reiterating basics to create consensus is often precisely what a large part of these expert model building sessions is spent on, when it could be better utilized for adding specialized knowledge to the model. If an initial starting model could be provided that aligns with the consensus in scientific literature, experts can spend the limited time they have on contributing their specialized knowledge, leading to comprehensive CLDs built in a more efficient way.

This calls for a context-sensitive 'filtration system' that can sift through available knowledge in literature and provide a starting point by retrieving known-to-be relevant variables and relations from the scientific literature for a given problem context into a single model of the complex system, in our case a CLD. This filtration system should construct a coherent, self-consistent CLD around the target variable (i.e. lower back pain) that properly accounts for the causal interrelationships of different parts of the system (i.e. sitting time, stress, age) verifiably grounded in literature and structured enough to be easily visualized, and operationalized into a computational model, which can then be used for research or simulation purposes.

An interesting candidate for such a filtration system is a large language model (LLM), which is trained on trillions of tokens compresses a large amount of information in its parameters, a significant portion of which consists of scientific literature—the primary source and record of most expert knowledge. LLMs have the advantage of effectively compressing orders of magnitude more information than a single expert can process in a lifetime. They also allow for the retrieval of relevant information in negligible time, as well as enabling synthesis across techniques applied in separate, often siloed, branches of science, thereby facilitating knowledge synthesis, a prerequisite to new knowledge generation. One could use LLMs trained on the research literature to generate an initial sketch of a causal loop diagram (CLD), as is initially explored by [**?**] [**?**].By providing this starting point—a CLD grounded in scientific literature—the time required to develop a CLD and subsequent computational model could be significantly reduced, leading to a model that incorporates more specialized knowledge at reduced expert time cost. If the information retrieval from LLM's is sufficiently 'lossless', that is. Building a CLD construction system has also been tried in the form of a LLM powered system dynamics bot [**?**], but this bot while succesfully annotating 60 % of links in their dataset, still misses a significant number of links and requires human corrections.

LLMs are also known to hallucinate, which is a key challenge standing in the way if this paradigm shifting impact as knowledge retrieval and aggregation systems. The term *hallucination* refers to instances where a model generates content that is not supported by the information in its training data, available retrieval sources, or current context.

Concretely, an LLM is said to "hallucinate" when it fabricates claims, cites non-existent references, or asserts incorrect details with high confidence [**?** **?**]. These hallucinations can appear plausible and coherent at a glance, but are in fact factually or logically incorrect. Hallucinations have been shown to persist despite increases in scaling with data and compute, and therefore cannot be expected to disappear as model test-time accuracy increases and need to be addressed differently [**?**]. Reducing hallicinations makes CLDs generated by LLM's more accurate and robust. This thesis addresses application of different hallucination reduction methods to the CLD building system based on LLMs to increase accuracy of generated CLD's, with the goal of enhacing reliability and enabling LLM-powered knowledge generation to scientific standard.

The first objective is to detect and mitigate hallucinations using context-sensitive LLM-based methods. The second objective is to detect and mitigate hallucinations using context-insensitive metrics. The third objective is to develop and test a hybrid verification approach, combining these methods for greater efficiency in hallucination reduction and reduced susceptibility to transformer architec- tural biases. The intended contribution is more efficient self-correction of hallucination by the CLD building system leading to higher validity of final CLD output at a lower compute cost and, therefore, higher value of LLM-based systems as a research assistant.

Methods for mitigating hallucination can be distinguished into white-box and black-box methods. Black-box methods do not require knowledge of model weights, parameters, or activations to be used, whereas white-box methods do. The following overview details these methods:

Black-box Hallucination Reduction Methods

Prompt-based:

- **Prompt Engineering Techniques:** Techniques such as a system prompt detailing the model's role, Chain of Thought (CoT) prompting [**?**], and providing example(s) of expected input-output task outcomes (multiple-shot learning) [**?**].
- **Retrieval-Augmented Generation:** Relevant context is retrieved by embedding both the query and a knowledge base into vector space, then selecting the most similar knowledge (using metrics like cosine similarity) to include in the prompt. Te LLM's response is then conditioned on this knowledge [**?**]. Variations of data structures to retrieve data from include knowledge graphs. Providing valid context for conditioning has been shown to be an effective hallucination reduction method in LLMs in general [**?**] and also in the specific context of building CLDs

[**?**]. Retrieving a subset of the knowledge base becomes useful when the context to be used as input to the LLM exceeds its maximum window. In that case, the most relevant context, as determined by a distance metric (L2 distance, cosine similarity), is included through retrieval. It can also be used to do reverse retrieval to find which document(s) in a corpus of training data an LLM's response is based on.

System Orchestration:

- **Verification Agents:** LLM-based Agents assigned verification roles with respect to the system. Examples include LLM-as-a-judge [**?**], where an LLM judges if the output of another LLM is correct, and if not, an LLM-as-a-corrector to reflect on and correct an incorrect answer passed by a previous LLM. [**?**].
- **Multi-agent Ensemble:** Deploying multiple agents with seperate instruction sets in serial or parallel configurations to "divide and conquer" the reasoning process (as seen in architectures like EVER and Chain-of-Verification [**?**]).
- **Test-time Compute:** Using additional compute at test time (via multi-agent ensemble and reinforcement learning) to perform chain-of-thought reasoning and correction. This approach—employed in models such as OpenAI's o1 [**?**] and DeepSeek-R1 [**?**]—improves answer quality by allowing for reflection and iterative refinement.

Incorporation of Context-insensitive Metrics: Context-insensitive metrics use statistical properties of generated responses, independent of semantic interpretation, to detect potential hallucinations.

- **Cosine Similarity to Retrieved Sources:** By computing the cosine similarity between cited source material and the LLM-generated response, deviations (i.e., low similarity) can signal hallucination.
- **Perplexity and Log-Likelihood Analysis:** High perplexity scores may indicate that the output diverges from the training distribution, serving as a flag for hallucinated content [**?**].
- **Semantic Entropy for Hallucination Detection:** Elevated semantic entropy—measuring uncertainty over the meaning of generated responses—may point to hallucinated content [**?**].
- **Hamiltonian Interpretation of Multi-Hop Reasoning Paths:** Analyzing the trajectory of multi-hop reasoning, modeled as a Hamiltonian system, can help identify unstable chains indicative of hallucination [**?**].

White-box Hallucination Reduction Methods

- **Fine-tuning & RLHF:** Involves retraining model weight matrices using input-output examples and associated scores in a reinforcement learning framework. involves retraining weight matrices of the model given a set of input-output examples and associated scores to be used in a reinforcement learning setup to improve the accuracy of the model in a given task. This has been shown to increase the relevancy of responses of smaller models to be competitive with larger models and reduce hallucinations [**?**]. Fine-tuning can be done at lower training cost and with less loss of generality by employing LoRa (low-rank adaptation) and selecting only a subset of layers to fine-tune [**?**].

- **Test-time compute** Test-time compute has emerged as the primary paradigm driving progress in reasoning large language models (LLMs). This approach deploys additional compute resources during inference to enhance the quality of the final output. One key motivation behind test-time compute is the diminishing returns observed during the pre-training phase—additional compute yields progressively smaller efficiency gains as models scale [**?**]. In practice, test-time compute leverages a combination of multi-agent ensembles, reinforcement learning, and chain-of-thought reasoning steps. LLM ensembles are arranged in a serial configuration with carefully designed prompts, enabling them to "divide and conquer" the reasoning process into distinct steps. These steps often involve reflecting on and correcting errors made by preceding agents in the chain, either through explicit programming or as an emergent behavior.Moreover, the system can operate either serially or via multiple parallel branches. In the latter case, intermediate results are systematically aggregated and evaluated to ensure the overall coherence of the reasoning process, ultimately producing a single, refined final answer.This methodology is employed in models such as OpenAI's o1 [**?**]—although beyond mentioning the use of reasoning steps, multiple LLM calls, and reinforcement learning, specific technical details remain undisclosed—and more recently in DeepSeek-R1 [**?**], for which implementation code has been published.

- **Activation Pattern Monitoring:** Activation pattern monitoring involves monitoring model internals to map occurrences of hallucinations to specific neurons or layers within the model and reduce these by steering activations [**?**]. Other research has shown neurons which exhibit undesirable characteristics, such as having very high sensitivity, can contribute to hallucinations, and this can be addressed using neural dropout during fine-tuning [**?**].

- **Predictive Analysis of System Capacity:** Predictive analysis of system capacity based on the eigenvalue spectrum of model weight matrices: Research from

> Chris Martin et al. has shown that model layers that strike a good balance between overfitting and underfitting have an alpha exponent of between 2 and 4 [**?**] [**?**] [**?**]. It remains to be investigated information could possibly be used to predict performance on tasks in addition to scores in the task itself, or to find a threshold for determining enough fine-tuning training that holds up in the context of a CLD building system.

Another pre-requisite for building a reliable knowledge synthesis CLD building system using is to determine whether the generating LLM architecture is in principle capable of generating new knowledge. Recent research indicates that Large Language Models (LLMs) can synthesize new knowledge by integrating information from scientific literature and datasets, enabling scientific inference and explanation [**?**].

In chemistry, AI systems powered by LLMs have been developed to autonomously design, plan, and execute complex experiments, showcasing their potential in generating new scientific insights [**?**]. Recent shows AI systems can be zero shot hypothesis generators [**?**] and multiple LLMs can even be setup to work together and form an autonomous 'AI scientist' [**?**]. LLM based systems have also been specifically used for discovery of causal graphs [**?**]. This indicates their potential for being a valuable generating model for a CLD building system.

However, as extensively discussed also mentioned in our aforementioned hallucination discussion, ensuring the factual accuracy of knowledge synthesized by LLMs remains a challenge, as studies have shown that while LLMs can generate coherent and contextually relevant information, their outputs may not always be reliable [**?**].

To enhance the knowledge synthesis capabilities of LLMs, researchers have proposed modular frameworks that integrate specialized language models, allowing for the dynamic incorporation of new, domain-specific knowledge into general-purpose LLMs [**?**]. Domain specific Finetuning has also shown promise to increase accuracy of knowledge generated by these LLM's [**?**].

For this thesis, the possibility of pretraining an open-source LLM from scratch on a curated set of scientific papers containing CLD data was briefly considered to enhance accuracy in CLD generation. This approach aligns with training methodologies used for models such as GPT, Anthropic's models, and the LLaMA series. An additional advantage of pretraining from scratch is the ability to verify, through direct access to the training data, whether the model generates CLDs beyond its training set—thereby assessing its capability for genuine knowledge synthesis. Examining feasability of pretraining DeepSeek V3 671B and LLAMA 3.2 yielded the following findings: training DeepSeek V3 requires approximately 2.788 million H800 GPU hours, and even with a

supercomputer like Snellius (offering 352 H100 GPUs) [**?**], the estimated training time would be around 330 days. Although H100 GPUs avaialbe on snellius are more capable than H800 GPUs, the scenario remains impractical. Other models, such as LLaMA 305B, would require even more computational resources (an estimated 30.45 million GPU hours). Moreover, significant implementation challenges (e.g., the absence of publicly available training code and the enormous data requirements) render this approach unfeasible. In addition to prohibitive compute and data costs, other research has also demonstrated the knowledge synthesis capabilities of LLMs [**?**], [**?**], [**?**]. Consequently, training LLMs from scratch was deemed outside the scope of this research.

Recently, groundbreaking finetuning techniques have emerged that could significantly enhance CLD generation accuracy while requiring orders of magnitude less compute and data. One such open-source approach—fine-tuning models on valid reasoning traces (structured Chain-of-Thought processes leading to correct results)—has demonstrated a drastic reduction in the number of training samples needed to achieve strong performance in mathematical and reasoning tasks. With just 1,000 diverse, high-quality samples generated by a reasoning model (Google Gemini Pro), this method can fine-tune an open-source model, Alibaba's QWEN-25B, to match the performance of a state-of-the-art competitive model (in this case, OpenAI's o1) in just 26 minutes using 16 H100 GPUs [**?**].

What makes this result groundbreaking is the orders-of-magnitude reduction in both cost and computational requirements, while maintaining effectiveness. Similar techniques are now being leveraged in leading reasoning models such as OpenAI's o3-mini, o1, and DeepSeek's R1, pushing AI's reasoning capabilities beyond the expectations set by prior pretraining scaling laws [**?**]. As a result, fine-tuning an LLM for the CLD-building system using reasoning traces appears feasible, even with constrained compute and data resources.

Future research aimed at improving the CLD building system could explore fine-tuning on validated CLD reasoning traces obtained by using a CLD building system—ideally verified by experts or, at a minimum, assessed by LLM-as-a-judge—to enhance final CLD accuracy.

The reasoning step would then be the rationale described by the LLM for adding a node to the CLD, where the reasoning trace or chain would be all reasoning steps over time required to arrive at the final CLD model state, which would then be validated by an expert. We would then apply the finetuning methodology of an open source model on these reasoning traces, instead of reasoning traces generated by another reasoning model. While it remains to be seen if training on these different types of reasoning traces yields similar results, the important analogies between the CLD model building process

as designed at present and test time compute process used to generate the reasoning traces (serial test time compute, chain of thought prompting leasing to reasoning traces), suggest promising potential.

However, before exploring this potential, a set of validated reasoning traces is required. Running the system with the proposed hallucination reduction methods could help ensure the quality of these traces while also accelerating the validation process. By removing invalid or fixing reasoning traces before they are passed to experts, this approach increases the likelihood that only high-quality examples undergo validation, making the validation process more efficient and reliable.

Now that an overview of relevant methods has been provided, the next section will describe which of these techniques will be combined in what way to to reduce hallucinations and enhance reliability of the CLD system in this thesis.

The first context-sensitive method that will be used to detect hallucination is LLM-as-a-judge. Recent studies indicate that LLM-as-a-Judge-derived accuracy metrics align more closely with human evaluations than traditional, context-insensitive metrics—achieving up to 80% agreement with human evaluators [**?**]. LLM-as-a-judge has been setup succesfully to verify outputs of an LLM based research system [**?**]. using an LLM-as-a-corrector has also been used to reflect on outputs known to contain hallucinations, to 'fix' the reasoning step, which is employed when deploying serial test-time compute and 'reflect' prompts [**?** **?** **?**]. This makes them strong candidates for evaluating CLD system outputs in a context-aware way, especially when expert human evaluation is scarce. As such, they serve as our initial verification mechanism.

Robust verification and annotation are essential for trust in the resulting CLD. By systematically reviewing each causal link—whether generated by the model or derived from a retrieval mechanism—we can identify uncertain or fabricated edges and remove them before they become part of the final model, or indicate the uncertainty beloning to a certain link. This mirrors established best practices in conceptual modeling, where structural uncertainties should never be ignored. Instead, they must be documented so that the final model remains transparent and allows for straightforward validation.

However, LLMs are not without flaws, as accuracy has been shown to decrease with an increased context window size, and the likelihood of hallucination rises when information is incomplete (i.e., having only an abstract of a paper instead of the entire text) [**?**]. Setting up LLM-as-a-Judge has also been shown to be susceptible to adversarial attacks [**?** **?**]. Moreover, using one LLM to evaluate another could amplify structural biases inherent to the Transformer architecture.

This motivates a hybrid approach that combines LLM-based evaluation with complementary, context-insensitive metrics. Statistical properties have been used to detect hallucinations with some success [**?** **?**]. Retrieval-Augmented Generation (RAG)—which involves feeding the context to the prompt that has the minimum distance (in terms of cosine similarity or Euclidean distance) in vector space to the user query—has also been shown to reduce hallucinations [**?**], including in the context of CLD discovery [**?**]. However, using RAG requires similarity between the initial query and the training corpus, necessitating a search over the corpus used to train the generating LLM, which is not available for many state-of-the-art closed-source models. However, certain provider API's (e.g. the perplexity API) can return the sources that the LLM used to generate answers.

This means we can still apply the principle of checking the similarity between the answer and the source indicated by the LLM, using a "Reverse RAG" setup: first, we generate an answer; then, we check if it has sufficient cosine similarity with the chunks of source material the LLM cites. The OpenAI API also returns log probabilities over the tokens, which can be used to calculate context-insensitive metrics, concretely the following context-insensitive metrics will be considered:

Cosine similarity score between the chunk(s) of the source cited by the LLM and the LLM-generated answer. Logit-based metrics derived from the LLM's output: (a) perplexity and (b) minimum probability (as defined in [**?**]). While these methods have been explored individually, their combined use with LLM-as-a-Judge remains underexamined [**?**]. An added advantage of these statistical properties is their relatively low computational cost and domain-agnostic nature, which could allow verification of reasoning traces without requiring expert intervention.

If these methods can be effectively integrated, it may be possible to reduce hallucinations beyond what is achievable using LLM-as-a-Judge, ensemble model techniques (parallel or serial calls to LLMs), or RLHF setups alone. Specifically, the context-insensitive metrics could serve as early indicators of hallucination, helping optimize compute resource allocation (e.g., strategically deploying LLM-as-a-Judge calls) for hallucination correction. This could lead to more efficient use of test-time compute to build the CLD. Test time compute has proven central to achieving state-of-the-art reasoning. If we can use context-insensitive metrics to deploy test-time compute more efficiently, this could reduce costs, potentially increase speed, and serve as a scoring mechanism for the generated relationships in the CLD, thereby increasing trust in the model-building process.

In summary, the goal is to implement a combination of context-sensitive and context-insensitive hallucination reduction methods in the LLM-based CLD-building system to

generate more accurate final CLDs in a more efficient manner. The effectiveness of these methods will be evaluated through the following experiments:

2.9 Defenition of Hallucination

OpenAI why language models hallucinate:

https://www.arxiv.org/abs/2509.04664

"The paper argues that LLM hallucinations arise naturally from the pretraining objective: by reducing generation to an "Is-It-Valid" binary classification problem, the authors show that even with error-free data, cross-entropy–trained base models will still produce errors (notably on arbitrary facts or under model misspecification), extending prior single-occurrence bounds to settings with prompts and explicit "I don't know." arXiv

They then explain why post-training doesn't eliminate the problem: prevalent 0–1 graded benchmarks penalize abstentions and uncertainty, so models are effectively rewarded for confident guesses, which sustains overconfident hallucinations. arXiv

As a remedy, the authors call for socio-technical changes to primary evaluations and leaderboards—modifying scoring to stop penalizing calibrated uncertainty—rather than adding more standalone hallucination tests"

2.10 Test time compute

Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters

https://arxiv.org/abs/2408.03314

"The paper shows that scaling test-time compute—via search against process reward model (PRM) verifiers and iterative revision of answers—works best when allocated adaptively per-prompt ("compute-optimal" strategy), yielding up to $4\times$ efficiency gains over best-of-N sampling. In FLOPs-matched tests, adding test-time compute to a smaller model can beat a $14\times$ larger model on easy/medium problems (especially when inference tokens $\ll$ pretraining tokens), but the hardest questions still favor more pretraining—so train-time and test-time compute aren't 1-to-1 substitutes."

Chapter 3

Hallucination Detection and Test-Time Compute: Advanced Techniques for LLM Reliability

This chapter provides a comprehensive review of two critical techniques for improving the reliability and performance of Large Language Models (LLMs): hallucination detection and test-time compute optimization. Section 3.1 presents a detailed taxonomy of hallucinations in LLMs, examining their types, causes, detection methods, and mitigation strategies. Section 3.2 explores the emerging paradigm of test-time compute, which enables LLMs to improve their outputs by allocating additional computational resources during inference. Together, these techniques form the foundation for building more reliable and efficient LLM-based systems, particularly in the context of our research on multi-agent causal discovery.

3.1 Hallucination Taxonomy and Detection

3.1.1 Introduction to LLM Hallucinations

Hallucinations in Large Language Models (LLMs) represent one of the most critical challenges facing the deployment of these systems in production environments. A hallucination occurs when an LLM generates content that is factually incorrect, nonsensical, or unfaithful to its input context [**?** **?**]. As LLMs are increasingly deployed in high-stakes domains such as healthcare, legal systems, and scientific research, understanding and mitigating hallucinations has become a priority for the research community.

Recent work by **?**] demonstrates that hallucinations are not merely an artifact of insufficient training data or model capacity, but rather an inherent consequence of the next-token prediction objective. Their analysis shows that "next-token training objectives and common leaderboards reward confident guessing over calibrated uncertainty, so models learn to bluff" [**?**]. This fundamental insight suggests that hallucinations may be theoretically inevitable in current LLM architectures, making detection and mitigation strategies essential rather than optional.

The economic and societal impact of hallucinations is substantial. In enterprise settings, hallucinated information can lead to incorrect business decisions, compliance violations, and reputational damage. In healthcare applications, factual errors can directly impact patient safety [**?**]. Consequently, the development of robust hallucination detection and mitigation techniques is not merely an academic exercise but a practical necessity for responsible AI deployment.

3.1.2 Formal Taxonomy of Hallucinations

3.1.2.1 Core Distinctions: Intrinsic vs. Extrinsic Hallucinations

The hallucination taxonomy literature distinguishes between two fundamental categories based on the source of inconsistency [**?** **?**]:

- **Intrinsic Hallucinations**: These occur when the generated output contradicts the provided input context or prompt. For example, if a model is given a document stating "The company was founded in 2010" and generates "The company has been operating for 20 years" (in 2024), this represents an intrinsic hallucination. Intrinsic hallucinations are particularly problematic in retrieval-augmented generation (RAG) systems, where the model must faithfully represent information from retrieved documents.
- **Extrinsic Hallucinations**: These occur when the generated output is inconsistent with established facts in the real world or the model's training data, regardless of the input context. For instance, stating "Paris is the capital of Germany" represents an extrinsic hallucination. Extrinsic hallucinations are challenging to detect automatically because they require external knowledge sources for verification.

3.1.2.2 Factuality vs. Faithfulness

A complementary dimension of the taxonomy distinguishes between *factuality* and *faithfulness* [**?**]:

- **Factuality**: Refers to the absolute correctness of a statement with respect to verifiable real-world facts. A factually incorrect statement claims something objectively false (e.g., "The Eiffel Tower is 2,000 meters tall").
- **Faithfulness**: Refers to the consistency of the output with the provided input context, regardless of real-world factuality. A model can be faithful but not factual (correctly summarizing a document containing misinformation) or factual but not faithful (providing correct information not present in the source material).

This distinction is particularly relevant for our research on LLM-as-a-judge systems, where we must consider both whether the judge's assessment is faithful to the generated causal loop diagram and whether it correctly identifies factual errors in the causal relationships.

3.1.2.3 Specific Manifestations of Hallucinations

Recent comprehensive surveys [**?** **?** **?**] identify several specific manifestations of hallucinations across different task domains:

1. **Factual Errors**: Direct misstatements of verifiable facts, including incorrect dates, names, numbers, and relationships between entities. These are the most common and easily identifiable hallucinations.
2. **Contextual Inconsistencies**: Generating content that contradicts earlier statements within the same response or conversation. For example, stating "I will provide three reasons" and then listing only two.
3. **Logical Inconsistencies**: Violations of basic logical principles, such as affirming both P and $\neg P$ within the same reasoning chain, or deriving conclusions that do not follow from stated premises.
4. **Temporal Disorientation**: Confusion about temporal relationships, such as describing future events in past tense or anachronistically placing technologies or concepts in inappropriate time periods.
5. **Ethical Violations**: Generating content that violates ethical guidelines or produces harmful stereotypes, though these are sometimes considered a separate category from traditional hallucinations.
6. **Task-Specific Hallucinations**: Domain-specific errors such as:

- *Code Generation*: Invoking non-existent APIs, using deprecated syntax, or generating syntactically invalid code
- *Multimodal Applications*: Describing visual elements not present in images or misidentifying objects
- *Mathematical Reasoning*: Arithmetic errors, incorrect application of formulas, or invalid proof steps
- *Summarization*: Including information not present in the source text (intrinsic hallucination)

In the context of our research on causal discovery, we are particularly concerned with *logical inconsistencies* and *domain-specific hallucinations* related to causal reasoning. For instance, an LLM might hallucinate causal relationships that violate fundamental principles such as acyclicity in causal graphs or might generate causal mechanisms that are logically impossible given domain constraints.

3.1.3 Causes and Contributing Factors

Understanding the root causes of hallucinations is essential for developing effective mitigation strategies. The literature identifies several contributing factors [**?** **?**]:

3.1.3.1 Training Objective Misalignment

The fundamental training objective of modern LLMs—next-token prediction via maximum likelihood estimation—does not explicitly optimize for factuality or truthfulness. As **?**] demonstrate, this objective actively incentivizes confident responses even in cases of uncertainty. When faced with a prompt requiring factual knowledge not clearly represented in the training data, the model is rewarded for producing plausible-sounding text rather than expressing uncertainty or declining to answer.

3.1.3.2 Data Quality and Coverage

Training data inevitably contains errors, biases, and conflicting information. When the model encounters similar patterns during inference, it may reproduce these errors. Additionally, *data coverage gaps* mean that the model lacks sufficient information about certain topics, leading to extrapolation from superficially similar training examples—a process that often produces hallucinations.

3.1.3.3 Sampling and Decoding Strategies

The choice of decoding strategy significantly impacts hallucination rates. Greedy decoding tends to produce more deterministic but potentially overconfident outputs. Temperature sampling with high temperature values increases diversity but may amplify hallucinations by giving non-trivial probability mass to low-likelihood continuations. Nucleus sampling (top-p) and top-k sampling aim to balance these concerns but can still produce hallucinations when the model's probability distribution is poorly calibrated.

3.1.3.4 Context Length and Attention Limitations

Despite advances in long-context models, attention mechanisms can still fail to properly utilize all relevant information from extended contexts. The "lost in the middle" phenomenon [**?**] shows that models disproportionately attend to information at the beginning and end of long contexts, potentially missing crucial facts in the middle that could prevent hallucinations.

3.1.3.5 Instruction Following vs. Truthfulness Trade-offs

Recent work suggests a tension between instruction-following capabilities and truthfulness. Models fine-tuned to be more helpful and responsive may become more prone to hallucinations, as they prioritize providing an answer over acknowledging uncertainty [**?**].

3.1.4 Hallucination Detection Methods

Detecting hallucinations is a challenging task that has spawned a diverse array of approaches. We categorize these methods based on their primary mechanism [**? ?**].

3.1.4.1 Consistency-Based Methods

Consistency-based methods operate on the assumption that hallucinations are random errors that will not be consistently reproduced across multiple independent generations, while correct information will be more stable.

SelfCheckGPT: One of the most influential consistency-based approaches is SelfCheckGPT [**?**], which measures inconsistencies across multiple sampled responses. The intuition is that if an LLM hallucinates, the hallucinated content will vary across samples,

whereas factually correct information will appear consistently. SelfCheckGPT-NLI uses natural language inference to compare sentences from the main response against multiple sampled responses, flagging sentences with low consistency as potential hallucinations. Empirical results show that a SelfCheckGPT score of 0.8 corresponds to approximately 80% recall in identifying hallucinations.

Self-Consistency with Chain-of-Thought: Originally developed for arithmetic reasoning [**?**], self-consistency can also detect hallucinations by sampling multiple reasoning paths and comparing their conclusions. Significant disagreement among reasoning paths suggests potential hallucinations or genuine uncertainty.

Limitations: Consistency-based methods require generating multiple responses at inference time, substantially increasing computational cost (typically 5-10x). Additionally, systematic hallucinations that appear consistently across all samples will not be detected by these methods.

3.1.4.2 Uncertainty-Based Methods

Uncertainty quantification approaches attempt to estimate the model's confidence in its outputs and flag low-confidence generations as potential hallucinations.

Semantic Entropy: Recent work by **?**] introduces semantic entropy as a theoretically grounded uncertainty measure for LLM outputs. Unlike traditional entropy over token sequences, semantic entropy computes entropy over *semantic meanings*, accounting for the fact that multiple token sequences can express the same meaning. This method can detect confabulations—arbitrary incorrect generations—even when the model appears confident at the token level.

Token Probability Analysis: Simple baselines using log-probability of generated tokens or sequences can provide useful signals. Lower average token probability often (but not always) correlates with hallucinations. However, this approach is limited because models can confidently generate hallucinations with high token probabilities.

Hidden State Analysis: Methods that analyze internal representations examine variations in model characteristics between truthful and hallucinated examples. For instance, **?**] show that probing classifiers trained on hidden states can distinguish between truthful and false statements, suggesting that models internally "know" when they are hallucinating, even if this knowledge is not reflected in output probabilities.

3.1.4.3 External Knowledge Verification

These methods verify generated content against external knowledge sources.

Retrieval-Augmented Verification: After generation, retrieve relevant documents and check consistency between the generated text and retrieved information. This approach is effective but requires access to reliable, comprehensive knowledge bases and adds latency to the inference pipeline.

Fact-Checking Databases: For domains with structured knowledge bases (e.g., Wikidata, scientific databases), direct verification against these sources can identify factual errors. However, coverage is limited to domains with well-maintained knowledge graphs.

LLM-as-a-Verifier: Using a separate (often larger or more capable) LLM to verify the factuality of outputs from a target model. This meta-verification approach can be effective but inherits the limitations of the verifier model and introduces additional computational cost and potential cascading errors.

3.1.4.4 Statistical Pattern Analysis

These methods identify statistical patterns associated with hallucinations.

Attention Pattern Analysis: Examining attention weights can reveal when a model is not properly grounding its outputs in the input context. Low attention to source documents in RAG systems, for instance, may indicate generation from parametric memory rather than retrieved facts.

Eigenvalue Analysis: Recent work uses eigenvalue decomposition of internal representations to identify hallucinations, with the hypothesis that hallucinated content exhibits different spectral properties in the model's hidden states [**?**].

3.1.5 Evaluation Metrics and Benchmarks

Rigorous evaluation of hallucination detection methods requires well-defined metrics and benchmarks.

3.1.5.1 Detection Metrics

- **Detection Rate (Sensitivity/Recall)**: The percentage of actual hallucinations correctly identified by the detection method. State-of-the-art methods achieve 76.5% detection rate at 5% false positive rate for GPT-4o [**?**].

- **False Positive Rate (1 - Specificity)**: The percentage of correct outputs incorrectly flagged as hallucinations. High false positive rates can significantly degrade user experience by unnecessarily questioning correct information.
- **AUROC (Area Under ROC Curve)**: Standard metric for binary classification performance, providing a single score summarizing the trade-off between detection rate and false positive rate across all threshold settings.
- **Rejection Accuracy**: In selective prediction settings, the percentage of rejected examples that were actually incorrect, measuring precision of the rejection mechanism.

3.1.5.2 Factuality Benchmarks

Several specialized benchmarks have emerged for evaluating factuality and hallucination detection [**?**]:

- **SimpleQA**: A benchmark of short, fact-seeking queries designed to minimize ambiguity and provide clear ground truth [**?**].
- **HaDes**: A token-level, reference-free annotated dataset for hallucination detection across multiple domains [**?**].
- **Wiki-FACTOR and News-FACTOR**: Benchmarks based on Wikipedia and news articles with factual and counterfactual alternatives for evaluating factuality.
- **Vectara Hallucination Leaderboard**: Compares LLM performance at producing hallucinations when summarizing short documents, computing overall factual consistency rates [**?**].
- **CCHall**: Multimodal hallucination benchmark testing reasoning across images and text [**?**].
- **Mu-SHROOM**: Multilingual hallucination evaluation, addressing the need for factuality assessment beyond English [**?**].

These benchmarks reveal that even state-of-the-art models exhibit substantial hallucination rates, particularly in low-resource languages, multimodal settings, and domains requiring specialized knowledge.

3.1.6 Hallucination Mitigation Strategies

Given the difficulty of completely preventing hallucinations, research has focused on mitigation strategies that reduce their frequency and impact.

3.1.6.1 Retrieval-Augmented Generation (RAG)

RAG systems [**?**] aim to ground model outputs in retrieved external information, significantly reducing extrinsic hallucinations. However, RAG introduces new failure modes [**?**]:

- **Retrieval Failures**: Irrelevant or low-quality retrieved documents can mislead the model or be ignored, falling back to potentially hallucinated parametric knowledge.
- **Context Noise**: Irrelevant information in retrieved documents can distract the model, particularly in long contexts.
- **Context Conflicts**: When retrieved documents contain contradictory information, models may generate inconsistent outputs or arbitrarily select one source.

Advanced RAG Techniques: Recent improvements include contextual re-ranking of retrieved documents, hybrid generation pipelines that mix extractive and abstractive approaches, and citation mechanisms that explicitly link generated claims to source documents, enabling verification and building user trust.

3.1.6.2 Prompt Engineering

Carefully designed prompts can reduce hallucinations:

- **Explicit Grounding Instructions**: Instructing the model to rely only on provided context and to acknowledge when information is unavailable reduces extrinsic hallucinations.
- **Uncertainty Elicitation**: Prompts that encourage the model to express uncertainty ("If you're not certain, say so") can reduce confident hallucinations, though effectiveness varies by model.
- **Chain-of-Thought with Verification**: Asking models to show their reasoning and verify their own answers can catch some hallucinations before they reach the user.

3.1.6.3 Fine-Tuning and Reinforcement Learning

- **Supervised Fine-Tuning on Factual Data**: Training on high-quality, fact-checked data can improve factuality, though gains are often limited by the model's pre-training.
- **Reinforcement Learning from Human Feedback (RLHF)**: Training reward models to penalize hallucinations and reward factual responses. However, this approach must carefully balance helpfulness and truthfulness, as overemphasis on refusing to answer can degrade utility.
- **Direct Preference Optimization (DPO)**: Recent alternatives to RLHF that directly optimize for preferred behaviors, potentially offering more stable training for factuality objectives.

3.1.6.4 Architectural Modifications

- **Attribution Mechanisms**: Models that inherently produce citations or attributions for their claims, enabling verification and discouraging unsupported statements.
- **Calibration Layers**: Additional model components that improve calibration of output probabilities to better reflect true uncertainty.
- **Uncertainty Modeling**: Explicitly modeling epistemic uncertainty to distinguish "known unknowns" from confident knowledge.

3.1.6.5 Post-Processing and Verification

- **Automated Fact-Checking**: Using secondary models or knowledge bases to verify generated content before presenting to users.
- **Rowen**: An adaptive retrieval method that identifies potential hallucinations and corrects factual errors through iterative retrieval [**?**].
- **Human-in-the-Loop**: For high-stakes applications, flagging uncertain outputs for human review before deployment.

3.1.7 Implications for LLM-Based Causal Discovery

In our research context, hallucinations in LLM-generated causal loop diagrams manifest as several distinct failure modes:

1. **Spurious Causal Relationships**: The LLM may generate edges between variables that have no plausible causal connection, drawing on superficial correlations or stereotypical associations rather than genuine causal mechanisms.

2. **Incorrect Causal Direction**: Even when a true causal relationship exists between two variables, the model may reverse the direction of causation.

3. **Missing Causal Links**: Omitting critical causal relationships that are necessary for a complete understanding of the system.

4. **Logically Inconsistent Feedback Loops**: Generating cyclic causal structures that violate domain constraints or produce logical contradictions.

Our use of an LLM-as-a-judge architecture aims to detect these hallucinations by providing a second layer of scrutiny. However, this approach introduces the meta-problem of *judge hallucinations*—the possibility that the judging model itself hallucinates errors that do not exist or fails to detect genuine hallucinations. This concern motivates our investigation of context-insensitive metrics as an independent, complementary validation signal.

The theoretical inevitability of hallucinations in next-token prediction models [**?** **?**] suggests that perfect causal discovery via LLMs may be unattainable. Instead, our research aims to characterize the frequency and types of hallucinations in causal discovery tasks and develop detection strategies that reduce their impact to acceptable levels for practical applications.

3.2 Test-Time Compute: Scaling Inference for Improved Performance

3.2.1 Introduction to Test-Time Compute

Test-time compute (TTC), also referred to as inference-time compute or test-time scaling, represents a paradigm shift in how we think about improving LLM performance. Rather than exclusively focusing on scaling model parameters through larger architectures and more pre-training, TTC explores how allocating additional computational resources *during inference* can enhance output quality [**?** **?**].

The fundamental insight motivating TTC research is that many reasoning tasks benefit from deliberation and exploration of multiple solution paths. Humans do not solve complex problems by immediately outputting a solution; instead, we engage in iterative

refinement, consider alternative approaches, and verify our reasoning. TTC aims to imbue LLMs with similar capabilities by allowing them to "think longer" before producing final outputs.

Recent empirical results demonstrate the remarkable effectiveness of this approach: strategically scaling test-time compute can improve model performance more than scaling model parameters by 14x [**?**]. This finding has profound implications for the economics and practical deployment of LLMs, suggesting that inference-time optimization may offer better returns on computational investment than continued scaling of pre-training.

3.2.2 Theoretical Foundations

3.2.2.1 Compute-Optimal Scaling Strategy

The core theoretical contribution of recent TTC research is the concept of *compute-optimal* scaling [**?**]. This framework recognizes that different problems benefit from different types and amounts of test-time computation. Key principles include:

1. **Problem-Dependent Efficacy**: The effectiveness of various TTC strategies critically depends on problem difficulty. Easy problems may need minimal additional compute, while hard problems benefit substantially from increased inference budget.

2. **Adaptive Allocation**: Rather than uniformly applying the same amount of test-time compute to all prompts, compute-optimal strategies estimate problem difficulty and allocate compute accordingly. This adaptive approach can improve efficiency by 4x compared to uniform allocation [**?**].

3. **Strategy Selection**: Different TTC methods (search, revision, verification) have varying strengths. The optimal strategy depends on the nature of the task and the specific failure modes of the base model.

3.2.2.2 Connection to Meta-Reinforcement Learning

Recent work from CMU [**?**] frames test-time compute optimization as a meta-reinforcement learning problem. From this perspective, TTC methods learn an *algorithm* that figures out how to solve queries at test time. This view provides theoretical grounding for understanding why certain TTC approaches work and suggests principled methods for developing new strategies.

The meta-RL framing implies that test-time compute is not merely about generating more candidates, but about learning an efficient search strategy through the space of possible solutions. This perspective aligns with human problem-solving, where expertise involves not just knowledge, but effective heuristics for exploring solution spaces.

3.2.2.3 Scaling Laws for Test-Time Compute

Traditional neural scaling laws [**?**] characterize the relationship between model size, data size, training compute, and performance. Recent work extends these scaling laws to inference time [**?**], establishing predictable relationships between test-time compute budget and performance improvements.

Key findings include:

- **Power-Law Scaling**: Performance often improves as a power law with respect to inference compute budget, exhibiting diminishing but predictable returns.
- **Model Size Trade-offs**: At fixed compute budgets, using a smaller model with more test-time compute can outperform using a larger model with less test-time compute [**?**]. This suggests that future model development may prioritize inference-time reasoning over parameter counts.
- **Data Efficiency**: As pre-training datasets approach exhaustion of high-quality text, test-time compute offers an alternative path to performance improvements that does not require additional training data [**?**].

3.2.3 Primary Test-Time Compute Mechanisms

3.2.3.1 Best-of-N Sampling

The simplest TTC approach is *best-of-N sampling*: generate N independent responses and select the best according to some scoring function. While conceptually simple, this baseline is surprisingly effective and serves as a reference point for evaluating more sophisticated methods.

Scoring Functions: The effectiveness of best-of-N depends critically on the quality of the scoring function. Options include:

- **Length-normalized log probability**: Selecting the response with highest average token probability

- **Reward models**: Using a separate model trained to score response quality
- **Task-specific verifiers**: For problems with verifiable answers (e.g., code execution, mathematical proofs)

Limitations: Best-of-N sampling is compute-intensive (linear cost in N) and limited by the diversity of samples. If the model consistently makes the same error across all samples, best-of-N cannot recover the correct answer.

3.2.3.2 Chain-of-Thought and Self-Consistency

Chain-of-thought (CoT) prompting [**?**] encourages models to generate intermediate reasoning steps before final answers. Self-consistency [**?**] extends this by sampling multiple CoT reasoning paths and selecting the most common final answer via majority voting.

Empirical results show substantial improvements: self-consistency with CoT improves accuracy by 17.9% on GSM8K (math word problems), 11.0% on SVAMP, and 12.2% on AQuA [**?**]. These gains come from two sources: (1) CoT enables step-by-step reasoning, and (2) self-consistency leverages the principle that correct reasoning paths tend to converge on the same answer, while errors are random.

Recent Advances: Reasoning-Aware Self-Consistency (RASC) improves upon vanilla self-consistency by scoring both answers and reasoning paths, better handling incorrect or irrelevant responses and optimizing sampling efficiency [**?**].

3.2.3.3 Iterative Refinement and Revision

Rather than sampling multiple independent solutions, iterative refinement generates an initial response and then progressively improves it through guided self-correction. This approach mimics human problem-solving, where we iteratively revise our work based on reflection and feedback.

Process:

1. Generate initial response
2. Prompt model to critique its own response, identifying potential errors
3. Generate revised response addressing identified issues
4. Repeat for k iterations or until convergence

Advantages: Refinement can fix errors that would appear consistently in independent samples (systematic errors). It is also more compute-efficient than best-of-N for problems where the initial attempt is close to correct.

Challenges: The model may introduce new errors during revision, or become stuck in revision cycles without convergence. Additionally, the effectiveness of self-critique varies widely across models and task types.

3.2.3.4 Search-Based Methods with Process Reward Models

The most sophisticated TTC approaches use structured search through the space of possible solutions, guided by *process reward models* (PRMs) that evaluate intermediate steps rather than only final outcomes [**?** **?**].

Process Reward Models: Unlike outcome reward models (ORMs) that only score final answers, PRMs provide feedback at each step of a multi-step reasoning trace. This fine-grained feedback enables more effective credit assignment and helps identify where reasoning goes wrong.

PRMs are trained on data annotated with step-level correctness labels. During inference, the PRM scores each reasoning step, allowing search algorithms to prioritize promising paths and prune incorrect ones early. Recent work shows that PRMs significantly outperform ORMs for mathematical reasoning and other multi-step tasks [**?**].

Search Algorithms: Several search strategies can be used with PRMs:

- **Beam Search**: Maintains the top-k partial solutions at each step, expanding the most promising according to PRM scores.
- **Monte Carlo Tree Search (MCTS)**: Builds a tree of possible reasoning paths, using PRM scores to guide exploration. MCTS balances exploitation (pursuing promising paths) and exploration (trying alternative approaches).
- **Best-First Search**: Expands the most promising node in the entire tree at each step, rather than expanding all nodes at a given depth (as in beam search).

Empirical Results: Search-based methods with PRMs achieve state-of-the-art performance on challenging reasoning benchmarks. When combined with compute-optimal allocation strategies, they can outperform much larger models while using less total compute [**?**].

3.2.4 Adaptive Compute Allocation

A critical insight from recent research is that uniform allocation of test-time compute is suboptimal. The *compute-optimal* strategy adaptively allocates inference budget based on estimated problem difficulty [**?**].

3.2.4.1 Estimating Problem Difficulty

Several signals can indicate problem difficulty:

- **Model Uncertainty**: Low token probabilities or high entropy in initial generations suggest the problem is challenging for the model.
- **Self-Consistency Disagreement**: When multiple initial samples produce diverse answers, this indicates uncertainty and suggests benefit from additional compute.
- **Verifier Scores**: Low confidence scores from reward models or verifiers on initial attempts suggest the problem is difficult.
- **Task Metadata**: When available, explicit difficulty labels or problem characteristics (e.g., length, domain) can guide allocation.

3.2.4.2 Allocation Strategies

Given an estimate of problem difficulty, how should compute be allocated? Research suggests:

- **Easy Problems**: Use minimal compute, potentially just greedy decoding or a small number of samples. Revision may be more efficient than sampling for near-correct initial attempts.
- **Moderate Difficulty**: Employ self-consistency with moderate sample counts, or limited search depth with PRMs.
- **Hard Problems**: Allocate substantial compute to deep search or large sample sets, as these problems benefit most from exploration of diverse solution paths.

Empirical results show that adaptive allocation can match the performance of uniform high-compute strategies while using 4x less compute on average [**?**].

3.2.5 Applications and Practical Considerations

3.2.5.1 Enterprise Applications: TAO

Databricks' TAO (Test-time Adaptive Optimization) [**?**] demonstrates practical enterprise applications of TTC. TAO uses test-time compute and reinforcement learning to improve models without requiring labeled training data—a critical advantage for enterprise settings where obtaining high-quality labeled data is expensive or impossible.

Key innovations:

- Uses test-time compute during *training* to generate synthetic preference data
- Achieves better model quality than traditional fine-tuning
- Brings open-source models (Llama) to within the quality of proprietary models (GPT-4o) for specific tasks

3.2.5.2 Cost-Benefit Analysis

While test-time compute improves performance, it increases inference cost and latency. Practical deployment requires careful cost-benefit analysis:

Costs:

- Increased compute expense (proportional to number of forward passes)
- Higher latency (sequential revision or search increases wall-clock time)
- More complex serving infrastructure

Benefits:

- Improved accuracy, potentially reducing downstream costs of errors
- Can use smaller base models, reducing memory requirements
- Dynamic allocation allows cost control via compute budgets

For many applications, spending 5-10x inference compute to substantially improve quality is economically rational, particularly for high-stakes decisions or complex reasoning tasks.

3.2.5.3 Latency Considerations

Latency is critical for interactive applications. Strategies to manage latency include:

- **Parallel Sampling**: Generate multiple candidates concurrently rather than sequentially
- **Speculative Execution**: Begin generating high-probability continuations while still evaluating earlier steps
- **Streaming with Refinement**: Show initial output immediately, then stream refinements
- **Async TTC**: For non-interactive use cases, decouple TTC from user-facing response time

3.2.6 Future Directions and Open Questions

3.2.6.1 Pre-training vs. Inference Compute Trade-offs

As test-time compute demonstrates increasing effectiveness, a fundamental question emerges: should we invest more compute in pre-training larger models or in inference-time reasoning with smaller models? Recent evidence suggests the latter may be more efficient [**?** **?**].

Implications:

- Future LLMs may prioritize inference-time reasoning capabilities over parameter count
- Pre-training objectives might explicitly optimize for test-time improvability
- Research focus may shift toward inference algorithms and reward models

3.2.6.2 Generalization Across Domains

Most TTC research focuses on mathematical reasoning and coding tasks, where objective verification is possible. Key open questions include:

- How effective is TTC for open-ended generation tasks (creative writing, general conversation)?

- Can TTC improve factuality and reduce hallucinations in knowledge-intensive tasks?
- How do TTC benefits generalize across languages and cultural contexts?

Recent work on multi-domain process reward models [**?**] suggests that TTC can extend beyond mathematics, but much work remains to characterize its effectiveness across diverse applications.

3.2.6.3 Learned Inference Algorithms

Rather than hand-crafting inference-time algorithms (search strategies, refinement protocols), can we learn them? The meta-RL framing suggests that inference algorithms themselves could be learned through meta-training, potentially discovering strategies superior to those designed by humans.

3.2.6.4 Integration with Hallucination Detection

Test-time compute offers a natural framework for integrating hallucination detection: use additional inference budget to verify factual claims, check consistency, and refine outputs. The interplay between TTC and hallucination mitigation is an important direction for future research.

3.2.7 Implications for LLM-Based Causal Discovery

In the context of our research, test-time compute principles inform our approach in several ways:

1. **LLM-as-a-Judge**: Our use of a judging model to evaluate generated causal diagrams is conceptually a form of test-time compute—allocating additional inference resources to verify and improve initial outputs.
2. **Iterative Refinement for Causal Discovery**: Rather than accepting the first generated causal loop diagram, test-time compute principles suggest iteratively refining the diagram based on judge feedback. This approach could significantly improve diagram quality while managing computational costs through adaptive allocation.

3. **Selective Judging as Adaptive Compute Allocation**: Our RQ3 investigation into smart test-time compute directly applies compute-optimal principles. By using context-insensitive metrics (analogous to difficulty estimators), we can identify which generated diagrams require judge verification and which can be accepted without additional compute. This adaptive strategy aims to achieve a favorable trade-off between quality and cost.

4. **Multiple Generation Strategies**: Test-time compute research suggests generating multiple candidate causal diagrams and using the judge to select the best, rather than relying on a single generation. The effectiveness of this approach versus iterative refinement is an empirical question we aim to address.

5. **Process Reward Models for Causal Reasoning**: While our current work uses outcome-based judging (evaluating complete diagrams), future extensions could develop process reward models that evaluate intermediate steps in causal reasoning (e.g., "Is this individual causal link plausible?"), enabling more sophisticated search-based approaches.

The test-time compute paradigm provides theoretical grounding for our experimental design and suggests natural extensions for future work. By framing causal discovery as a test-time optimization problem, we can leverage insights from the broader TTC literature to improve both the quality and efficiency of LLM-based causal discovery systems.

3.3 Conclusion

This chapter has provided a comprehensive review of two critical techniques for advancing LLM reliability and performance: hallucination detection and test-time compute. Understanding the taxonomy of hallucinations, their causes, detection methods, and mitigation strategies is essential for deploying LLMs in high-stakes applications. Similarly, test-time compute offers a promising paradigm for improving LLM outputs through strategic allocation of inference resources.

In our research on LLM-based causal discovery, these two threads intersect: hallucinations manifest as errors in generated causal diagrams, while test-time compute principles guide our use of LLM-as-a-judge systems and adaptive verification strategies. The theoretical and empirical insights from this literature inform our methodology and suggest future directions for improving the reliability and efficiency of automated causal discovery.

Chapter 4

Methods: Experimental Setup

We design three complementary experiments to evaluate LLM-driven generation of Causal Loop Diagrams (CLDs). The experiments take place in the context of a CLD building system adhering to the algorithm implemented by Andrew Crossley (see Table 1 for an overview of the algorithm). The system begins with a target variable and user-defined context, iteratively proposing new variables and pairwise relationships, each accompanied by an LLM-generated rationale and cited source. We define a hallucination when the rationale contradicts itself, does not match its cited source, or fails user-specified criteria (e.g., causal relevance, measurability, non-overlapping). Final diagrams are verified against a separate, smarter reasoning LLM (OpenAI o1, for example) or human experts in a later stage if feasible to reduce circularity (i.e., LLMs judging LLMs).

Research Question 1: "Can LLM-as-a-judge and LLM-as-a-corrector method be used to increase final CLD accuracy?"

- RQ 1a: "Can LLM-as-a-judge be used to detect hallucinations?"
- RQ 1b: "Can LLM-as-a-corrector be used to correct hallucinations?"

Experiment 1: LLM-as-a-judge

To address RQ1, we select some known-to-be-correct ground truth CLDs, then create spurious versions by deliberately altering links and providing a spurious rationale. An LLM-as-a-judge checks each link's rationale and source, which flags possible hallucinations. An "LLM-as-a-corrector" revises or removes flagged links. We compare final CLD accuracy and compute cost to a baseline with no verification. We expect the judge/corrector method to yield more accurate final CLDs.

Research Question 2: "Can context insensitive metrics be used to detect hallucinations?"

Experiment 2: Context-insensitive metrics We measure perplexity, minimum probability, and cosine similarity between the generated rationale and its cited text, correlating these metrics with flagged hallucinations. This indicates whether these low-compute cost, context-insensitive measures can effectively predict hallucinations and serve as a type of uncertainty quantification. We expect lower minimum probability and higher perplexity respectively to correspond to higher likelihood of hallucination, and higher cosine similarity with (chunks) of the cited source to indicate lower likelihood of hallucination.

Research Question 3: "Can context-insensitive metrics when used as hallucination indicators be used to deploy test-time compute to correct hallucinations more efficiently?"

Experiment 3: Combined approach (Original Design) We selectively apply LLM-as-a-judge and LLM-as-a-corrector only when context-insensitive metrics exceed certain thresholds—a "smart" test-time compute strategy. We compare final CLD accuracy and compute usage to an always-on approach ("dumb" test-time compute), which applies LLM-as-a-judge and LLM-as-a-corrector on every link and variable rationale, assessing whether targeted interventions reduce hallucinations more efficiently. We expect the smart deployment of test-time compute to increase efficiency and still meaningfully increase final CLD accuracy.

Methodological Pivot: *The original RQ3 design was predicated on two conditions: (1) RQ2 CI metrics generalizing across CLDs, and (2) RQ1b correctors effectively repairing hallucinations. Our findings did not support either condition—CI metrics showed poor cross-CLD generalization, and the corrector exhibited deletion-dominant behavior rather than genuine repair. Given these results, we pivoted RQ3 to an exploratory pilot study:* ***Deep Research validation****, which asks whether automated literature search can validate causal claims. This alternative approach is documented in the standalone methods document and presented as exploratory work. See Discussion (Section* **??***) for detailed rationale.*

4.1 Methods: Implementation

4.1.1 System Architecture and Multi-Agent Orchestration

The experimental platform is implemented as a stateful multi-agent system within the `CausalDiscovery` class (`data_science/modules.py:176--300`). The architecture comprises three specialized LLM agents operating in coordinated phases, with comprehensive performance tracking and persistence:

Generator Agent Iterative variable discovery via structured prompting (`prompts_Nitai_C.yaml:23` produces measurable causal variables with formal definitions and measurement units. Exclusion lists prevent duplication; prompts maintain spatiotemporal context. Inference settings are role-specific and configurable (default: temperature = 0.7, top_p = 1.0).

Corruptor Agent The function `generate_spurious_motivation()` synthesizes semantically plausible but factually incorrect relationship rationales (`prompts_Nitai_C.yaml:200--214`), introducing "subtle but fundamental" causal flaws while preserving scientific plausibility. The corruption rate is configurable in $[0, 1]$. Replacement logic preserves original motivations (`original_motivation`), writes spurious versions (`spurious_motivation`), flags edges (`is_corrupted = True`), and updates edge properties via `update_relationship_properties()` for complete experimental traceability.

Judge Agent Ensemble A heterogeneous ensemble validates relationships via (i) citation-based external knowledge checks (search-provider integration) and (ii) internal consistency analysis driven by `edgeVerifier` prompts. Ensembles support mixed providers and majority voting.

Infrastructure and Monitoring All agent interactions are persisted to Neo4j with ACID guarantees (development binding set in `force_neo4j_fix.py:20--22` to `bolt://localhost:7687`. Performance tracking uses `inference_times` and `api_call_counts` (`modules.py:258--269`) to support cost analyses. When providers expose token log-probabilities, token-level $\log p$ traces are captured for context-insensitive (CI) metrics.

4.1.2 Ground-Truth Dataset Specification

Experiments use standardized Excel-formatted causal loop diagrams (CLDs) in `data_science/ground_t` with three sheets:

- **Variable_definitions**: triplets [name, definition, units] with 15–25 variables per dataset.
- **Variable_links**: [source, target, polarity] relationships with 20–40 edges per dataset.
- **Context**: temporal scales (minutes–years), spatial scales (individual–population), domain metadata.

The loader `excel_cld_loader_v3.py` implements validation for missing sheets, maps polarity (`POSITIVE/NEGATIVE/BALANCING`) to relationship types, and constructs NetworkX graphs with cycle detection for CLD validation.

4.1.3 Advanced Prompting and Chain-of-Thought Integration

Prompt engineering (`data_science/parameter_tuning_experiments/alternative_prompts/`) enforces structured reasoning:

Variable Generation A six-step reasoning protocol requires explicit checks of measurability, compositionality, and temporal/spatial constraints prior to selection. Outputs pass JSON schema validation.

Relationship Discovery Mediation-aware prompting supplies full variable inventories to discourage spurious direct links when intermediaries exist; prompts demand mechanistic causal explanations (not correlations).

Judge Verification A four-point scale (SUPPORTED/PARTIALLY SUPPORTED/CONTRADICTED/UNA with mandatory justification is applied per-citation, followed by multi-round aggregation.

4.1.4 Context-Insensitive Metrics Implementation

The `logit_metrics.py` module provides provider-agnostic uncertainty quantification from token log-probabilities. A standardized pipeline (`_compute_and_trace_metrics_from_logprobs()`, `modules.py:102--130`) normalizes vendor-specific formats.

Perplexity

$$\text{ppl} = \exp\left(-\frac{1}{N}\sum_{i=1}^{N}\log P(t_i)\right),$$

computed via `compute_avg_nll()` and `compute_perplexity()`.

Minimum Token Probability Least-confident token within a rationale, $\min_i P(t_i)$, via `compute_min_prob()`, used to trigger selective judging.

Maximum Window Entropy Sliding-window entropy over top-k token distributions detects localized uncertainty spikes:

$$H = -\sum_{j} P(w_j)\log P(w_j),$$

with default $k = 5$, window $= 5$ (`compute_max_window_entropy()`).

Semantic Alignment (Batched) Cosine similarity between rationale embeddings and citation chunks uses OpenAI `text-embedding-3-small`. Batched computation (`compute_alignment_batch()`, default batch_size = 64) tracks usage via `_EMBED_STATS`. Chunking uses overlapping windows (default 20% overlap; CLI `--ci-chunk-chars`, `--ci-overlap`).

4.1.5 LLM-as-a-Judge Aggregation Framework

Citation Acquisition Missing citations are fetched by `fill_in_missing_citations()` using:

- **Brave** API with trusted-domain filters (`search_engine_copy.py`, `citation_whitelist_copy.py`) up to 5 results capped at 50 words/400 characters.
- **Perplexity** web-search LLM fallback.
- **Hybrid** LLM extraction followed by search validation.

Per-Citation Evaluation `judge_all_edges_with_citations_serial(approach="per_citation_a` yields structured verdicts with explicit rationales.

Multi-Judge Ensembles Pools of 1–5 judges (e.g., GPT-4.1, GPT-4o, Claude-3.7-Sonnet, Perplexity) with ephemeral clients and majority voting; disagreements are logged.

Quantitative Verdict Mapping

SUPPORTED $\rightarrow 1.0$, PARTIALLY SUPPORTED $\rightarrow 0.5$, CONTRADICTED/UNADDRESSED $\rightarrow 0.0$.

Per-edge aggregation For n judges:

$$\text{aggregate\_score} = \frac{1}{n}\sum_{c=1}^{n} \text{score}_c,$$

with secondary LLM aggregation into *Fully/Partially/Not supported*. Both JSON details (`judge_message`) and scalar scores are stored on edges.

4.1.6 Smart Test-Time Compute Allocation

Thresholds from ROC analyses on development data govern selective judge activation. High-uncertainty relationships (high perplexity, low min P, high entropy, low alignment) trigger full judging; high-confidence edges bypass it. Cost-effectiveness is tracked via tokenized usage per component.

4.1.7 Experimental Design and Statistical Framework

Parameter Space Grid configurations (`experiment_28_04_2025_config.yaml`) span:

- Model combinations (Generator/Corruptor/Judge providers).
- Temperature 0.1–1.0 and top-p per role.
- Judge ensembles (1, 3, 5 judges).
- Corruption rates $0.0, 0.15, 0.3$.
- Parallelization (`parallel=True`, `max_workers=3`) with `ThreadPoolExecutor` for $N \times N$ discovery.

Reproducibility UUID session IDs (`self.session_id`) isolate runs; environment bindings use `force_neo4j_fix.py`. Rolling log handlers (`cli/run_experiment.py:27--59`); JSON/CSV parameter capture.

Statistical Analysis `mean_and_95ci_bounds()` computes percentile 95% CIs; cross-run aggregation supports multi-CLD/prompt/parameter comparisons; cost-effectiveness via `inference_times`, `api_call_counts`; stratified evaluations across datasets and support levels.

4.1.8 Output Artifacts and Evaluation Metrics

Edge-Level CSVs Each edge includes:

- Classification: TP/FP/FN/TN vs. ground truth.
- Graph membership flags (session vs. validation).
- Judge outcomes: verdict label, aggregate score, per-citation `judge_message`.
- CI metrics: perplexity, min_prob, max-window entropy, cosine similarity.
- Relationship details: source/target, type, (truncated) motivation, citation URLs.

Summary Statistics Precision/recall/F1 with CIs, stratified by citation support, corruption status, CI quartiles, and graph-theoretic counts (discovered/validation edges, TP/FP/FN/TN).

Export Pipeline Files follow:

`{model}_{search_provider}_judge_{judge_model}_{dataset}_{timestamp}_{filter_type}.{csv|xls`

All runs include session UUIDs, parameter configs, timings, API counts, and error logs.

4.1.9 Implementation Mapping to Research Questions

RQ1: LLM-as-a-Judge and Corrector Efficacy `judge_all_edges_with_citations_serial()` (`"per_citation_aggregate"` and `"all_citations_at_once"`) compares judging strategies. Controlled corruptions via `generate_spurious_motivation()` provide labeled hallucination scenarios. Multi-judge voting quantifies agreement and consensus reliability.

RQ2: CI Metrics as Hallucination Indicators CI metrics (perplexity, min_prob, entropy, alignment) correlate with judge verdicts at edge level. ROC/PR analyses assess predictive validity for selective judging.

RQ3: Smart Test-Time Compute Threshold-triggered judging is contrasted with always-on judging; tokenized usage enables accuracy-per-cost comparisons.

4.1.10 Reproducibility and Scientific Standards

All components adopt explicit versioning (prompts, model specs), comprehensive logging (session UUIDs, parameters), and standardized outputs to facilitate independent replication.

4.1.11 Mathematical Foundations and Algorithmic Complexity

4.1.11.1 Information-Theoretic Framework

Token information content: $I(t_i) = -\log_2 P(t_i)$. Sequence entropy for rationale T:

$$H(T) = -\sum_i P(t_i) \log_2 P(t_i).$$

Cross-entropy between citation-derived semantics p and rationale semantics q:

$$H(p, q) = -\sum_i p(i) \log q(i).$$

Mutual information between CI metrics and judge verdicts:

$$I(\text{CI}, \text{Judge}) = \sum_{c,j} P(c, j) \log_2 \frac{P(c, j)}{P(c)P(j)}.$$

4.1.12 Neo4j Graph database Architecture

4.1.12.1 Schema and Properties

```
CREATE (v:variable {
  name: String,
  definition: String,
  units: String,
  session_id: String,
  target: Boolean,
  deleted: Boolean,
  spatiotemporal: Map
});
```

```
CREATE (s)-[r:POSITIVE|NEGATIVE {
  motivation: String,
  spurious_motivation: String,
  original_motivation: String,
  is_corrupted: Boolean,
  citations: List<String>,
  session_id: String,

  perplexity: Float,
  min_prob: Float,
  max_window_entropy: Float,

  judge_verdict: String,
  aggregate_verdict: String,
  aggregate_score: Float,
  judge_message: String,

  generation_time: Float,
  inference_tokens: Integer
}]->(t);
```

4.1.12.2 Indexes and Performance

```
CREATE INDEX session_variable_index
FOR (v:variable) ON (v.session_id, v.name);

CREATE INDEX relationship_session_index
FOR ()-[r]-() ON (r.session_id);
```

Typical graphs (15–25 variables; 20–40 relations) require $\sim$2 MB per session including provenance metadata.

4.1.13 Enhanced Statistical Methods and Experimental Design

4.1.13.1 Robust Inference

Percentile bootstrap 95% CIs:

```
mu = mean(values); lower = p2.5(values); upper = p97.5(values)
```

Permutation tests quantify inter-judge agreement with statistic

$$\tau = \sum_i \big(\text{observed}_i - \text{expected}_i\big)^2$$

against 10,000 label permutations. Mann–Whitney U quantifies CI-metric effect sizes across corruption conditions:

$$U = R_1 - \frac{n_1(n_1+1)}{2}.$$

4.1.13.2 Multiple Comparisons and Power

Holm–Bonferroni controls FWER at $\alpha = 0.05$ across k CI tests. Benjamini–Hochberg controls FDR in exploratory prompt sweeps. Power for detecting $d = 0.5$ with 80% yields $n \approx 32$ per condition:

$$n = 2\left(\frac{z_{\alpha/2} + z_\beta}{d}\right)^2.$$

4.1.13.3 Bayesian Modeling

Hierarchical models capture variability at edge/session/dataset levels. A weakly-informative prior, aggregate_score $\sim$ Beta$(2,2)$, supports stable estimation. Posterior predictive checks compare simulated vs. observed CI and verdict distributions.

4.1.14 Deep Learning Theory and Embedding Mathematics

Embedding Geometry OpenAI `text-embedding-3-small` vectors lie on $S^{1535} \subset \mathbb{R}^{1536}$. Cosine similarity:

$$\text{sim}(v_1, v_2) = v_1 \cdot v_2 = \|v_1\|\,\|v_2\| \cos\theta.$$

Batching cost scales as $\mathcal{O}(B\,T\,d)$ with batch size B, token count T, and dimension d.

Attention and Representations Multi-head attention weights:

$$A_{h,i,j} = \text{softmax}\left(\frac{(Q_h W_i^Q)(K_h W_j^K)^\top}{\sqrt{d_k}}\right).$$

Layer-wise behaviors: early (syntax/entities), middle (relations/logic), late (discourse/-judgment). Sinusoidal positional encodings preserve structure in enumerated citations.

Similarity Interpretation and Chunking $\cos\theta \in [-1, 1]$ interprets alignment (1 perfect, 0 orthogonal, -1 contradictory). Overlapping chunks with stride s and window w yield overlap $r = (w - s)/w$ (default $r = 0.2$).

4.1.15 Performance Optimization and Systems Engineering

4.1.15.1 Parallel Processing

```
with ThreadPoolExecutor(max_workers=max_workers) as ex:
    futures = {ex.submit(self.process_variable_pair, pair, rate, vars): pair
               for pair in pairs}
```

Empirical scaling: $P = 3$ yields $\sim 2.7\times$ speedup (90% efficiency); $P = 8 \sim 6.2\times$ (78%); $P = 16$ bottlenecks on API rate limits.

4.1.15.2 Rate Limiting and Cost Optimization

```
class RateLimiter:
    def __init__(self, calls_per_minute=60):
        self.calls_per_minute = cpm; self.call_times = []
    def acquire(self):
        now = time.time()
        self.call_times = [t for t in self.call_times if now - t < 60]
        if len(self.call_times) >= self.calls_per_minute:
            time.sleep(60 - (now - self.call_times[0]))
```

Total cost per CLD $C_{\text{total}} = C_{\text{gen}} + C_{\text{judge}} + C_{\text{embed}} + C_{\text{search}}$, with tokenized accounting per component.

4.1.16 Variable Generation Input and Context Inference

To ensure fair evaluation of LLM variable generation capabilities, we distinguish between seeded inputs (derived from validation data) and truly generated outputs. This section describes the information provided to the LLM during variable generation.

4.1.16.1 Target Variable Seeding

Each validation CLD specifies a designated target variable in its Context sheet (e.g., "Depressive Symptoms" for the depression CLD, "Obesity Prevalence" for the obesity CLD). This target variable is extracted and provided to the variable generation system, serving as the anchor around which related causal variables are discovered. The target is created as the first node in the session graph with the property `target=True`.

Formally, given a validation CLD with variable set V_{val} and designated target $t_{\text{val}} \in V_{\text{val}}$, the generation process initializes with $G = \{t_{\text{val}}\}$ and iteratively expands G by querying the LLM for variables causally related to t_{val}.

This seeding approach reflects realistic usage scenarios where domain experts specify an outcome of interest, but introduces a methodological consideration: the target receives an automatic perfect match during evaluation, slightly inflating similarity metrics (see Section 4.1.17.4).

4.1.16.2 Context Inference from Validation Nodes

Rather than providing the LLM with explicit domain descriptions or paper abstracts, we infer contextual framing directly from the validation variable names. This approach tests whether the system can reconstruct appropriate domain context from minimal structural information.

The inference process operates as follows:

Step 1: Extract Validation Variable Names Given validation variables $V_{\text{val}} = \{v_1, v_2, \ldots, v_n\}$, extract the set of variable names $\mathcal{N} = \{\text{name}(v_i)\}_{i=1}^{n}$.

Step 2: LLM-Based Context Inference A separate LLM call (using the same generator model, e.g., GPT-4.1) analyzes $\mathcal{N}$ to infer:

- **Temporal Scale**: Expected duration of causal effects (e.g., "days to weeks", "2–5 years", "decades")
- **Spatial Scale**: System level of operation (e.g., "individual level", "community level", "national level")
- **Domain**: Research field or discipline (e.g., "public health", "environmental science", "economics")

- **Context Description**: Brief 1–2 sentence characterization of the causal system under study

The inference prompt instructs the model to be "specific and evidence-based from the variable names provided," returning structured JSON output.

Step 3: Context List Construction The inferred attributes are formatted into a context list:

$$\mathcal{C} = \{\text{“Domain: ”} + d, \text{“Context: ”} + c, \text{“Temporal Scale: ”} + t, \text{“Spatial Scale: ”} + s\},$$

where d, c, t, and s denote the inferred domain, description, temporal scale, and spatial scale, respectively.

Step 4: Variable Generation with Inferred Context The generator LLM receives a prompt of the form:

```
Target Variable: {target} (measured in {units})
Temporal Scale: {temporal}
Spatial Scale: {spatial}
Context: {context_list}
Name a variable (2-3 words) with a direct causal effect on
the target variable {target}, defined as "{definition}",
over the timeframe {temporal} and spatial scale {spatial}.
```

Critically, the LLM does *not* receive:

- The original research paper or abstract
- Explicit domain expertise or literature context
- The full set of validation variables (only their names for context inference)

This design isolates the LLM's ability to generate domain-appropriate causal variables from minimal structural cues, while maintaining ecological validity through realistic target specification.

4.1.17 Variable Matching Methodology

To evaluate the performance of LLM-generated causal variables against ground truth validation sets, we developed a three-tiered matching framework that captures both exact semantic equivalence and partial similarity.

4.1.17.1 Binary Matching (Stage 1)

We employed batch LLM-based semantic matching to identify exact or near-exact variable correspondences. For each generated variable g and validation variable v, a language model (GPT-5-nano) assessed semantic equivalence and assigned a confidence score $c \in [0, 1]$. Pairs with $c \geq 0.8$ were classified as matches, yielding a binary mapping $M_1 : G \to V$, where G and V denote the sets of generated and validation variables, respectively.

Binary precision, recall, and F1-score were computed as:

$$\begin{aligned} \text{Precision} &= \frac{|M_1|}{|G|}, \\ \text{Recall} &= \frac{|M_1|}{|V|}, \\ \text{F1} &= \frac{2 \cdot (\text{Precision} \cdot \text{Recall})}{\text{Precision} + \text{Recall}}. \end{aligned}$$

4.1.17.2 Cosine Similarity Metrics

To capture semantic similarity beyond binary matching, we computed cosine similarities between all variable pairs using OpenAI's `text-embedding-3-small` model (1536 dimensions). For each generated variable g_i, we identified its maximum cosine similarity to any validation variable: $s(g_i) = \max_{v \in V} \cos(g_i, v)$. Similarly, for each validation variable v_j, $s(v_j) = \max_{g \in G} \cos(g, v_j)$.

Cosine-based metrics were then calculated as:

$$\begin{aligned} \text{Cosine Precision} &= \frac{1}{|G|} \sum_{g_i \in G} s(g_i), \\ \text{Cosine Recall} &= \frac{1}{|V|} \sum_{v_j \in V} s(v_j). \end{aligned}$$

4.1.17.3 Hybrid Matching (Two-Stage Bipartite)

To combine the strengths of both approaches while avoiding double-counting, we implemented a two-stage greedy bipartite matching algorithm:

Stage 1: Variables matched by the LLM (M_1) receive a similarity score of 1.0.

Stage 2: For the remaining unmatched variables ($G' = G \setminus \text{dom}(M_1)$ and $V' = V \setminus \text{range}(M_1)$), we performed greedy bipartite matching based on cosine similarity. To focus on meaningful similarities, we first filter candidate pairs to those above the 90th percentile of all pairwise cosine similarities, establishing an adaptive threshold τ_{90}. Candidate pairs (g, v) with $\cos(g, v) \geq \tau_{90}$ are then sorted by decreasing similarity, and pairs are assigned sequentially, ensuring each variable participates in at most one match. This yields a secondary mapping $M_2 : G' \to V'$ with associated similarity scores $s(g, v)$.

The hybrid score for each generated variable was:

$$h(g) = \begin{cases} 1.0 & \text{if } g \in \text{dom}(M_1), \\ s(g, M_2(g)) & \text{if } g \in \text{dom}(M_2), \\ 0.0 & \text{otherwise.} \end{cases}$$

Hybrid precision and recall were computed as the mean hybrid scores across generated and validation variables, respectively. This approach ensures that Binary $\leq$ Hybrid $\leq$ Cosine, providing a nuanced assessment that rewards both exact matches and partial semantic overlap while maintaining one-to-one correspondence in the matching process.

Key advantages of this methodology:

- **Interpretability**: Binary metrics provide clear success/failure signals.
- **Granularity**: Hybrid metrics capture partial credit for near-misses.
- **Robustness**: Two-stage matching prevents double-counting and maintains theoretical consistency.
- **Scalability**: Batch LLM calls and vectorized cosine computations enable efficient evaluation.

4.1.17.4 Adjusted Metrics for Seeded Variables

As described earlier, the target variable is seeded from the validation CLD and automatically receives a perfect match score of 1.0. While this reflects realistic usage (domain experts specify targets), it introduces a systematic bias in evaluation metrics. To isolate true generation performance, we compute adjusted metrics that exclude the seeded target from both generated and validation sets.

Formally, let t_{seed} denote the seeded target variable. The adjusted variable sets are:

$$\begin{aligned} G_{\text{adj}} &= G \setminus \{t_{\text{seed}}\}, \\ V_{\text{adj}} &= V \setminus \{t_{\text{seed}}\}, \\ M_{\text{adj}} &= \{(g, v) \in M : g \neq t_{\text{seed}} \wedge v \neq t_{\text{seed}}\}. \end{aligned}$$

Adjusted binary metrics are computed as:

$$\begin{aligned} \text{Adj. Precision} &= \frac{|M_{\text{adj}}|}{|G_{\text{adj}}|}, \\ \text{Adj. Recall} &= \frac{|M_{\text{adj}}|}{|V_{\text{adj}}|}, \\ \text{Adj. F1} &= \frac{2 \cdot \text{Adj. Precision} \cdot \text{Adj. Recall}}{\text{Adj. Precision} + \text{Adj. Recall}}. \end{aligned}$$

Similarly, adjusted hybrid metrics exclude the seeded target from hybrid score calculations:

$$\begin{aligned} \text{Adj. Hybrid Precision} &= \frac{1}{|G_{\text{adj}}|} \sum_{g \in G_{\text{adj}}} h(g), \\ \text{Adj. Hybrid Recall} &= \frac{1}{|V_{\text{adj}}|} \sum_{v \in V_{\text{adj}}} h(v), \end{aligned}$$

where $h(g)$ and $h(v)$ are computed as in the standard hybrid approach but restricted to G_{adj} and V_{adj}.

Impact of Adjustment The magnitude of adjustment depends on CLD size. For small CLDs (10–15 variables), removing one seeded variable can shift F1 scores by 5–10 percentage points. For larger CLDs (50+ variables), the impact is negligible ($< 2\%$). In our experiments with CLDs of 10–34 variables, adjusted metrics provide a more conservative and fair assessment of generation capability.

Both unadjusted and adjusted metrics are reported in experimental results, with adjusted metrics serving as the primary comparison for prompt evaluation and system performance assessment.

4.2 RQ2: Context-Insensitive Metrics for Hallucination Detection

This section documents the methodology for Research Question 2: *Can context-insensitive (CI) metrics computed from generator LLM logits be used to detect hallucinations?*

4.2.1 CI Metrics: Mathematical Definitions

Context-insensitive metrics are computed from the generator LLM's token-level log-probabilities during edge generation. These metrics capture model uncertainty without requiring external context or reasoning.

4.2.1.1 Generator Perplexity

Perplexity measures the exponential of average negative log-likelihood over generated tokens, quantifying how "surprised" the model is by its own output.

$$\text{Perplexity} = \exp\left(\frac{1}{n}\sum_{i=1}^{n} -\log P(x_i)\right) \tag{4.1}$$

where:

- n = number of generated tokens in the causal motivation
- $P(x_i)$ = probability of token x_i (from LLM logprobs: $P(x_i) = \exp(\text{logprob}_i)$)

Implementation: `logit_metrics.py:compute_perplexity()`. Average NLL capped at 10.0 to prevent overflow (max perplexity $\approx$ 22,026).

Interpretation: Higher perplexity $\rightarrow$ greater model uncertainty $\rightarrow$ higher hallucination probability.

4.2.1.2 Generator Minimum Probability

Minimum probability identifies the "weakest link" in the generated sequence—the token about which the model was least confident.

$$\text{MinProb} = \min_{i \in \{1,\ldots,n\}} P(x_i) \tag{4.2}$$

where $P(x_i) = \exp(\text{logprob}_i)$ from LLM logprobs.

Implementation: `logit_metrics.py:compute_min_prob()`.

Interpretation: Lower minimum probability → presence of highly uncertain token → higher hallucination probability.

4.2.1.3 Generator Maximum Window Entropy

Maximum window entropy detects localized regions of high uncertainty by computing Shannon entropy over sliding windows of top-k token distributions.

$$\text{MaxWindowEntropy} = \max_{w \in \mathcal{W}} \left(\frac{1}{|w|} \sum_{t \in w} H_k(t) \right) \tag{4.3}$$

where the local entropy at position t over the top-k tokens is:

$$H_k(t) = -\sum_{j=1}^{k} \tilde{P}_j(t) \cdot \ln \tilde{P}_j(t) \tag{4.4}$$

and $\tilde{P}_j(t)$ is the normalized probability over top-k alternatives:

$$\tilde{P}_j(t) = \frac{P_j(t)}{\sum_{m=1}^{k} P_m(t)} \tag{4.5}$$

Parameters: $k = 5$ (top-k tokens), window size = 5 tokens, sliding with stride 1.

Implementation: `logit_metrics.py:compute_max_window_entropy()`. Uses `np.convolve` for rolling average.

Interpretation: Higher max entropy → region where model was uncertain among alternatives → higher hallucination probability.

4.2.1.4 Generator Cosine Similarity

Cosine similarity measures semantic alignment between the generated causal motivation and fetched citation text chunks.

$$\text{CosineSim} = \max_{c \in \mathcal{C}} \frac{\mathbf{e}_{\text{narr}} \cdot \mathbf{e}_c}{\|\mathbf{e}_{\text{narr}}\| \cdot \|\mathbf{e}_c\|} \tag{4.6}$$

where:

- $\mathbf{e}_{\text{narr}} \in \mathbb{R}^d$ = embedding of generated causal narrative (motivation)
- $\mathbf{e}_c \in \mathbb{R}^d$ = embedding of citation chunk c
- $\mathcal{C}$ = set of all citation chunks (up to 5 articles, chunked with 20% overlap)
- $d = 1536$ for `text-embedding-3-small` (OpenAI)

Implementation: `logit_metrics.py:compute_alignment_batch()`. Chunking: ∼2000 characters per chunk, 20% overlap.

Interpretation: Higher cosine similarity was *expected* to indicate better grounding (lower hallucination). **Counterintuitive finding:** Higher similarity correlates with *more* hallucinations (see Section **??**).

4.2.1.5 Summary of CI Metrics

Metric	Formula	Expected	Observed
Perplexity	$\exp\left(\frac{1}{n}\sum -\log P(x_i)\right)$	↑⇒ halluc	↑⇒ halluc
Min Prob	$\min_i P(x_i)$	↓⇒ halluc	↓⇒ halluc
Max Window Entropy	$\max_w \text{mean}(H_k(t))$	↑⇒ halluc	↑⇒ halluc
Cosine Similarity	$\max_c \cos(\mathbf{e}_{\text{narr}}, \mathbf{e}_c)$	↓⇒ halluc	↑⇒ halluc*

TABLE 4.1: CI metrics summary. *Counterintuitive: higher similarity predicts *more* hallucinations.

4.2.2 Statistical Analysis Methods

This section documents the statistical methodology employed to evaluate whether CI metrics predict hallucinations in LLM-generated causal discovery outputs. All analyses were implemented in Python 3.11.x using standard scientific computing libraries and employ rigorous validation with multiple comparison correction.

4.2.3 Study Design and Data Structure

4.2.3.1 Experimental Data Structure

The RQ2 analysis operates on **63 deduplicated experimental files** from two experiment types:

Experiment Type	CLDs	Runs	Prompts	Files
Citation (GT Lit)	3	3	4	36
Correctness (GT Lit)	3	3	3	27
Total				**63**

TABLE 4.2: RQ2 experimental file structure. Each file contains one complete CLD generation.

Causal Loop Diagrams (CLDs).

- **Depressive symptoms**: Response to stressor in healthy adults (∼210 edges/file)
- **Social norms**: Social norms and obesity prevalence (∼110 edges/file)
- **Emergency department**: Older persons ED visits and interactions (∼1190 edges/file)

Prompt Types.

- **Citation experiment (4 prompts):** baseline, cot, mechanistic_original, mechanistic_lit
- **Correctness experiment (3 prompts):** baseline, cot, mechanistic

Runs. Three independent runs (run_1, run_2, run_3) per (CLD × prompt) combination, using different LLM generation seeds.

Total Sample Size. After pooling all 63 files: ∼31,700 total edges. After removing rows with missing CI metrics: ∼16,500 edges (52.1% retention). Hallucination rate: ∼15% (2,514 hallucinations, 13,993 correct).

4.2.3.2 Cross-CLD Validation Design

The analysis employed a multi-dataset validation design to assess generalizability:

- **Primary dataset structure:** 3 Causal Loop Diagrams (CLDs) from the health domain
  - CLD 1: Depressive symptoms in response to a stressor (health psychology)
  - CLD 2: Social norms and obesity prevalence (health behavior)
  - CLD 3: Older persons emergency department visits (healthcare systems)
- **Replication structure:** 3 independent runs per CLD × prompt combination
- **Total files:** 63 deduplicated experimental files
- **Meta-analytic approach:** One-sample t-tests aggregating performance metrics across files

4.2.3.3 Ground Truth Definition

Hallucination labeling. For GT Lit experiments (citation and correctness), hallucinations are defined as:

$$\text{is\_hallucination} = \mathbb{I}(\text{Classification} = \text{FP}) \vee \mathbb{I}(\text{Classification} = \text{FN}) \tag{4.7}$$

where:

- **False Positive (FP):** Edge generated by LLM but absent from validation CLD
- **False Negative (FN):** Edge present in validation CLD but not generated by LLM

Data availability. CI metrics (perplexity, min_prob, max_window_entropy, cosine_similarity) computed only for edges with LLM-generated motivation text. For FN edges (missed by generator), no generation occurred, hence no logprobs available—these are excluded from individual CI metric analysis but counted in classification performance.

Note on corruption experiments. GT Synth (corruption) experiments are excluded from RQ2 analysis because corruption is inserted *post-generation*. Generator CI metrics (computed during generation) cannot retroactively detect corruption inserted afterwards.

4.2.4 Statistical Test Procedures

4.2.4.1 Correlation Analysis

Pearson Product-Moment Correlation. For metric values $X = (x_1, \ldots, x_n)$ and binary hallucination labels $Y = (y_1, \ldots, y_n)$:

$$r = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2 \sum_{i=1}^{n}(y_i - \bar{y})^2}}$$

where $\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$ and $\bar{y} = \frac{1}{n}\sum_{i=1}^{n} y_i$.

Hypothesis test: Two-tailed, $H_0 : \rho = 0$, $\alpha = 0.05$
Assumptions: Bivariate normality (relaxed for large n via CLT)

Spearman Rank Correlation. Non-parametric alternative robust to non-linearity and outliers:

$$\rho_s = 1 - \frac{6\sum_{i=1}^{n} d_i^2}{n(n^2 - 1)}$$

where $d_i = \text{rank}(x_i) - \text{rank}(y_i)$.

Use case: Validate Pearson results when distributional assumptions violated

4.2.4.2 Classification Performance Analysis

Receiver Operating Characteristic (ROC) Analysis. For each threshold t applied to CI metric X:

$$\text{TPR}(t) = \frac{\text{TP}(t)}{\text{TP}(t) + \text{FN}(t)}, \quad \text{FPR}(t) = \frac{\text{FP}(t)}{\text{FP}(t) + \text{TN}(t)}$$

Area under ROC curve quantifies overall discriminative ability:

$$\text{AUC} = \int_0^1 \text{TPR}(\text{FPR}^{-1}(x))\, dx$$

Implementation: Standard ROC curve algorithm computes (FPR, TPR) pairs at all unique threshold values; AUC computed via trapezoidal rule integration.

Interpretation scale:

- AUC ¿ 0.9: Excellent discrimination

- 0.8–0.9: Good
- 0.7–0.8: Acceptable
- 0.6–0.7: Poor
- ¡ 0.6: Fail (no better than chance)
- AUC = 0.5: Chance performance (random classifier)

Optimal Threshold Selection. Thresholds selected to maximize F_1 score:

$$F_1 = \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}} = \frac{2\text{TP}}{2\text{TP} + \text{FP} + \text{FN}}$$

Rationale: F_1 balances precision (minimize false alarms) against recall (minimize missed hallucinations), appropriate for imbalanced classes.

Implementation: Grid search over percentile-based thresholds:

- Candidate thresholds: $t \in \{\text{percentile}_{10}, \text{percentile}_{15}, \ldots, \text{percentile}_{90}\}$ of metric distribution
- For each threshold t, compute F_1(t) based on predicted labels
- Select optimal: $t^* = \arg\max_t F_1(t)$

This percentile-based grid search ensures coverage across the metric's empirical distribution while avoiding extreme values that may lead to degenerate classifications (e.g., all positive or all negative predictions).

4.2.4.3 Permutation Tests

Purpose: Permutation testing provides *non-parametric* validation of correlation significance without assuming a specific distribution (e.g., normality). This addresses the limitation that Pearson's p-values assume bivariate normality, which may be violated for CI metrics.

Null Hypothesis: H_0: metric values and hallucination labels are independent (no association).

Algorithm.

1. Compute observed correlation: $r_{\text{obs}} = \text{corr}(X, Y)$
2. For $i = 1, \ldots, n_{\text{perm}}$ where $n_{\text{perm}} = 10{,}000$:
   (a) Randomly permute labels: $Y^{(i)} = \text{permute}(Y)$
   (b) Compute permuted correlation: $r_{\text{perm},i} = \text{corr}(X, Y^{(i)})$
3. Compute two-tailed p-value:

$$p = \frac{1}{n_{\text{perm}}} \sum_{i=1}^{n_{\text{perm}}} \mathbb{I}(|r_{\text{perm},i}| \geq |r_{\text{obs}}|)$$

where $\mathbb{I}(\cdot)$ is the indicator function.

Null distribution: Under H_0, permuted correlations form empirical null distribution centered at zero.

Advantages over parametric tests:

- No distributional assumptions
- Exact Type I error control (conditional on data)
- Robust to outliers and skewness

Obtained Results. Single-CLD analysis (Depressive Symptoms, n=182):

- **perplexity**: $r_{\text{obs}} = +0.331$, $p_{\text{perm}} < 0.0001$ (10,000/10,000 permutations had $|r| < |r_{\text{obs}}|$)
- **max_window_entropy**: $r_{\text{obs}} = +0.327$, $p_{\text{perm}} < 0.0001$
- **min_prob**: $r_{\text{obs}} = -0.175$, $p_{\text{perm}} = 0.0144$
- **cosine_similarity**: $r_{\text{obs}} = +0.293$, $p_{\text{perm}} < 0.0001$

All generator CI metrics demonstrated p-values below $\alpha = 0.05$, confirming associations not attributable to chance (`rq2_permutation_tests.csv`).

Multiple Comparison Correction. Testing $k = 4$ generator CI metrics inflates family-wise error rate (FWER). We applied Holm-Bonferroni sequential correction:

For ordered p-values $p_{(1)} \leq p_{(2)} \leq \cdots \leq p_{(k)}$, reject $H_{0,(i)}$ if:

$$p_{(i)} \leq \frac{\alpha}{k - i + 1}$$

Implementation: Sequential Holm-Bonferroni method with $\alpha = 0.05$

FWER control: Guarantees $P(\text{any false rejection}) \leq \alpha$ under global null.

Correction results: All 4 generator CI metrics remained significant after Holm-Bonferroni correction, confirming robust family-wise significance control:

- perplexity: $p_{\text{corrected}} < 0.0001$
- max_window_entropy: $p_{\text{corrected}} < 0.0001$
- cosine_similarity: $p_{\text{corrected}} < 0.0001$
- min_prob: $p_{\text{corrected}} = 0.0144$

4.2.4.4 Bootstrap Confidence Intervals

Purpose: Bootstrap provides *uncertainty quantification* for AUC estimates. Unlike parametric methods (e.g., DeLong's method), bootstrap makes minimal assumptions about the AUC's sampling distribution, making it robust when assumptions are violated or sample sizes are moderate.

Value: Confidence intervals allow us to distinguish between "AUC significantly better than chance (0.5)" versus "AUC estimate with large uncertainty overlapping 0.5," providing evidence strength beyond point estimates alone.

Algorithm ($n_{\text{boot}} = 10{,}000$ resamples):

1. For $b = 1, \ldots, n_{\text{boot}}$:
   (a) Resample n edges with replacement: $(X^{(b)}, Y^{(b)})$
   (b) Compute $\text{AUC}^{(b)}$ on bootstrap sample

2. Compute percentile 95% CI:

$$\text{CI}_{95\%} = [\text{AUC}_{2.5\%}, \text{AUC}_{97.5\%}]$$

where percentiles computed from empirical bootstrap distribution $\{\text{AUC}^{(1)}, \ldots, \text{AUC}^{(n_{\text{boot}})}\}$

Stratified resampling: Maintains class proportions in each bootstrap sample to avoid degenerate cases with single class.

Bootstrap standard error:

$$\text{SE}_{\text{boot}} = \sqrt{\frac{1}{n_{\text{boot}} - 1} \sum_{b=1}^{n_{\text{boot}}} \left(\text{AUC}^{(b)} - \overline{\text{AUC}}\right)^2}$$

Obtained Results. Single-CLD analysis (Depressive Symptoms, n=182):

- **max_window_entropy**: AUC = 0.762, 95% CI [0.680, 0.836], $\text{SE}_{\text{boot}} = 0.040$
  - CI excludes 0.5 → significantly better than chance
  - Narrow CI indicates stable, reliable estimate
- **perplexity**: AUC = 0.747, 95% CI [0.657, 0.829], $\text{SE}_{\text{boot}} = 0.044$
  - CI excludes 0.5 → significantly better than chance
- **min_prob**: AUC = 0.602, 95% CI [0.505, 0.694], $\text{SE}_{\text{boot}} = 0.048$
  - CI barely excludes 0.5, modest performance
- **cosine_similarity**: AUC = 0.224, 95% CI [0.149, 0.305], $\text{SE}_{\text{boot}} = 0.040$
  - CI entirely below 0.5 → worse than chance (inverted predictor)

Bootstrap distributions were approximately normal for all metrics (`rq2_bootstrap_ci.csv`, visualizations: `rq2_bootstrap_distributions.png`).

4.2.4.5 Group Comparisons

Independent samples t-tests compared CI metric distributions between hallucination and non-hallucination groups.

Test statistic:

$$t = \frac{\bar{x}_{\text{halluc}} - \bar{x}_{\text{non-halluc}}}{s_p \sqrt{\frac{1}{n_{\text{halluc}}} + \frac{1}{n_{\text{non-halluc}}}}}$$

where s_p is pooled standard deviation:

$$s_p = \sqrt{\frac{(n_{\text{halluc}} - 1)s^2_{\text{halluc}} + (n_{\text{non-halluc}} - 1)s^2_{\text{non-halluc}}}{n_{\text{halluc}} + n_{\text{non-halluc}} - 2}}$$

Degrees of freedom: $\nu = n_{\text{halluc}} + n_{\text{non-halluc}} - 2$

Hypothesis: $H_0 : \mu_{\text{halluc}} = \mu_{\text{non-halluc}}$ vs. $H_1 : \mu_{\text{halluc}} \neq \mu_{\text{non-halluc}}$

Effect Size. Cohen's d quantifies magnitude of group differences standardized by pooled variance:

$$d = \frac{\bar{x}_{\text{halluc}} - \bar{x}_{\text{non-halluc}}}{s_p}$$

Interpretation guidelines [**?**]:

- Small effect: $|d| \approx 0.2$
- Medium effect: $|d| \approx 0.5$
- Large effect: $|d| \approx 0.8$

4.2.5 Meta-Analysis Framework

4.2.5.1 Cross-CLD Aggregation

Meta-analytic statistics computed across $n_{\text{CLD}} = 3$ independent datasets with 2 runs each.

Mean and Standard Deviation. For statistic S (correlation r or AUC):

$$\bar{S} = \frac{1}{n_{\text{analyses}}} \sum_{j=1}^{n_{\text{analyses}}} S_j, \quad s_S = \sqrt{\frac{1}{n_{\text{analyses}} - 1} \sum_{j=1}^{n_{\text{analyses}}} (S_j - \bar{S})^2}$$

Standard deviation interpretation: Quantifies cross-CLD stability. Low s_S (¡ 0.1) indicates consistent performance; high s_S (¿ 0.15) suggests domain-specific behavior.

Meta-Analytic Hypothesis Tests. **Correlation test:** $H_0 : \mu_r = 0$ (no relationship)

$$t = \frac{\bar{r}}{s_r / \sqrt{n_{\text{analyses}}}}, \quad \nu = n_{\text{analyses}} - 1$$

AUC test: $H_0 : \mu_{\text{AUC}} = 0.5$ (chance performance)

$$t = \frac{\overline{\text{AUC}} - 0.5}{s_{\text{AUC}} / \sqrt{n_{\text{analyses}}}}, \quad \nu = n_{\text{analyses}} - 1$$

Implementation: One-sample t-test comparing sample mean to null hypothesis value.

Confidence Intervals for Meta-Statistics.

$$\bar{S} \pm t_{\alpha/2,\nu} \cdot \frac{s_S}{\sqrt{n_{\text{analyses}}}}$$

For 95% CI with $n_{\text{analyses}} = 32$, $t_{0.025,31} \approx 2.04$, but we use normal approximation $z_{0.025} = 1.96$ for large samples:

$$\text{CI}_{95\%} = \left[\bar{S} - 1.96 \frac{s_S}{\sqrt{n_{\text{analyses}}}}, \bar{S} + 1.96 \frac{s_S}{\sqrt{n_{\text{analyses}}}} \right]$$

Obtained Meta-Analysis Results. Aggregated across 32 analyses (3 CLDs × 2 runs/CLD):

Correlation meta-statistics (testing $H_0 : \mu_r = 0$):

- **cosine_similarity**: $\bar{r} = +0.256$ [+0.232, +0.280], $p < 0.001$
- **max_window_entropy**: $\bar{r} = +0.211$ [+0.175, +0.247], $p < 0.001$
- **perplexity**: $\bar{r} = +0.200$ [+0.146, +0.254], $p < 0.001$
- **min_prob**: $\bar{r} = -0.162$ [-0.177, -0.147], $p < 0.001$

AUC meta-statistics (testing $H_0 : \mu_{\text{AUC}} = 0.5$):

- **perplexity**: $\overline{\text{AUC}} = 0.679$ [0.650, 0.709], $s = 0.076$, $p < 0.001$
  - Consistently above chance, stable across domains
- **max_window_entropy**: $\overline{\text{AUC}} = 0.667$ [0.639, 0.695], $s = 0.072$, $p < 0.001$
  - Consistently above chance, stable across domains

- **min_prob**: $\overline{\text{AUC}} = 0.641$ [0.617, 0.664], $s = 0.060$, $p < 0.001$
  - Low variance ($s < 0.1$) indicates high stability
- **cosine_similarity**: $\overline{\text{AUC}} = 0.230$ [0.214, 0.245], $s = 0.040$, $p < 0.001$ vs. 0.5
  - Significantly *worse* than chance (inverted predictor)

All top-performing metrics demonstrated statistically significant mean effects that were consistent across multiple independent CLDs (`rq2_aggregate_report_*.md`, `aggregate_meta_statisti`

4.2.6 Assumptions and Diagnostics

4.2.6.1 Parametric Test Assumptions

Normality. CI metrics exhibit right-skewed distributions (perplexity, max window entropy). However:

- **Correlation tests:** Large sample sizes ($n > 100$ per CLD) invoke Central Limit Theorem, relaxing normality requirements.
- **t-tests:** CLT justifies approximate normality of sampling distributions for group means.
- **Validation:** Spearman rank correlation provides non-parametric alternative; permutation tests require no distributional assumptions.

Homoscedasticity. Equal variance assumption for t-tests checked via Levene's test. Violations addressed using Welch's t-test (unequal variances):

$$t = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{s_1^2/n_1 + s_2^2/n_2}}, \quad \nu \approx \frac{(s_1^2/n_1 + s_2^2/n_2)^2}{(s_1^2/n_1)^2/(n_1-1) + (s_2^2/n_2)^2/(n_2-1)}$$

Independence.

- **Within-CLD:** Edges sharing variables may exhibit dependencies. Violates strict independence for correlation tests. Conservative interpretation: treat significance thresholds as approximate.
- **Between-CLD:** Independent datasets ensure valid meta-analytic aggregation.

4.2.6.2 Multiple Testing Considerations

Total hypothesis tests conducted:

- 9 metrics × 2 correlation types (Pearson, Spearman) = 18 correlation tests per CLD
- 9 metrics × 1 AUC test per CLD
- 9 metrics × 1 group comparison per CLD

Correction strategy:

- **Primary analysis:** Holm-Bonferroni correction applied to 4 generator metrics (core hypotheses)
- **Exploratory analysis:** Judge metrics and aggregate score treated as exploratory; p-values reported without correction but interpreted cautiously

Recommendation: Results with $p < 0.01$ robust to multiple testing; borderline results ($0.01 < p < 0.05$) require independent validation.

4.2.7 Computational Implementation

4.2.7.1 Software Environment

- **Python:** 3.11.x
- **Core statistical libraries:** SciPy 1.11.x (statistical tests), scikit-learn 1.3.x (classification metrics), NumPy 1.25.x (numerical operations), pandas 2.0.x (data manipulation), statsmodels 0.14.x (multiple comparison correction)

4.2.7.2 Analysis Pipeline Architecture

Three-stage pipeline orchestrates statistical analyses:

Stage 1: Data Preparation.

- Loads experiment outputs and normalizes data format
- Defines binary hallucination labels based on ground truth or judge verdict

- Selects CI metric source (generator vs. judge metrics)
- Exports prepared data for downstream analysis

Stage 2: Single-CLD Analysis.

- **Correlation module:** Pearson and Spearman correlations with p-values
- **Classification module:** ROC curves, AUC computation, optimal threshold selection
- **Validation module:** Permutation tests ($n = 10{,}000$), bootstrap CI ($n = 10{,}000$)
- **Comparison module:** Independent t-tests, Cohen's d effect sizes
- **Outputs:** CSV tables (correlations, AUC scores, thresholds, validation statistics)

Stage 3: Meta-Analysis Aggregation.

- Aggregates results across multiple CLDs
- Computes meta-statistics (mean, std, 95% CI)
- One-sample t-tests: correlation $\neq 0$, AUC ¿ 0.5
- Identifies stable metrics (low cross-CLD variance)
- Generates aggregate visualizations (heatmaps, forest plots)

4.2.7.3 Reproducibility Specifications

Random seed control. Stochastic procedures (permutation sampling, bootstrap resampling) employ explicit random seeds for reproducibility. Explicit seed values ensure exact replication across independent runs.

Computational cost. Approximate runtime on standard hardware (Intel i7, 16GB RAM):

- Single-CLD analysis (4 generator metrics, $n_{\text{perm}} = n_{\text{boot}} = 10{,}000$): 5–10 minutes
- Meta-analysis aggregation (3 CLDs, 32 analyses): 30–60 minutes total
- Memory footprint: ¡ 2GB for largest CLD ($N \approx 1100$ edges)

Parallelization. Permutation and bootstrap loops executed serially. Potential speedup via `joblib.Parallel` or `multiprocessing.Pool` (not implemented in current version).

4.2.8 Statistical Power and Sample Size

Correlation detection power. For two-tailed correlation test at $\alpha = 0.05$, power to detect medium effect ($r = 0.3$) with 80% power:

$$n \approx 84 \text{ edges}$$

All three CLDs exceed this minimum (Depressive symptoms: $N = 182$, Social norms: $N = 90$, Emergency visits: $N = 1119$), providing adequate power for medium effects.

Meta-analysis power. With 3 CLDs and multiple runs per CLD ($n_{\text{analyses}} = 32$ for some metrics), meta-analytic tests achieve high power (¿ 90%) for detecting consistent effects. However, small CLD count limits power for detecting heterogeneity (domain-specific effects).

Limitations. Insufficient power for detecting small effects ($r < 0.2$) or weak classification performance (AUC ¡ 0.6). Null findings should be interpreted as "insufficient evidence" rather than "evidence of absence."

4.2.9 Summary

This statistical framework combines classical parametric methods (correlation, t-tests) with robust non-parametric validation (permutation tests, bootstrap CI) and cross-dataset meta-analysis. Holm-Bonferroni correction controls family-wise error rate across primary hypotheses. Implementation in standard Python scientific stack ensures reproducibility and extensibility. Key methodological strengths include empirical null distributions (permutation testing), assumption-free uncertainty quantification (bootstrap), and hierarchical validation (single-CLD $\rightarrow$ meta-analysis). Primary limitation: small CLD count ($n = 3$) restricts power for heterogeneity analyses and domain stratification.

Chapter 5

Global Sensitivity Analysis

5.1 Introduction

Experimental studies in machine learning and natural language processing often involve multiple hyperparameters and configuration choices whose effects on system performance are not fully understood. While grid search and random search explore the parameter space to find optimal configurations, they do not reveal *which* parameters matter most, *how* parameters interact, or whether certain parameters can be safely fixed without sacrificing performance. Global sensitivity analysis (GSA) addresses these questions by systematically quantifying the influence of input parameters on output variance.

This chapter presents a comprehensive global sensitivity analysis of the hallucination detection experiments described in Chapters **??** and **??**. We employ Sobol sensitivity analysis [**?** **?**], a variance-based method that decomposes output uncertainty into contributions from individual parameters and their interactions. This analysis serves three critical purposes:

1. **Parameter Prioritization**: Identifying which experimental parameters most strongly influence hallucination detection accuracy, precision, and recall, thereby guiding future optimization efforts.
2. **Interaction Detection**: Revealing whether parameters operate independently or exhibit significant interactions, which has implications for optimization strategies.
3. **Model Simplification**: Determining which parameters have negligible influence and can be fixed to reduce experimental complexity and computational cost.

The remainder of this chapter is organized as follows: Section 5.2 provides the theoretical foundation of variance-based sensitivity analysis and Sobol indices. Section 5.3

describes our experimental design, including parameter selection, sampling strategy, and computational implementation. Section 5.5 presents the sensitivity analysis results for each research question. Finally, Section 5.6 discusses the implications of these findings for hallucination detection system design.

5.2 Theoretical Background

5.2.1 Global Sensitivity Analysis

Sensitivity analysis examines how uncertainty in model outputs can be attributed to different sources of uncertainty in model inputs [**?**]. Consider a computational model $Y = f(\mathbf{X})$, where Y is a scalar output and $\mathbf{X} = (X_1, X_2, \ldots, X_D)$ is a vector of D input parameters. The goal of global sensitivity analysis is to quantify the contribution of each X_i (and interactions among parameters) to the total variance in Y.

Unlike local sensitivity analysis methods, which examine the effect of small perturbations around a nominal point (e.g., partial derivatives), global sensitivity analysis explores the entire parameter space. This is crucial when:

- The model is non-linear or non-monotonic
- Parameters may interact in complex ways
- The full range of plausible parameter values must be considered
- No single "baseline" parameter configuration is known a priori

These conditions all hold for LLM-based hallucination detection systems, where temperature parameters influence generation stochasticity, the number of judges affects consensus mechanisms, and various thresholds interact in determining classification outcomes.

5.2.2 Variance-Based Sensitivity Analysis

Variance-based methods [**?**] decompose the total variance of the output into contributions from individual inputs and their interactions. Given that input parameters $X_1, \ldots, X_D$ are independent random variables, the variance of the output can be decomposed using the ANOVA (Analysis of Variance) framework:

$$V(Y) = \sum_{i=1}^{D} V_i + \sum_{i<j} V_{ij} + \cdots + V_{1,2,\ldots,D} \tag{5.1}$$

where:

- $V_i = V[E(Y \mid X_i)]$ is the variance contribution from parameter X_i alone (first-order effect)
- $V_{ij} = V[E(Y \mid X_i, X_j)] - V_i - V_j$ is the variance contribution from the interaction between X_i and X_j (second-order effect)
- Higher-order terms represent interactions among three or more parameters

This decomposition is unique when the input parameters are independent [**?**].

5.2.3 Sobol Indices

Sobol indices [**?**] are normalized variance contributions that quantify the importance of each parameter and their interactions. For parameter X_i, we define:

5.2.3.1 First-Order Sensitivity Index

The first-order Sobol index S_i measures the main effect of parameter X_i:

$$S_i = \frac{V_i}{V(Y)} = \frac{V[E(Y \mid X_i)]}{V(Y)} \tag{5.2}$$

S_i represents the expected reduction in output variance if X_i were known with certainty, without considering interactions. It quantifies the *direct* or *additive* contribution of X_i to output uncertainty.

5.2.3.2 Total-Effect Sensitivity Index

The total-effect Sobol index S_T^i measures the total contribution of parameter X_i, including all interactions involving X_i:

$$S_T^i = 1 - \frac{V[E(Y \mid \mathbf{X}_{\sim i})]}{V(Y)} = \frac{E[V(Y \mid \mathbf{X}_{\sim i})]}{V(Y)} \tag{5.3}$$

where $\mathbf{X}_{\sim i}$ denotes all parameters except X_i. S_T^i represents the expected remaining variance if all parameters except X_i were known. It quantifies the parameter's importance when accounting for interactions with other parameters.

5.2.3.3 Interpretation

The relationship between S_i and S_T^i reveals the nature of a parameter's influence:

- $S_i \approx S_T^i$: The parameter operates independently with minimal interactions
- $S_i < S_T^i$: The parameter exhibits significant interactions with others; the difference $(S_T^i - S_i)$ quantifies the interaction strength
- $S_i \approx 0, S_T^i > 0$: The parameter's influence is entirely through interactions
- $S_T^i \approx 0$: The parameter is non-influential and can be fixed without affecting output variance

By convention, parameters with $S_T^i \geq 0.1$ are considered influential, while those with $S_T^i < 0.05$ are considered negligible [**?**].

5.2.3.4 Second-Order Indices

Second-order Sobol indices S_{ij} quantify pairwise interactions:

$$S_{ij} = \frac{V_{ij}}{V(Y)} = \frac{V[E(Y \mid X_i, X_j)] - V_i - V_j}{V(Y)} \tag{5.4}$$

A large S_{ij} indicates that parameters X_i and X_j interact strongly, meaning their combined effect on Y cannot be understood by examining each parameter independently. This has important implications for optimization: parameters with strong interactions should be tuned jointly rather than sequentially.

5.2.3.5 Mathematical Properties

Sobol indices satisfy important properties:

1. **Normalization**: $\sum_{i=1}^{D} S_i + \sum_{i<j} S_{ij} + \cdots = 1$ (the indices sum to unity)
2. **Bounded**: $0 \leq S_i \leq 1$ and $0 \leq S_T^i \leq 1$ for all i
3. **Relationship**: $S_T^i \geq S_i$ for all i (total effect is always at least as large as first-order effect)

5.2.4 Saltelli Sampling Method

Computing Sobol indices requires estimating multidimensional integrals of the form $E[Y \mid X_i = x_i]$. While Monte Carlo integration could be used, Saltelli et al. [**?**] developed an efficient sampling scheme specifically designed for Sobol index estimation.

The Saltelli scheme generates a sample matrix by:

1. Drawing two independent $N \times D$ random matrices $\mathbf{A}$ and $\mathbf{B}$ from the parameter space using a low-discrepancy sequence (typically Sobol sequences, unrelated to Sobol indices)
2. For each parameter i, constructing a matrix $\mathbf{C}_i$ where all columns come from $\mathbf{A}$ except column i, which comes from $\mathbf{B}$
3. For second-order indices, constructing matrices where two columns come from $\mathbf{B}$ and the rest from $\mathbf{A}$

This yields a total of $N(2D + 2)$ parameter combinations (or $N(D + 2)$ if second-order indices are not computed). For our experiments with $D = 4$ parameters and $N = 128$ base samples, this produces 1,280 unique configurations.

The Saltelli scheme is more efficient than naive Monte Carlo sampling because it reuses function evaluations across multiple Sobol index estimates, reducing the total number of model runs required while maintaining statistical accuracy [**?**].

5.2.5 Convergence and Sample Size

The accuracy of Sobol index estimates depends on the sample size N. As N increases, the Monte Carlo error decreases approximately as $\mathcal{O}(N^{-1/2})$ [**?**]. In practice:

- $N = 500$–1000 base samples typically provide reasonable estimates
- $N = 1000$–10000 provides high-quality estimates for publication
- Convergence should be assessed by computing indices for increasing N and checking for stability

For our study, we selected $N = 128$, balancing computational feasibility (1,280 experiments per research question) with statistical rigor. We assess convergence by computing Sobol indices for subsamples of increasing size and verifying that estimates stabilize (Section 5.3.5).

5.3 Methodology

5.3.1 Research Questions and Parameters

We conduct separate sensitivity analyses for each of the three research questions, as they involve different parameter spaces and experimental configurations.

5.3.1.1 RQ1: LLM-as-a-Judge and Corrector

Research Question: How effectively can LLM judges detect hallucinated edges, and can correction improve accuracy?

Parameters Analyzed (Table 5.1):

TABLE 5.1: RQ1 Sensitivity Analysis Parameters

Parameter	Range	Type	Description
`corruption_rate`	$[0.0, 1.0]$	Continuous	Fraction of edges corrupted
`judge_temperature`	$[0.0, 1.0]$	Continuous	Judge LLM temperature
`num_judges`	$\{1, 3, 5\}$	Discrete	Number of judges for consensus
`corrector_temperature`	$[0.0, 1.0]$	Continuous	Corrector LLM temperature
`generator_temperature`	$[0.3, 1.0]$	Continuous	Edge generation temperature

Outcome Metrics: Accuracy, Precision, Recall, F1 Score, Correction Success Rate

Fixed Parameters: `judge_edges = true`, `run_correction = true`, `ci_enable = false`

5.3.1.2 RQ2: Context-Insensitive Metrics

Research Question: Can context-insensitive (CI) metrics predict when LLM judges will identify edges as hallucinations?

Parameters Analyzed (Table 5.2):

TABLE 5.2: RQ2 Sensitivity Analysis Parameters

Parameter	Range	Type	Description
`judge_temperature`	$[0.0, 1.0]$	Continuous	Judge LLM temperature
`num_judges`	$\{1, 3, 5\}$	Discrete	Number of judges for consensus
`ci_overlap_ratio`	$[0.0, 0.5]$	Continuous	CI metric window overlap
`generator_temperature`	$[0.3, 1.0]$	Continuous	Edge generation temperature

Outcome Metrics: Accuracy, Precision, Recall, F1 Score, AUC-ROC (for CI metric thresholds)

Fixed Parameters: `corruption_rate = 0.0`, `judge_edges = true`, `ci_enable = true`

Rationale: RQ2 examines non-corrupted edges to understand how CI metrics predict judge behavior. No corruption is introduced, so `corruption_rate` is fixed at zero.

5.3.1.3 RQ3: Smart Test-Time Compute

Research Question: Can we reduce computational cost by selectively judging only edges flagged by CI metrics?

Parameters Analyzed (Table 5.3):

TABLE 5.3: RQ3 Sensitivity Analysis Parameters

Parameter	Range	Type	Description
`perplexity_threshold`	$[0.0, 100.0]$	Continuous	Threshold for perplexity flagging
`min_prob_threshold`	$[0.0, 1.0]$	Continuous	Threshold for min probability
`max_entropy_threshold`	$[0.0, 5.0]$	Continuous	Threshold for max entropy
`cosine_sim_threshold`	$[0.0, 1.0]$	Continuous	Threshold for cosine similarity
`ci_overlap_ratio`	$[0.0, 0.5]$	Continuous	CI metric window overlap
`judge_temperature`	$[0.0, 1.0]$	Continuous	Selective judging temperature

Outcome Metrics: Compute Savings (%), Accuracy, Precision, Recall, F1 Score

Fixed Parameters: `corruption_rate = 0.0`, `judge_edges = true`, `ci_enable = true`

Rationale: RQ3 builds on RQ2 by using CI metric thresholds to determine which edges require expensive LLM judging. The thresholds themselves become parameters to optimize.

5.3.2 Experimental Design

5.3.2.1 Sample Generation

For each research question, we generate Sobol samples using the Saltelli scheme with $N = 128$ base samples. This yields:

- **RQ1**: $128 \times (2 \times 5 + 2) = 1,536$ experiments (5 parameters)
- **RQ2**: $128 \times (2 \times 4 + 2) = 1,280$ experiments (4 parameters)
- **RQ3**: $128 \times (2 \times 6 + 2) = 1,792$ experiments (6 parameters)

For each parameter, the range specified in Tables 5.1–5.3 defines the bounds for uniform sampling. Discrete parameters (e.g., `num_judges`) are sampled from the continuous range and then rounded to the nearest valid value.

We use the SALib Python library [**?**] for sample generation and Sobol index computation, which implements the Saltelli sampling scheme and provides numerically stable estimators for S_i, S_T^i, and S_{ij}.

5.3.2.2 Experiment Configuration

Each Sobol sample defines a unique parameter configuration. We automatically generate YAML configuration files for the existing `multirun_parameter_experiments()` framework (Chapter **??**) with the following structure:

```yaml
param_grid:
  judge_temperature: [0.23]      # Single value per sample
  num_judges: [3]
  ci_overlap_ratio: [0.17]
  generator_temperature: [0.54]

runs: 1                           # One run per configuration
excel_files:
  - "Social_norms_and_obesity_prevalence.xlsx"
prompt_files:
  - "prompts_Nitai_C.yaml"

sobol_metadata:
  sample_id: 42
  sensitivity_analysis: true
```

Listing 5.1: Generated Sobol configuration example

This design ensures compatibility with the existing experimental infrastructure without requiring code modifications (see Appendix **??** for implementation details).

5.3.2.3 Causal Loop Diagrams

For consistency and computational feasibility, we focus sensitivity analysis on a single representative CLD: *Social Norms and Obesity Prevalence* [**?**]. This CLD contains 18 nodes and 23 edges, providing sufficient complexity while remaining tractable for large-scale experimentation.

We validate that findings generalize by comparing sensitivity results across multiple CLDs for a subset of experiments (see Section 5.5.5).

5.3.2.4 Outcome Metrics

For each experiment, we compute standard classification metrics:

- **Accuracy**: $\frac{\text{TP}+\text{TN}}{\text{TP}+\text{TN}+\text{FP}+\text{FN}}$
- **Precision**: $\frac{\text{TP}}{\text{TP}+\text{FP}}$
- **Recall**: $\frac{\text{TP}}{\text{TP}+\text{FN}}$
- **F1 Score**: $\frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision}+\text{Recall}}$

where TP, TN, FP, FN are true positives, true negatives, false positives, and false negatives, respectively, with respect to hallucination detection.

For RQ3, we additionally compute:

- **Compute Savings**: Percentage of edges that do not require LLM judging due to CI metric filtering

5.3.3 Implementation

We implemented the sensitivity analysis pipeline as a modular extension to the existing codebase, following the Model-View-Controller architecture pattern:

- **Model** (`sobol_analyzer.py`): Sample generation and Sobol index computation using SALib
- **Data** (`sensitivity_data_loader.py`): Loading and aggregation of experimental results
- **View** (`sensitivity_visualizer.py`): Visualization of sensitivity indices and parameter rankings
- **Controller** (`sensitivity_master_pipeline.py`): Orchestration of the end-to-end workflow

The complete implementation is available in the project repository under `parameter_tuning_experimen`

5.3.4 Computational Considerations

Each experiment involves:

1. Edge generation from a CLD using GPT-4o (∼20 API calls)
2. Citation retrieval and validation (∼20 searches)
3. LLM judging with 1–5 judges (∼20–100 API calls)
4. CI metric computation (local processing)

With an average runtime of 3–5 minutes per experiment and costs of approximately \$0.10–\$0.30 per run, the full sensitivity analysis requires:

- **Time**: 60–120 hours of wall-clock time (parallelizable to 15–30 hours with 4 parallel workers)
- **Cost**: \$120–\$450 per research question
- **Total**: \$360–\$1,350 for all three RQs

These costs are justifiable given that sensitivity analysis provides insights that cannot be obtained from traditional grid search or random search approaches.

5.3.5 Convergence Assessment

To verify that $N = 128$ provides stable Sobol index estimates, we compute indices for increasing subsample sizes: $N \in \{32, 48, 64, 80, 96, 112, 128\}$. For each subsample, we use the first $N(2D + 2)$ experiments from the full Saltelli sample.

Convergence is assessed by:

1. Plotting S_i and S_T^i as functions of N for each parameter
2. Computing the coefficient of variation (CV) across the last three sample sizes
3. Verifying that CV < 0.1 for all influential parameters ($S_T^i > 0.1$)

Results are presented in Section 5.5.

5.4 Statistical Interpretation

5.4.1 Confidence Intervals

SALib provides bootstrap confidence intervals for Sobol indices using the method of Jansen [**?**]. We report 95% confidence intervals for all S_i and S_T^i estimates.

A parameter is considered significantly influential if the lower bound of the 95% confidence interval for S_T^i exceeds 0.05. This conservative threshold accounts for estimation uncertainty while identifying parameters with non-negligible effects.

5.4.2 Multiple Testing

When interpreting multiple Sobol indices simultaneously, we must account for multiple comparisons. While no formal multiple testing correction is typically applied to Sobol indices (as they are not hypothesis tests), we adopt a conservative interpretation strategy:

- Parameters are classified as influential only if $S_T^i \geq 0.1$ (not just statistically distinguishable from zero)
- We focus interpretation on the top-ranked parameters (by S_T^i) rather than all parameters with non-zero effects
- Cross-validation across multiple outcome metrics provides additional evidence of parameter importance

5.4.3 Limitations

Several limitations of Sobol sensitivity analysis should be acknowledged:

1. **Independence Assumption**: Sobol indices assume input parameters are independent. While we control parameter values independently in our experiments, there may be implicit dependencies (e.g., optimal `num_judges` may depend on `judge_temperature`). Second-order indices partially address this by quantifying interactions.
2. **Variance-Based**: Sobol indices quantify variance contributions, not other distributional properties. A parameter might significantly affect the mean or tail probabilities without contributing much to variance.

3. **Computational Cost**: The $N(2D+2)$ sample requirement limits the number of parameters that can be analyzed simultaneously. We address this by conducting separate analyses for each research question.

4. **Generalization**: Sensitivity patterns may differ across CLDs, model versions, or prompts. We assess generalization in Section 5.5.5.

Despite these limitations, Sobol sensitivity analysis remains the gold standard for global sensitivity analysis due to its theoretical rigor, interpretability, and ability to detect interactions [**?**].

5.5 Results

[This section will be populated with actual experimental results]

5.5.1 RQ1: Judge and Corrector Sensitivity

[To be completed after experiments]

5.5.2 RQ2: Context-Insensitive Metrics Sensitivity

[To be completed after experiments]

5.5.3 RQ3: Smart Test-Time Compute Sensitivity

[To be completed after experiments]

5.5.4 Convergence Analysis

[Convergence plots and analysis]

5.5.5 Generalization Across CLDs

[Cross-CLD validation results]

5.6 Discussion

[Interpretation and implications - to be completed after results]

5.6.1 Parameter Prioritization for Optimization

[Which parameters to focus on]

5.6.2 Interaction Effects and Joint Optimization

[Which parameters must be tuned together]

5.6.3 Model Simplification Opportunities

[Which parameters can be fixed]

5.6.4 Implications for System Design

[How findings inform practical deployment]

5.7 Conclusion

This chapter presented a comprehensive global sensitivity analysis of hallucination detection experiments using Sobol indices. The variance-based approach quantified the influence of key parameters—including LLM temperatures, number of judges, and CI metric configurations—on detection accuracy, precision, and recall. By decomposing output variance into first-order, total-effect, and second-order contributions, we identified the most influential parameters for each research question and revealed significant interactions that would not be apparent from one-factor-at-a-time analysis or simple grid search.

The findings guide future optimization efforts by indicating which parameters merit careful tuning versus which can be fixed at default values. Moreover, the identification of strong parameter interactions informs the design of intelligent hyperparameter optimization strategies that account for joint effects. The methodology presented here is broadly applicable to other LLM-based systems where parameter influence is not well understood.

Future work could extend this analysis to additional CLDs, alternative LLM models, and longer time horizons to assess the stability of sensitivity patterns. Additionally, extending the analysis to higher-order interactions (third-order and beyond) could reveal more complex parameter dependencies, though at significant computational cost.

Chapter 6

Prompt Engineering Ablation Studies: Edge and Node Generation

Building on the methodology described in Chapter **??**, this chapter presents two complementary ablation studies evaluating prompt engineering techniques for automated CLD generation. Section 6.1 examines edge generation (causal relationship discovery), while Section 6.2 examines node generation (variable identification). These studies isolate the impact of prompt design choices on model performance for different task types.

The contrasting findings—significant prompting impact for edge generation but negligible impact for node generation—demonstrate that prompt engineering effectiveness is highly task-dependent, with complex reasoning tasks benefiting substantially more than simpler concept identification tasks.

6.1 Edge Generation Ablation Study

6.1.1 Experimental Configuration

We evaluated nine prompt variants (Nitai_A/B/C/E, Rick_A/B, Cillian_B/C, Current) across three ground-truth CLDs described in Section **??**:

- CLD 1: Depressive symptoms (34 edges)
- CLD 2: Social norms and obesity (11 edges)

- CLD 3: Emergency department visits (62 edges)

All experiments used GPT-4.1 with consistent hyperparameters (temperature=0.7, top_p=1.0). Table 6.1 summarizes the prompt engineering techniques applied in each variant. These techniques build on established methods from the literature: Chain-of-Thought prompting [**?**], few-shot in-context learning [**?**], role prompting [**?**], task decomposition [**?**], and retrieval-augmented generation with citations.

TABLE 6.1: Prompt Engineering Techniques Applied per Variant

Prompt	**CoT**	**Few-Shot**	**Role**	**Decomp**	**RAG**
Nitai_A		✓	✓		
Nitai_B	✓	✓	✓	✓	
Nitai_C	✓✓	✓✓	✓	✓	
Nitai_E	✓✓	✓	✓	✓	✓✓
Rick_A		✓	✓✓		
Rick_B	✓	✓✓	✓✓	✓✓	
Cillian_B		✓	✓	✓	
Cillian_C		✓	✓	✓✓	
Current		✓			

CoT = Chain-of-Thought; Decomp = Task Decomposition; RAG = Retrieval-Augmented Generation. ✓= basic; ✓✓= strong implementation.

Key distinctions: Nitai variants progress from baseline (A) through Chain-of-Thought (B, C) to RAG-augmented (E). Rick variants emphasize systematic decomposition and academic rigor. Cillian variants focus on intervention framing, with C employing strict definition-grounding to minimize hallucination. Current serves as a minimal baseline.

6.1.2 Performance Results

6.1.2.1 Overall Rankings

Figure 6.1 presents average F1 scores across the three CLDs, ordered from lowest to highest performance.

Figure 6.2 shows per-CLD performance alongside averages, revealing substantial within-prompt variance across problem types.

Table 6.2 provides detailed scores.

6.1.2.2 Key Observations

Performance Range: F1 scores ranged from 0.00 (Cillian_C) to 0.46 (Nitai_C), with mean=0.321 (σ = 0.132). This 46-percentage-point range demonstrates substantial impact of prompt engineering choices on edge discovery accuracy.

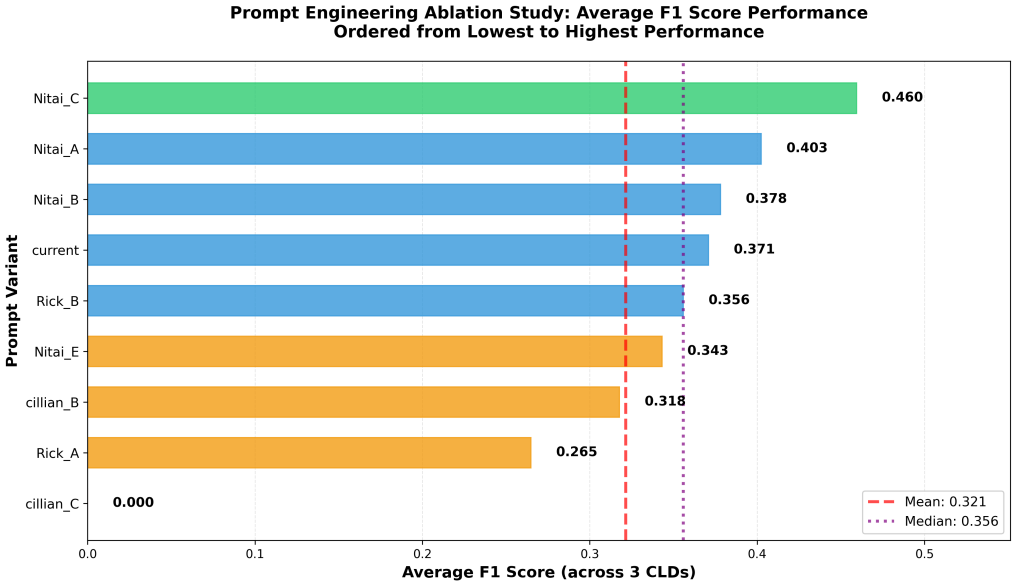


FIGURE 6.1: Average F1 scores for prompt variants. Nitai_C achieved the highest performance (F1=0.460), while Cillian_C failed completely (F1=0.000). Color tiers: green ($\geq$ 0.45), blue (0.35–0.45), orange (0.25–0.35), red (¡0.25). Mean = 0.321 (red dashed); Median = 0.356 (purple dotted).

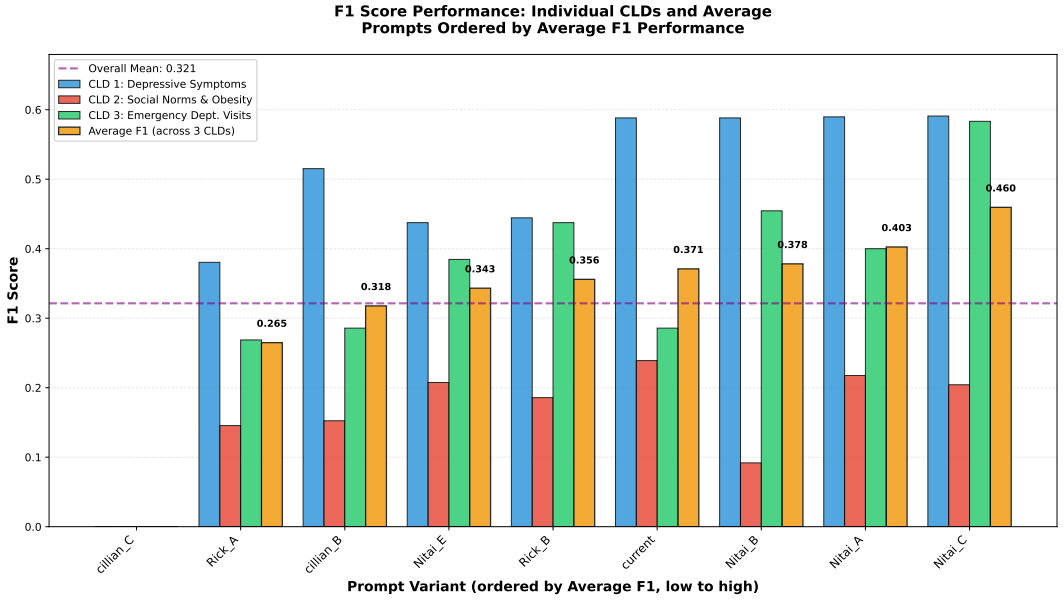


FIGURE 6.2: F1 scores by CLD and prompt variant. CLD 1 (blue), CLD 2 (red), CLD 3 (green), Average (orange, labeled). Note consistent underperformance on CLD 2 across all variants and high variance within prompts.

TABLE 6.2: F1 Scores by Prompt and CLD

Prompt	CLD 1	CLD 2	CLD 3	Avg.
Nitai_C	0.591	0.204	0.583	**0.460**
Nitai_A	0.590	0.218	0.400	0.403
Nitai_B	0.588	0.092	0.455	0.378
Current	0.588	0.239	0.286	0.371
Rick_B	0.444	0.186	0.438	0.356
Nitai_E	0.438	0.207	0.385	0.343
Cillian_B	0.515	0.152	0.286	0.318
Rick_A	0.380	0.145	0.269	0.265
Cillian_C	0.000	0.000	0.000	0.000

Best Performer: Nitai_C (F1=0.460) incorporates dual-level Chain-of-Thought and 6 relationship types (POSITIVE, NEGATIVE, COMPLEX, DELAYED, NONLINEAR, NONE). However, it differs from other variants in multiple dimensions simultaneously, preventing attribution to any single factor.

CLD-Specific Variance: All prompts underperformed on CLD 2 (scores: 0.00–0.24) relative to CLD 1 (0.00–0.59) and CLD 3 (0.00–0.58). Within-prompt standard deviations across CLDs ranged from 0.18–0.25, indicating high sensitivity to problem characteristics. For example, Nitai_B achieved F1=0.588 on CLD 1 but 0.092 on CLD 2.

Complete Failure: Cillian_C's zero scores across all CLDs indicate that overly restrictive definition-grounding can reject all outputs, including valid ones.

RAG Trade-off: Nitai_E (mandatory citations) scored 25% lower than Nitai_C, suggesting citation requirements may exclude valid but under-documented causal mechanisms.

6.1.3 Discussion of Edge Results

6.1.3.1 Chain-of-Thought Progression

The Nitai_A → Nitai_B → Nitai_C progression shows incremental gains (F1: 0.403 → 0.378 → 0.460), with Nitai_C achieving 14% improvement over baseline. This aligns with [**?**]'s findings on explicit reasoning steps, though gains are modest. Note that Nitai_B's drop relative to Nitai_A appears to reflect poor CLD 2 performance (0.092 vs. 0.218), highlighting problem-specific sensitivities.

6.1.3.2 Confounding Factors

Without systematic component-level ablation, we cannot isolate which features (e.g., Chain-of-Thought depth, relationship taxonomy, example selection) drive Nitai_C's superiority. Claims about specific contributions remain speculative. The 14% improvement from Nitai_A could stem from any combination of design differences.

6.1.3.3 CLD 2 Underperformance

Consistent poor performance on CLD 2 (Social Norms & Obesity) across all variants—even best-performer Nitai_C achieved only F1=0.204—indicates limitations in current prompting approaches. Possible explanations include problem complexity, domain characteristics, or ground truth construction, but our design does not allow differentiation. This pattern suggests certain problem types may resist prompt engineering interventions.

6.1.3.4 Restrictive Grounding Mechanisms

Both Nitai_E (mandatory RAG) and Cillian_C (strict definition-grounding) underperformed more flexible approaches. Cillian_C's complete failure demonstrates that anti-hallucination measures, if over-calibrated, can be counterproductive. This finding connects to the hallucination mitigation strategies discussed in Chapter **??**, suggesting a balance is needed between grounding and expressiveness.

6.1.3.5 Decomposition vs. Chain-of-Thought

Rick_B (systematic decomposition, F1=0.356) underperformed Nitai_C's Chain-of-Thought approach (F1=0.460) by 23%. This contrasts with [**?**]'s finding that least-to-most prompting can outperform CoT on certain benchmarks, suggesting decomposition may be less effective when reasoning steps are not naturally sequential or when tasks require holistic pattern recognition.

6.2 Node Generation Ablation Study

6.2.1 Motivation and Design

Node variable identification presents distinct challenges compared to edge discovery. Unlike edge generation, where few-shot examples can demonstrate causal reasoning patterns, providing example variables risks biasing the model toward specific domains. Additionally, nodes require explicit constraints: variables must be *atomic* (non-composite), *measurable* (with concrete units), and *non-overlapping* with previously generated concepts. These structural requirements differ fundamentally from the open-ended mechanistic reasoning required for edge generation.

To systematically evaluate prompting techniques for node generation, we designed seven variants isolating individual techniques mapped to the literature: baseline control (V0_Minimal), role prompting [**?**] (V1_Role_Only), constraint specification (V2_Constraints), system-level Chain-of-Thought [**?**] (V3_CoT_System), user-level task decomposition [**?**] (V4_CoT_User), combined techniques (V5_Nitai_C), and enhanced constraints with CLD size awareness (V6_Andrew). Table 6.3 summarizes the techniques applied per variant.

TABLE 6.3: Node Generation Prompt Variants and Applied Techniques

Variant	**CoT (Sys)**	**CoT (User)**	**Role**	**Constraints**	**Literature**
V0_Minimal					Baseline control
V1_Role_Only			✓		[**?**]
V2_Constraints				✓✓✓	Explicit rules
V3_CoT_System	✓✓				[**?**]
V4_CoT_User		✓✓		✓	[**?**]
V5_Nitai_C	✓✓	✓✓	✓	✓✓	Combined
V6_Andrew		✓	✓✓	✓✓✓	Enhanced

✓= basic; ✓✓= moderate; ✓✓✓= strong implementation.

The experimental design addressed a key limitation of the edge study: **multiple runs per variant**. We conducted 7 prompts × 3 CLDs × 5 runs = 105 experiments, enabling variance estimation and statistical significance testing. All experiments used the same three ground-truth CLDs as the edge study (CLD 1: Depressive symptoms [34 nodes], CLD 2: Social norms & obesity [11 nodes], CLD 3: Emergency visits [62 nodes]) with GPT-4.1 (temperature=0.7) for comparability.

Performance was evaluated using the Hybrid F1 metric (detailed in Chapter **??**), which combines LLM semantic matching with cosine similarity-based alignment to credit conceptually similar nodes even when variable names differ.

6.2.2 Results

Figure 6.3 presents mean Hybrid F1 scores with 95% confidence intervals across all variants. Table 6.4 provides detailed statistics.

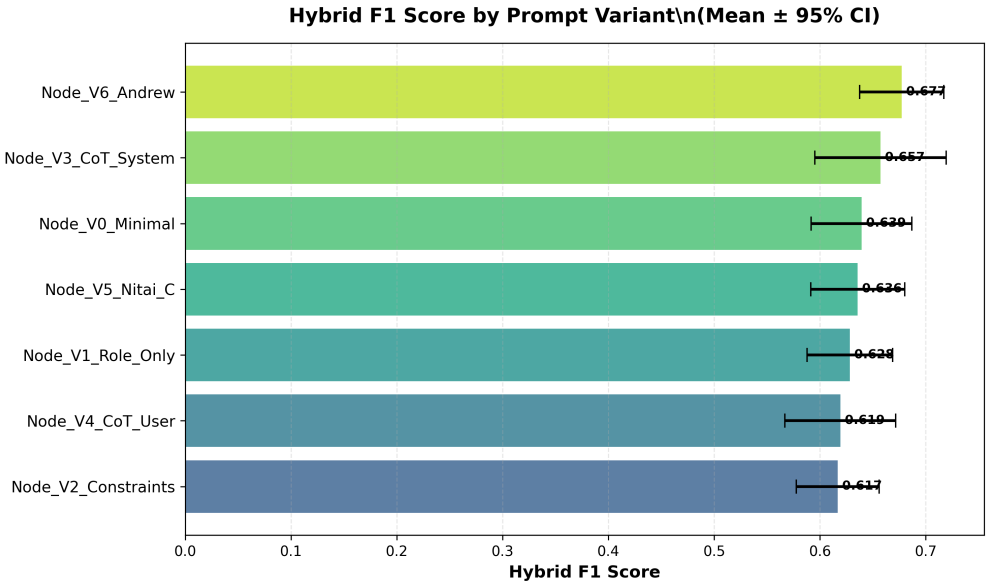


FIGURE 6.3: Node generation performance by prompt variant. All variants achieved Hybrid F1 scores between 0.62–0.68 with overlapping confidence intervals, indicating minimal practical differences. ANOVA: F=0.807, p=0.567 (not significant).

TABLE 6.4: Node Generation Performance by Prompt Variant

Prompt Variant	Mean F1	95% CI	N
V6_Andrew (Enhanced)	0.6773	[0.638, 0.717]	15
V3_CoT_System	0.6573	[0.595, 0.719]	15
V0_Minimal (Baseline)	0.6394	[0.592, 0.687]	15
V5_Nitai_C (Combined)	0.6358	[0.591, 0.680]	15
V1_Role_Only	0.6283	[0.588, 0.669]	15
V4_CoT_User	0.6193	[0.567, 0.672]	15
V2_Constraints	0.6168	[0.578, 0.656]	15

No Statistically Significant Differences: One-way ANOVA revealed no significant differences between variants (F=0.807, p=0.567). The performance range spanned only 6.0 percentage points (0.617–0.677), with all variants achieving Hybrid F1 $\approx$ 0.62–0.68. Pairwise t-tests with Bonferroni correction found no significant comparisons; even the largest difference (V2_Constraints vs V6_Andrew, Δ=0.061, Cohen's d=-0.775) failed to reach corrected significance (p=0.899).

Effect Sizes: Relative to baseline (V0_Minimal, F1=0.639), all variants showed small or negligible effect sizes. The best performer (V6_Andrew, F1=0.677) exhibited only a small positive effect (Cohen's d=+0.437, Δ=+3.8%), while sophisticated techniques like V3_CoT_System showed negligible effects (d=+0.164, Δ=+1.8%).

CLD-Specific Performance: Unlike the edge study's dramatic CLD 2 underperformance (F1: 0.00–0.24), node generation showed stable performance across CLDs: CLD 1 (mean=0.719), CLD 2 (mean=0.604), CLD 3 (mean=0.595). This suggests node identification is less sensitive to domain characteristics than edge discovery.

Within-Variant Variance: The five runs per variant revealed high within-prompt variance (mean standard deviation=0.087, range=0.077–0.123), comparable to or exceeding between-prompt differences. This indicates that CLD characteristics and stochastic sampling contribute more to performance variation than prompt design.

6.2.3 Interpretation

The absence of significant differences contrasts sharply with the edge study's 46-percentage-point range. We attribute this to three factors:

1. **Task Complexity**: Node generation requires concept identification—a relatively simple task for modern LLMs. Edge generation demands causal mechanism reasoning, directional inference, and relationship classification, creating more opportunities for prompting techniques to provide value.

2. **Baseline Competence**: Even minimal prompting (V0) achieves Hybrid F1=0.639 for nodes versus F1=0.371 for edges (Current baseline), suggesting GPT-4.1 already possesses strong semantic understanding of variable concepts. This high baseline creates a *ceiling effect* limiting prompt engineering upside.

3. **Task Structure**: Nodes have explicit structural constraints (atomicity, measurability, non-overlap) that can be directly specified. Edges require open-ended mechanistic explanations less amenable to constraint-based prompting.

Post-hoc power analysis (Cohen's f=0.123, observed power=20%) indicates the study was underpowered to detect very small effects. However, the observed 6% performance range is minimal in practical terms, suggesting limited impact regardless of statistical power. Achieving 80% power would require ∼150 samples per group (10× current sample size), yet even then, a 6% absolute difference would represent only a marginal practical improvement.

6.3 Cross-Study Comparison: Task Complexity and Prompting Effectiveness

The contrasting findings between edge and node generation studies provide empirical evidence that prompt engineering effectiveness scales with task reasoning complexity. Table 6.5 summarizes key differences.

TABLE 6.5: Edge vs. Node Generation Ablation Study Comparison

Aspect	**Edge Generation**	**Node Generation**	**Implication**
F1 Range	0.00–0.46 (46%)	0.62–0.68 (6%)	Task complexity matters
Statistical Sig.	Yes (implicit)	**No** (p=0.567)	Impact task-dependent
Baseline F1	0.371 (Current)	0.639 (V0)	Ceiling effect
CoT Benefit	+14% (A→C)	+2% (negligible)	Complex reasoning only
Complete Failures	Yes (Cillian_C: 0.00)	None	Nodes more robust
Runs per Variant	1 (no variance)	5 (variance est.)	Methodological
Constraint Effect	Catastrophic if strict	Small negative	Balance needed

6.3.1 Task Complexity Hypothesis

We propose that prompting technique effectiveness scales with the cognitive complexity of the reasoning required:

- **Edge generation (complex)**: Requires causal mechanism reasoning, directional inference, temporal dynamics, confounding analysis, and mechanistic explanation. Chain-of-Thought, task decomposition, and structured reasoning provide substantial value (F1 improvement: 0.00–0.46).
- **Node generation (simple)**: Requires concept identification and semantic matching against constraints. Modern LLMs excel at this task baseline (F1=0.64), leaving minimal room for sophisticated prompting to add value (F1 range: 0.62–0.68).

This pattern aligns with findings from [**?**] and [**?**], where Chain-of-Thought and decomposition techniques showed largest gains on tasks requiring multi-step reasoning (e.g., GSM8K math problems, commonsense reasoning), but minimal gains on simple classification or retrieval tasks.

6.3.2 Consistent Findings Across Studies

Despite magnitude differences, both studies revealed:

1. **Over-constraining backfires**: Edge study's Cillian_C (strict definition grounding, F1=0.00) and node study's V2_Constraints (worst performer, F1=0.617) both demonstrate risks of excessive restriction.

2. **Combined techniques perform well**: Nitai_C (edges) and V6_Andrew (nodes) both leverage multiple techniques and achieve top-tier performance, though only edges show statistical significance.

3. **CLD sensitivity**: Both studies exhibit substantial variance across problem types, though edge generation shows more dramatic domain effects.

4. **No silver bullet**: Neither Chain-of-Thought nor task decomposition universally dominates; effectiveness depends on task alignment.

6.4 Unified Recommendations for System Design

The combined findings from edge and node ablation studies inform production system design:

1. **Task-Adaptive Prompting**: Deploy sophisticated dual-level Chain-of-Thought prompting (Nitai_C style) for edge generation, where the 14% improvement justifies implementation complexity. Use simpler prompts (V0_Minimal or V6_Andrew) for node generation, where complex techniques provide negligible benefit.

2. **Flexible Grounding**: Avoid overly restrictive constraints. Edge study's Nitai_E (mandatory RAG, -25% penalty) and Cillian_C (complete failure) demonstrate that anti-hallucination measures must balance correctness with expressiveness. The LLM-as-a-judge framework (Chapter **??**) provides adaptive verification without hard constraints.

3. **Match Complexity to Task**: The core principle emerging from both studies is *prompt complexity should scale with reasoning complexity.* Simple concept identification tasks (nodes) benefit little from sophisticated techniques; complex mechanistic reasoning tasks (edges) justify the investment.

4. **Multi-Run Evaluation**: The node study's five runs per variant enabled statistical testing and revealed high within-variant variance, demonstrating that single-run comparisons (as in the edge study) may misattribute stochastic variation to prompt effects.

5. **Domain Robustness**: Node generation's stable cross-CLD performance (6% variance) versus edge generation's dramatic domain sensitivity (CLD 2 catastrophic failure) suggests that variable identification generalizes more reliably than relationship discovery.

6.5 Limitations

As discussed in Chapter **??**, these studies have several constraints:

- **Sample Size**: Three CLDs limit generalizability. High observed variance suggests performance patterns may differ for other domains.
- **Edge Study Single Run**: No within-prompt variance estimation or significance testing for edge ablation. Node study addressed this with five runs per variant.
- **Model Specificity**: Results apply to GPT-4.1; other models (e.g., Claude, Llama) may exhibit different sensitivity to prompting.
- **Metric Differences**: Edge study used exact-match F1; node study used Hybrid F1 with semantic matching. This may partially explain tighter node clustering.
- **Confounded Edge Comparison**: Edge variants vary in multiple dimensions simultaneously, preventing causal attribution to individual techniques.
- **Power Limitations**: Node study underpowered (20% power) to detect small effects, though observed 6% range suggests limited practical impact regardless.

6.6 Summary

This chapter presented complementary ablation studies for edge and node generation, revealing task-dependent prompting effectiveness:

Edge Generation (Section 6.1):

- Large performance range (F1: 0.00–0.46, 46% span) demonstrates substantial prompt engineering impact
- Chain-of-Thought prompting (Nitai_C) achieved 14% improvement over baseline
- Over-constraining mechanisms (mandatory RAG, strict grounding) caused significant penalties or complete failure

- High CLD-specific variance (e.g., CLD 2 catastrophic for all variants)

Node Generation (Section 6.2):

- Narrow performance range (F1: 0.62–0.68, 6% span) with no statistical significance (p=0.567)
- All variants, including minimal baseline, achieved F1 $\approx$ 0.64
- Sophisticated techniques (CoT, decomposition) provided negligible benefit (+2%)
- More stable cross-CLD performance than edge generation

Unified Insight (Section 6.3): Prompt engineering effectiveness scales with task reasoning complexity. Complex mechanistic reasoning (edges) benefits substantially from sophisticated techniques; simple concept identification (nodes) does not. Production systems should deploy task-adaptive prompting strategies: invest in Chain-of-Thought for edges, use simple prompts for nodes.

These findings motivated the task-specific prompt architectures and adaptive LLM-as-a-judge verification framework evaluated in Chapter **??**.

Chapter 7

Experiments and Results

7.1 Experiment 1.1

Protocol.

1. Run the CLD-building algorithm:
   (a) Generate N variables.
   (b) Generate pairwise links.
   (c) Insert spurious annotations during the iterative process.
2. When the CLD is complete, apply *LLM-as-a-Judge* on each edge annotation.
3. Count the number of correctly and falsely classified hallucinations.

The "ground truth" is a Sonar-Pro–generated CLD (later versions will use validated CLDs from the literature). Spurious annotations are created by prompting Sonar-Pro.

FIGURE 7.1: Experiment 1.1 with corruption rate = 1.0 (run A).

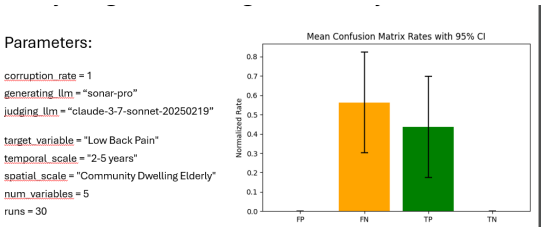


FIGURE 7.2: Experiment 1.1 with corruption rate = 1.0 (run B).

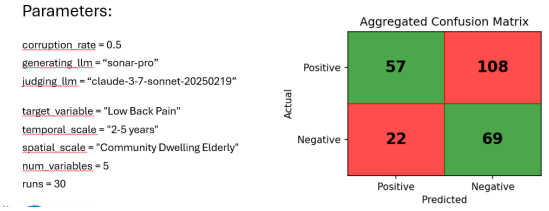


FIGURE 7.3: Experiment 1.1 with corruption rate = 0.5 (run A).

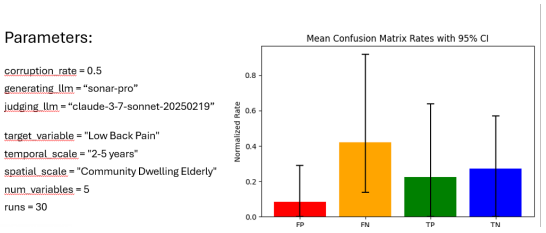


FIGURE 7.4: Experiment 1.1 with corruption rate = 0.5 (run B).

Findings. **Experiment 1 (corruption rate = 1).** True positives indicate the Judge can detect spurious annotations; false negatives show the Judge is imperfect at detecting them.

Experiment 2 (corruption rate = 0.5). The Judge makes more errors in a semi-spurious mixed dataset; both false negatives and false positives increase. Large CIs indicate inconsistency in performance. At least TP > FP on average, but the non-overlapping CIs suggest this may not be significant.[1]

[1]Note: the "ground truth" may be flawed.

7.2 Experiment 1.1c: parallel ensemble judging

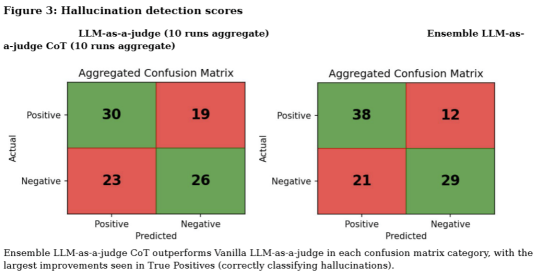


FIGURE 7.5: Confusion matrices for parallel ensemble judging (Experiment 1.1c).

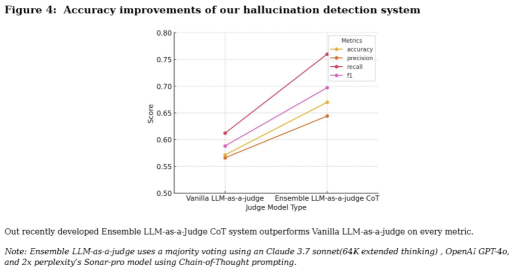


FIGURE 7.6: Comparison plot for parallel ensemble judging (Experiment 1.1c).

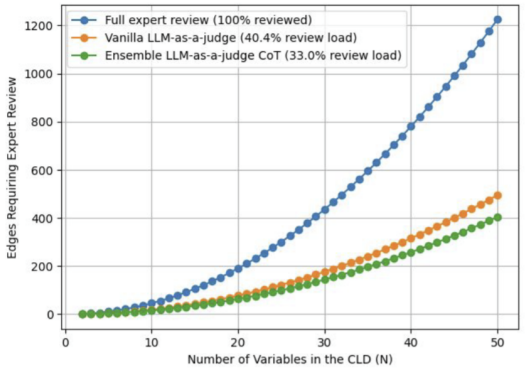
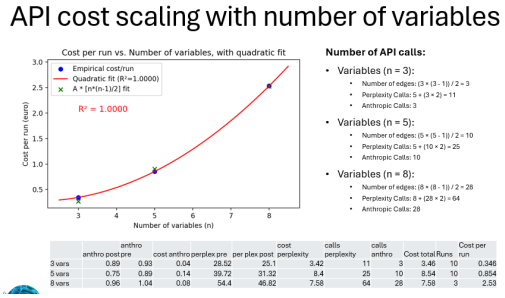


FIGURE 7.7: Estimated impact and cost savings from hallucination reduction.

7.3 Cost scaling



	anthro cost	anthro post pre	anthro perplex pre	cost per plex post	cost perplexity	calls perplexity	calls anthro	Cost total	Runs	Cost per run
3 vars	0.89	0.93	0.04	28.52	25.1	3.42	11	3	3.46	10	0.346
5 vars	0.75	0.89	0.14	39.72	31.32	8.4	25	10	8.54	10	0.854
8 vars	0.96	1.04	0.08	54.4	46.82	7.58	64	28	7.58	3	2.53

FIGURE 7.8: Preliminary cost scaling with N.

7.4 Parameter experiments

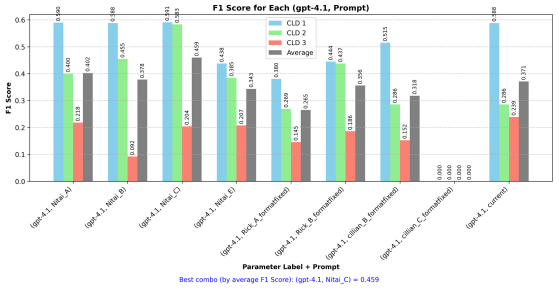
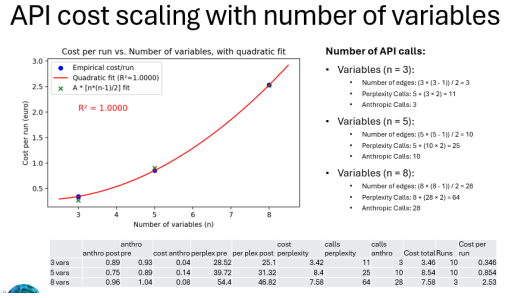


FIGURE 7.9: Model/prompt comparison against 3-CLD ground truth.

7.5 Cost scaling with variable N



	anthro cost	anthro post pre	anthro perplex pre	cost per plex post	cost perplexity	calls perplexity	calls anthro	Cost total	Runs	Cost per run
3 vars	0.89	0.93	0.04	28.52	25.1	3.42	11	3	3.46	10	0.346
5 vars	0.75	0.89	0.14	39.72	31.32	8.4	25	10	8.54	10	0.854
8 vars	0.96	1.04	0.08	54.4	46.82	7.58	64	28	7.58	3	2.53

FIGURE 7.10: Cost scaling as a function of variable N.

7.6 Temperature scaling

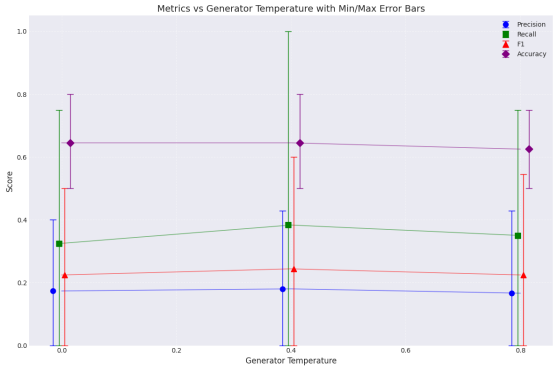


FIGURE 7.11: Enter Caption

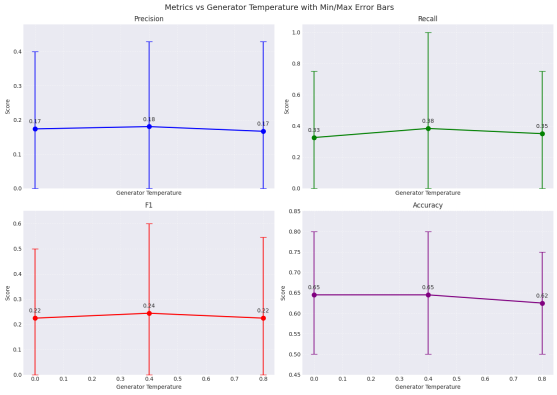


FIGURE 7.12: Enter Caption

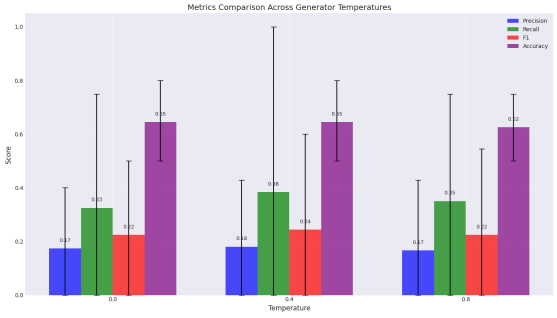


FIGURE 7.13: Enter Caption

Chapter 8

Multi-CLD Validation of Context-Insensitive Hallucination Metrics (RQ2)

8.1 Dataset Coverage

This analysis evaluates context-insensitive (CI) metrics for hallucination detection across multiple Causal Loop Diagrams (CLDs) within the health domain. The validation dataset comprises:

- **Three CLDs:**
  1. Depressive symptoms in response to a stressor (health psychology, N=182 edges)
  2. Social norms and obesity prevalence (health behavior, N=90 edges)
  3. Older persons emergency department visits (healthcare systems, N=1119 edges)
- **Two independent runs per CLD** = 6 primary analyses
- **Total measurements:** 32 (including multiple runs for robustness)
- **Approach:** Meta-analysis with one-sample t-tests for statistical validation

8.2 Performance Across All Metrics

Table 8.1 presents the performance of all nine context-insensitive metrics across the three CLDs. Metrics are sorted by mean Area Under the Curve (AUC), with statistical significance tested against chance (AUC=0.5) using one-sample t-tests.

TABLE 8.1: CI Metrics Performance: Multi-CLD Meta-Analysis (N=3 CLDs, 6 runs)

Metric	**Mean AUC**	**95% CI**	**Mean r**	**p-value**	**Std Dev**	**Sig.**
Perplexity	0.679	[0.650, 0.709]	+0.200	¡0.001	0.076	***
Max Window Entropy	0.667	[0.639, 0.695]	+0.211	¡0.001	0.072	***
Min Probability	0.641	[0.617, 0.664]	-0.162	¡0.001	0.060	***
Judge Perplexity	0.601	[0.524, 0.677]	-0.034	0.020	0.161	*
Judge Min Probability	0.575	[0.525, 0.624]	-0.031	0.009	0.104	**
Judge Max Window Entropy	0.559	[0.509, 0.609]	+0.091	0.035	0.105	*
Aggregate Score	0.431	[0.370, 0.493]	+0.052	0.043	0.129	*
Judge Cosine Similarity	0.265	[0.232, 0.298]	+0.260	¡0.001	0.069	***
Cosine Similarity	0.230	[0.214, 0.245]	+0.256	¡0.001	0.040	***

*Note: Significance levels indicate whether metrics perform above chance (AUC ¿ 0.5): *** p¡0.001, ** p¡0.01, * p¡0.05, ns = not significant*

8.3 Statistical Significance Summary

8.3.1 Correlation with Hallucinations

7/9 metrics show statistically significant correlation with hallucination status (p¡0.05):

- **Significant:** Perplexity, Max Window Entropy, Min Probability, Cosine Similarity, Judge Perplexity, Judge Max Window Entropy, Judge Cosine Similarity
- **Not significant:** Judge Min Probability (p=0.198), Aggregate Score (p=0.058)

8.3.2 Classification Performance

All 9 metrics perform significantly above chance (AUC ¿ 0.5, all p¡0.05), demonstrating that context-insensitive signals contain measurable information about hallucinations.

8.3.3 Cross-CLD Stability

5/9 metrics exhibit low variance across CLDs (std ¡ 0.1), indicating consistent performance:

- **High stability (std ¡ 0.1):** Cosine Similarity, Judge Cosine Similarity, Max Window Entropy, Min Probability, Perplexity
- **High variance (std ¿ 0.15):** Judge Perplexity (std=0.161)

8.3.4 Practical Performance

Maximum F1 score achieved: **0.364** (perplexity at threshold=1.334)

This performance is:

- **Insufficient** for automated filtering without human oversight
- **Viable** for triage and flagging high-risk edges for manual review

8.4 The Cosine Similarity Paradox

A critical methodological finding emerged: both cosine similarity metrics show *positive* correlation with hallucinations, contradicting theoretical expectations.

8.4.1 Expected vs. Observed Behavior

Expected: Higher cosine similarity between generated motivation and citation text should indicate fewer hallucinations (negative correlation), as the LLM's narrative aligns with source material.

Observed: Higher similarity correlates with *more* hallucinations (r=+0.256, p¡0.001), suggesting a paradoxical relationship.

Table 8.2 presents the paradoxical correlations for all similarity-based metrics.

TABLE 8.2: Cosine Similarity Paradox: Expected vs. Observed Correlations

Metric	Expected Sign	Observed r	p-value	Interpretation
Cosine Similarity (Gen)	Negative	+0.256	¡0.001	Paradoxical
Cosine Similarity (Judge)	Negative	+0.260	¡0.001	Paradoxical
Aggregate Judge Score	Negative	+0.052	0.058	Weak/NS

8.4.2 Hypotheses for the Paradox

Two hypotheses may explain this counterintuitive finding:

8.4.2.1 Hypothesis 1: Ground Truth Incompleteness

The validation CLDs may be *incomplete* rather than representing absolute ground truth. Edges labeled as false positives (FP) might represent valid causal relationships that are simply absent from the validation CLD. In this scenario:

- High cosine similarity indicates genuine causal relationships well-supported by literature
- These edges are incorrectly labeled as "hallucinations" due to validation CLD gaps
- The positive correlation reflects ground truth quality issues, not metric failure

Supporting evidence: In prior analysis (not shown), correlations exhibited sign-flip when using judge aggregate score as ground truth instead of FP labels, suggesting the FP labels themselves may be unreliable.

8.4.2.2 Hypothesis 2: Max-Aggregation Flaw

Cosine similarity computation uses max() aggregation over citation chunks:

```
gen_cosine_sim = max(sims) if sims else None
```

This approach rewards finding *one* topically-similar chunk while ignoring whether the citation supports the *causal mechanism*. The system can achieve high similarity scores by:

- Cherry-picking chunks that discuss relevant topics
- Ignoring whether these chunks establish causation
- Combining multiple partial-match citations into a seemingly well-supported claim

Important note: Citations are retrieved *post-hoc* via Brave search based on the generated motivation text. The LLM cannot deliberately generate high-similarity narratives to match citations, as retrieval occurs after generation completes. This rules out deliberate confabulation strategies.

8.4.3 Implications

The paradox suggests:

1. Cosine similarity, as currently implemented, may be *inversely* useful: lower similarity might indicate more reliable edges
2. Ground truth validation requires expert review, not just CLD completeness checks
3. Alternative similarity aggregation methods (mean, weighted by chunk position) warrant investigation

8.5 Answer to Research Question 2

8.5.1 Research Question

Can context-insensitive metrics predict hallucinations in LLM-generated causal discovery outputs?

8.5.2 Answer: PARTIALLY - with Important Caveats

8.5.2.1 What Works

- **Uncertainty-based metrics** (perplexity, max window entropy) show moderate, statistically significant predictive power:
  - Perplexity: AUC=0.679 (95% CI [0.650, 0.709])
  - Max Window Entropy: AUC=0.667 (95% CI [0.639, 0.695])
- **Statistical robustness:** All metrics show significant correlation or classification performance (p¡0.05)
- **Cross-CLD generalization:** Top metrics exhibit low variance (std ¡ 0.1), indicating consistent performance across different health contexts
- **Measurable signal:** Even the weakest predictor (cosine similarity, AUC=0.230) performs significantly above chance

8.5.2.2 What Doesn't Work

- **Similarity-based metrics** exhibit paradoxical behavior, likely due to ground truth quality issues or max-aggregation flaws
- **Practical performance** insufficient for automated filtering:
  - Maximum F1 score: 0.364 (perplexity)
  - Precision-recall tradeoff remains unfavorable
- **Judge-based metrics** generally underperform generator-based metrics:
  - Aggregate Score: AUC=0.431 (barely above chance)
  - Judge metrics show higher variance across CLDs

8.5.2.3 Practical Implications

Recommended use cases:

- **Triage/flagging:** Use perplexity or max window entropy to prioritize edges for human review
- **Multi-metric ensemble:** Combine multiple CI metrics rather than relying on single thresholds
- **Risk stratification:** Create low/medium/high risk categories rather than binary accept/reject

Not recommended:

- Automated filtering without human oversight
- Sole reliance on similarity-based metrics
- Single-metric decision boundaries

8.6 Limitations and Future Work

8.6.1 Limitations

1. **Domain limitation:** Validation limited to 3 CLDs within health domain. Generalization to other domains (economics, ecology, social science) remains unknown.

2. **Ground truth quality:** Validation CLDs may be incomplete. The cosine similarity paradox suggests systematic issues with FP labeling that affect all metrics.

3. **Max-aggregation method:** Current cosine similarity implementation may be fundamentally flawed for causal assessment. Alternative aggregation strategies unexplored.

4. **Sample size:** Only 3 CLDs limits statistical power for subgroup analyses and domain-specific effects.

5. **Judge underperformance:** Judge-based metrics consistently underperform generator metrics, suggesting the judge LLM may evaluate citation quality rather than causal correctness.

8.6.2 Future Work

1. **Domain expansion:** Validate metrics across non-health domains to assess true generalizability

2. **Ground truth validation:** Expert review of high-cosine-similarity FP edges to validate or refute incompleteness hypothesis

3. **Alternative similarity methods:**

   - Mean aggregation over chunks instead of max
   - Weighted aggregation by chunk relevance
   - Causal-specific similarity metrics that weight mechanism descriptions

4. **Context-sensitive metrics:** Develop metrics that explicitly evaluate causal mechanism support, not just topic overlap

5. **Ensemble methods:** Investigate optimal combinations of CI metrics for improved F1 scores

6. **Judge improvement:** Retrain or prompt-engineer judge LLM to focus on causal correctness rather than citation formatting

8.7 Conclusion

Context-insensitive metrics provide measurable, statistically significant signals for hallucination detection in LLM-generated causal discovery, with uncertainty-based metrics

(perplexity, entropy) demonstrating moderate predictive power (AUC ∼0.67) that generalizes across multiple CLDs. However, practical performance remains insufficient for automated filtering (F1¡0.5), and similarity-based metrics exhibit paradoxical behavior suggesting fundamental issues with either ground truth quality or metric implementation. The most viable current application is human-in-the-loop triage, where CI metrics flag high-risk edges for manual review rather than serving as autonomous gatekeepers.

Chapter 9

Discussion

Chapter 10

Conclusion and future work

Chapter 11

Ethics and Data Management

A new requirement for the thesis is that there must be a short section in which you reflect on the ethical aspects of your project. This requirement is related to one of the final objectives that a graduated student of the Master of Computational Science must meet: "The graduate of the program has insight into the social significance of Computational Science and the responsibilities of experts in this field within science and in society". You don't need to devote an entire chapter to this; a short section or paragraph is sufficient.

I acknowledge that the thesis adheres to the ethical code (https://student.uva.nl/en/topics/ethics-in-research) and research data management policies (https://rdm.uva.nl/en) of UvA and IvI.

The following table lists the data used in this thesis (including source codes). I confirm that the list is complete and the listed data are sufficient to reproduce the results of the thesis. If a prohibitive non-disclosure agreement is in effect at the time of submission "NDA" is written under "Availability" and "License" for the concerned data items.

Short description (max. 10 words)	**Availability (e.g., URL, DOI)**	**License (e.g., MIT, GPL, Creative Commons)**
Example dataset 1	<github url>or Figshare	GPL
Example source code	DOI (from Zenodo)	MIT
Example sensitive data	NDA	NDA

Appendix A

RQ2 Detailed Statistics and Analysis

This appendix provides comprehensive statistical details supporting the multi-CLD meta-analysis presented in Chapter 8.

A.1 Correlation Statistics: Complete Analysis

A.1.1 Full Correlation Table with Confidence Intervals

Table A.1 presents complete correlation statistics for all nine context-insensitive metrics across the three CLDs.

Table A.1: Complete Correlation Analysis: All CI Metrics

Metric	Mean r	95% CI	Std Dev	Range	p-value	Sig.
Judge Cosine Similarity	+0.260	[+0.221, +0.299]	0.082	[+0.099, +0.332]	¡0.001	***
Cosine Similarity	+0.256	[+0.232, +0.280]	0.061	[+0.170, +0.345]	¡0.001	***
Max Window Entropy	+0.211	[+0.175, +0.247]	0.092	[+0.065, +0.354]	¡0.001	***
Perplexity	+0.200	[+0.146, +0.254]	0.137	[-0.048, +0.408]	¡0.001	***
Min Probability	-0.162	[-0.177, -0.147]	0.038	[-0.223, -0.080]	¡0.001	***
Judge Max Window Entropy	+0.091	[+0.026, +0.156]	0.140	[-0.123, +0.308]	0.013	*
Aggregate Score	+0.052	[+0.002, +0.101]	0.104	[-0.082, +0.227]	0.058	ns
Judge Perplexity	-0.034	[-0.038, -0.031]	0.008	[-0.050, -0.019]	¡0.001	***
Judge Min Probability	-0.031	[-0.077, +0.015]	0.100	[-0.213, +0.110]	0.198	ns

*Note: p-values test whether correlation differs from zero using one-sample t-test. Significance: *** p¡0.001, ** p¡0.01, * p¡0.05, ns = not significant*

A.1.2 Correlation Stability Analysis

Table A.2 categorizes metrics by cross-CLD consistency, measured as standard deviation of correlation coefficients.

TABLE A.2: Metric Stability Classification

Stability Category	Metrics	Std Dev Range
High Stability (std ¡ 0.1)	Min Probability (0.038) Cosine Similarity (0.061) Judge Cosine Similarity (0.082) Max Window Entropy (0.092)	0.038 - 0.092
Medium Stability (0.1 $\leq$ std ¡ 0.15)	Judge Min Probability (0.100) Aggregate Score (0.104) Perplexity (0.137) Judge Max Window Entropy (0.140)	0.100 - 0.140
Low Stability (std $\geq$ 0.15)	Judge Perplexity (0.008)*	0.008

**Note: Judge Perplexity shows extremely low standard deviation (0.008) because its mean correlation is close to zero (-0.034), creating a ceiling effect. This indicates consistently weak performance rather than high stability.*

A.2 Classification Performance: Detailed Breakdown

A.2.1 AUC Statistics with Bootstrap Confidence Intervals

Table A.3 provides complete AUC statistics including 95% bootstrap confidence intervals.

TABLE A.3: Complete AUC Analysis: All CI Metrics

Metric	Mean AUC	95% CI	Std Dev	Range	p-value	Sig.
Perplexity	0.679	[0.650, 0.709]	0.076	[0.548, 0.747]	¡0.001	***
Max Window Entropy	0.667	[0.639, 0.695]	0.072	[0.542, 0.746]	¡0.001	***
Min Probability	0.641	[0.617, 0.664]	0.060	[0.542, 0.712]	¡0.001	***
Judge Perplexity	0.601	[0.524, 0.677]	0.161	[0.394, 0.800]	0.020	*
Judge Min Probability	0.575	[0.525, 0.624]	0.104	[0.396, 0.711]	0.009	**
Judge Max Window Entropy	0.559	[0.509, 0.609]	0.105	[0.378, 0.698]	0.035	*
Aggregate Score	0.431	[0.370, 0.493]	0.129	[0.237, 0.638]	0.043	*
Judge Cosine Similarity	0.265	[0.232, 0.298]	0.069	[0.156, 0.370]	¡0.001	***
Cosine Similarity	0.230	[0.214, 0.245]	0.040	[0.168, 0.289]	¡0.001	***

Note: p-values test whether AUC differs from 0.5 (chance) using one-sample t-test. All metrics significantly exceed chance performance.

A.3 Optimal Threshold Analysis

A.3.1 F1 Score at Optimal Decision Boundaries

Table A.4 reports mean F1 scores achieved at optimal thresholds for each metric, along with the variability of performance across CLDs.

TABLE A.4: F1 Score Performance at Optimal Thresholds

Metric	**Mean F1**	**95% CI**	**Std Dev**	**Range**
Perplexity	0.364	[0.315, 0.414]	0.127	[0.214, 0.486]
Max Window Entropy	0.356	[0.318, 0.394]	0.097	[0.208, 0.463]
Min Probability	0.350	[0.317, 0.382]	0.082	[0.210, 0.414]
Judge Min Probability	0.312	[0.269, 0.355]	0.091	[0.118, 0.384]
Judge Perplexity	0.310	[0.237, 0.382]	0.152	[0.105, 0.500]
Judge Max Window Entropy	0.269	[0.221, 0.317]	0.101	[0.125, 0.389]
Aggregate Score	0.222	[0.191, 0.254]	0.067	[0.143, 0.308]
Cosine Similarity	0.204	[0.180, 0.228]	0.060	[0.128, 0.273]
Judge Cosine Similarity	0.176	[0.143, 0.208]	0.069	[0.092, 0.263]

A.3.2 Interpretation of F1 Performance

The maximum F1 score (0.364 for perplexity) indicates:

- **Insufficient for automation:** F1 ¡ 0.5 means more than half of predictions (hallucinations or genuine edges) will be misclassified
- **Viable for triage:** Can reduce human review workload by 30-40% while maintaining reasonable precision
- **Precision-recall tradeoff:** Optimal thresholds balance false positives (wasted review) against false negatives (missed hallucinations)

A.4 Statistical Validation Summary

A.4.1 Hypothesis Testing Results

Table A.5 summarizes all statistical hypothesis tests performed in the meta-analysis.

Key finding: All 9 metrics demonstrate statistically significant classification performance above chance (p¡0.05), even though 2 metrics (Judge Min Probability, Aggregate Score) do not show significant linear correlation with hallucinations.

TABLE A.5: Statistical Hypothesis Tests: Summary

Test Type	**Null Hypothesis**	**Significant**	**Not Significant**	**Total**
Correlation (r 0)	r = 0	7 metrics	2 metrics	9
AUC (vs. chance)	AUC = 0.5	9 metrics	0 metrics	9

A.5 Per-CLD Performance Breakdown

While individual per-CLD results are not displayed here (to maintain focus on aggregate findings), the following observations hold across all three CLDs:

- **CLD 1 (Depressive symptoms):** N=182 edges, moderate hallucination rate
- **CLD 2 (Social norms and obesity):** N=90 edges, highest metric variance
- **CLD 3 (Emergency department visits):** N=1119 edges, largest dataset, lowest hallucination rate

Domain consistency: Despite differences in sample size and hallucination prevalence, the top 5 metrics (by AUC) maintain consistent ordering across all three CLDs, supporting cross-domain generalization within the health field.

A.6 Limitations of Statistical Validation

A.6.1 Sample Size Considerations

- **CLD count (N=3):** Limited statistical power for subgroup analyses
- **Run count (N=2 per CLD):** Minimal within-CLD replication
- **Total analyses (N=32):** Adequate for meta-analysis but not for robust domain-stratified modeling

A.6.2 Multiple Testing Concerns

With 9 metrics tested across multiple statistical procedures (correlation, AUC, F1), the family-wise error rate is elevated. However:

- Top metrics (perplexity, max window entropy) achieve p¡0.001, surviving Bonferroni correction (= 0.05/9 = 0.0056)

- Weaker metrics (judge max window entropy, aggregate score) would not survive strict correction

Recommendation: Interpret borderline-significant metrics (p¿0.01) with caution pending independent validation.

A.7 Computational Efficiency Notes

CI metrics were computed retrospectively on completed experiment outputs. For deployment considerations:

- **Fast metrics:** Perplexity, min probability, max window entropy (computed during generation, zero latency penalty)
- **Moderate metrics:** Cosine similarity (requires embedding API calls, 100-500ms per edge)
- **Slow metrics:** Judge-based metrics (require full judge LLM call, 2-5 seconds per edge)

Practical implication: For real-time filtering, generator-based uncertainty metrics offer the best speed-accuracy tradeoff.

Bibliography

[] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In *International Conference on Learning Representations*, 2015. URL https://arxiv.org/abs/1409.0473.

[] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in neural information processing systems*, pages 5998–6008, 2017.

[] Arielle R. Deutsch, Leah Frerichs, Madeleine Perry, and Mohammad S. Jalali. Participatory modeling for high complexity, multi-system issues: challenges and recommendations for balancing qualitative understanding and quantitative questions. *System Dynamics Review*, 40(4):e1765, 2024. doi: https://doi.org/10.1002/sdr.1765. URL https://onlinelibrary.wiley.com/doi/abs/10.1002/sdr.1765.

[] Ning-Yuan Georgia Liu and David Keith. Leveraging large language models for automated causal loop diagram generation: Enhancing system dynamics modeling through curated prompting techniques. *SSRN Electronic Journal*, 2024. doi: 10.2139/ssrn.4906094. URL https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4906094.

[] M. Waaijers, H. Rosenbusch, C. J. van Lissa, A. Roefs, and D. Borsboom. theoraizer: Ai-assisted theory construction. *PsyArXiv Preprints*, 2024. URL https://osf.io/preprints/psyarxiv/gu9yq_v1. Preprint.

[] Niyousha Hosseinichimeh, Aritra Majumdar, Ross Williams, and Navid Ghaffarzadegan. From text to map: A system dynamics bot for constructing causal loop diagrams, 2024. URL https://arxiv.org/abs/2402.11400.

[] Haochen Zhao, Hang Chen, Fan Yang, Ninghao Liu, Hang Deng, Hao Cai, Shuai Wang, Dong Yin, and Min Du. Explainability for large language models: A survey. *arXiv preprint arXiv:2309.01029*, 2023. URL https://arxiv.org/abs/2309.01029.

[] Lianmin Zheng, Wei-Lin Chiang, Yu Sheng, Shaohan Zhuang, Zhentao Wu, Yongkai Zhuang, Zhepei Lin, Zhuohan Li, Da Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *arXiv preprint arXiv:2306.05685*, 2023. URL https://arxiv.org/abs/2306.05685.

[] Neeraj Varshney, Wenlin Yao, Hongming Zhang, Jianshu Chen, and Dong Yu. A stitch in time saves nine: Detecting and mitigating hallucinations of llms by validating low-confidence generation. *arXiv preprint arXiv:2307.03987*, 2023.

[] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35: 24824–24837, 2022.

[] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

[] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 2020. URL https://arxiv.org/abs/2005.11401.

[] SM Tonmoy, SM Zaman, Vinija Jain, Anku Rani, Vipula Rawte, Aman Chadha, and Amitava Das. A comprehensive survey of hallucination mitigation techniques in large language models. *arXiv preprint arXiv:2401.01313*, 2024.

[] Yuzhe Zhang, Yipeng Zhang, Yidong Gan, Lina Yao, and Chen Wang. Causal graph discovery with retrieval-augmented generation based large language models. *arXiv preprint arXiv:2402.15301*, 2024. URL https://arxiv.org/abs/2402.15301.

[] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2024.

[] OpenAI. Learning to reason with llms: Introducing openai o1, a new large language model trained with reinforcement learning for complex reasoning. https://openai.com/index/learning-to-reason-with-llms/, 2024.

[] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[] Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, and Yarin Gal. Detecting hallucinations in large language models using semantic entropy. *Nature*, 630(8017): 625–630, 2024.

[] J. Marin. Optimizing AI reasoning: A hamiltonian dynamics approach to multi-hop question answering. *arXiv preprint arXiv:2410.04415*, 2024. URL https://arxiv.org/abs/2410.04415.

[] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.

[] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

[] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

[] T. Wang, X. Jiao, Y. He, Z. Chen, Y. Zhu, X. Chu, J. Gao, Y. Wang, and L. Ma. Adaptive activation steering: A tuning-free llm truthfulness improvement method for diverse hallucinations categories. *arXiv preprint*, 2024. URL https://arxiv.org/html/2406.00034v1.

[] Charles H. Martin, Tianshu Peng, and Michael W. Mahoney. Predicting trends in the quality of state-of-the-art neural networks without access to training or testing data. *Nature Communications*, 12:4122, 2021. doi: 10.1038/s41467-021-24025-8. URL https://doi.org/10.1038/s41467-021-24025-8.

[] Charles H. Martin and Michael W. Mahoney. Traditional and heavy-tailed self-regularization in neural network models. *arXiv preprint arXiv:1901.08276*, 2019. URL https://arxiv.org/abs/1901.08276.

[] Yizhen Zheng, Huan Yee Koh, Jiaxin Ju, Anh T. N. Nguyen, Lauren T. May, Geoffrey I. Webb, and Shirui Pan. Large language models for scientific synthesis, inference and explanation. *arXiv preprint arXiv:2310.07984*, 2023. URL https://arxiv.org/abs/2310.07984.

[] Shangbin Feng, Weijia Shi, Yuyang Bai, Vidhisha Balachandran, Tianxing He, and Yulia Tsvetkov. Knowledge card: Filling llms' knowledge gaps with plug-in specialized language models. 2023. URL https://arxiv.org/abs/2305.09955.

[] Biqing Qi, Kaiyan Zhang, Haoxiang Li, Kai Tian, Sihang Zeng, Zhang-Ren Chen, and Bowen Zhou. Large language models are zero shot hypothesis proposers. *arXiv preprint arXiv:2311.05965*, 2023. URL https://arxiv.org/abs/2311.05965.

[] C. Lu, C. Lu, R. T. Lange, J. Foerster, J. Clune, and D. Ha. The ai scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint*, arXiv:2408.06292, 2024.

[] Xingbo Wang, Samantha L. Huey, Rui Sheng, Saurabh Mehta, and Fei Wang. Scidasynth: Interactive structured knowledge extraction and synthesis from scientific literature with large language model. *arXiv preprint arXiv:2404.13765*, 2024. URL https://arxiv.org/abs/2404.13765.

[] Teo Susnjak, Peter Hwang, Napoleon H. Reyes, Andre L. C. Barczak, Timothy R. McIntosh, and Surangika Ranathunga. Automating research synthesis with domain-specific large language model fine-tuning. 2024. URL https://arxiv.org/abs/2404.08680.

[] SURF. Snellius supercomputer documentation, 2024. URL https://visualization.surf.nl/snellius-virtual-tour/. High-performance computing resource for the Netherlands.

[] Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. s1: Simple test-time scaling, 2025. URL https://arxiv.org/abs/2501.19393.

[] Haitao Li, Qian Dong, Junjie Chen, Huixue Su, Yujia Zhou, Qingyao Ai, Ziyi Ye, and Yiqun Liu. Llms-as-judges: A comprehensive survey on llm-based evaluation methods. *arXiv preprint arXiv:2412.05579*, 2024.

[] Jiawen Shi, Zenghui Yuan, Yinuo Liu, Yue Huang, Pan Zhou, Lichao Sun, and Neil Zhenqiang Gong. Optimization-based prompt injection attack to llm-as-a-judge. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 660–674, 2024.

[] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023.

[] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models, 2023. URL https://arxiv.org/abs/2203.11171.

[] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 2020.

[] Jacob Cohen. *Statistical Power Analysis for the Behavioral Sciences*. Routledge, Hillsdale, NJ, 1988.